

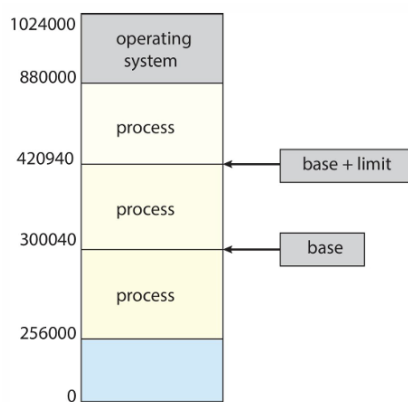
PROGRAMMAZIONE DI SISTEMA: OS-INTERNAL

1 GESTIONE DELLA MEMORIA

1.1 Background

Nella gestione della memoria e' importante capire come gestire lo spazio di memoria da allocare per ogni processo. Alla base vi e' quindi lo **spostamento di un programma da disco a RAM** in modo che sia **accessibile dalla CPU**.

1.2 Protezione



Bisogna quindi far si che un processo non acceda agli indirizzi di memoria allocati per un altro processo. Questo e' **importante** per la **programmazione concorrente**. Un modo semplice per implementare questa protezione e' usare:

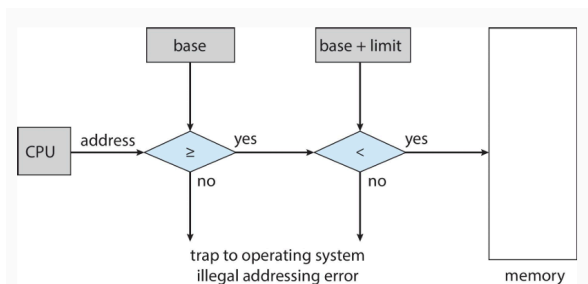
- **Base register** $\Rightarrow$ Indirizzo di **partenza** della RAM.
- **Limit register** $\Rightarrow$ Spazio del **range** allocato per il processo.

Questo viene fatto per ogni processo

(vedi immagine).

Quindi la **CPU controlla se l'indirizzo a cui il processo vuole accedere fa parte di questo spazio di indirizzamento allocato per il processo stesso**.

Essa controlla se indirizzo e' dentro il range (base + limit).



1.3 Address Binding

Il **binding** e' il processo che **identifica l'indirizzo di memoria** a quale il **programma** (o processo in esecuzione) e' stato assegnato. La vita di un processo e' composta da:

- Compile time, codice convertito in ling. macchina.
- Load time, passaggio da disco a RAM.

- Execution time, CPU genera indirizzo simbolico che viene convertito da MMU in indirizzo effettivo.

L'**address binding** puo' essere fatto in tutti questi momenti, noi ci focalizziamo sull'**execution time**.

1.4 Dynamic Loading e Linking

Invece di caricare il processo nella sua interezza subito, si puo' avere un **dynamic loading**, dove **solo le parti del processo che servono al momento vengono caricate in memoria** (si vedra' in memoria virtuale). Questo permette di evitare di caricare la stessa libreria (che serve a due processi magari) due volte in memoria (**Dynamic Linked Libraries DLL**).

Il linking delle librerie richiede **accesso concorrente** tra i processi.

Quindi:

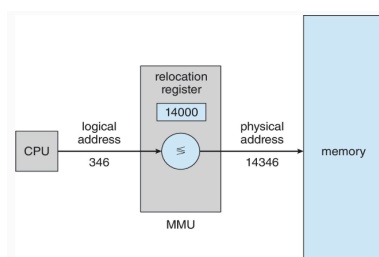
- **Dynamic Loading** ⇒ Supponiamo di avere un unico programma con diverse routine all'interno. Se non tutte le routine vengono utilizzate, ha senso caricarle tutte in memoria? Questo è il concetto di Dynamic Loading: **si caricano le parti del programma necessarie per la sua esecuzione** (quindi quelle che servono in un secondo momento vengono caricate quando necessario). In questo modo si ha un utilizzo ottimale della memoria. Per attuare questa strategia non serve un supporto speciale del sistema operativo, ma è invece necessario che le varie parti di programma siano rilocabili.
- **Dynamic Linking** ⇒ Normalmente, le varie librerie e i codici compilati vengono fusi nella cosiddetta loadable image da parte del linker. Questo è quello che prende il nome di static linking: il pro è che si rivela sufficiente essere in possesso dell'eseguibile per far eseguire il programma. Nel Dynamic Linking, invece, **si risolvono i riferimenti durante l'esecuzione stessa**: opportunamente, le varie funzioni di librerie linkate a run-time vengono sostituite da parti di codice, chiamati stub, che servono a richiamare il caricamento di quella particolare funzione di libreria. Questo sistema torna molto utile nel contesto delle librerie condivise.

Quindi il linking puo' essere effettuato staticamente o dinamicamente e risulta essere indipendente dal caricamento dinamico o meno. Ne risulta che **e' possibile caricare un programma dinamicamente senza dynamic linking**.

Inoltre e' anche possibile linkare dinamicamente un programma (libreria condivisa gia' caricata) senza caricamento dinamico.

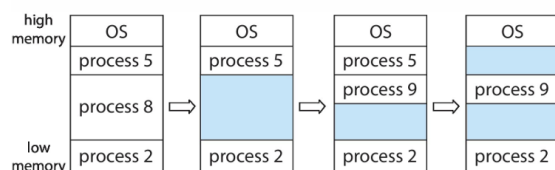
1.5 Indirizzi fisici e logici

La **CPU** durante l'esecuzione delle istruzioni di un processo **crea** quindi indirizzi simbolici (**INDIRIZZO LOGICO o INDIRIZZO VIRTUALE**). Questi indirizzi devono essere **tradotti in INDIRIZZI FISICI** per riferire alla RAM. Questo mapping viene effettuato dalla componente hardware chiamata **MMU (Memory Management Unit)**.



Considerando la strategia con [base + limit] si cambia il base. Il **base viene quindi sommato al reallocation register**. In particolare vi e' un reallocation register unico per ogni processo.

1.6 Allocazione contigua



Una parte della memoria e' allocata staticamente per il SO. La parte restante viene usata per allocare i processi. Essi possono essere allocati in maniera **contigua**. Questo significa che la **parte di memoria allocata per ciascun processo deve essere in posizioni contigue della memoria**. Se quindi non c'e' spazio contiguo disponibile un processo non puo' essere allocato. Per risolvere queste limitazioni si usano vari metodi.

1.7 Partizioni variabili

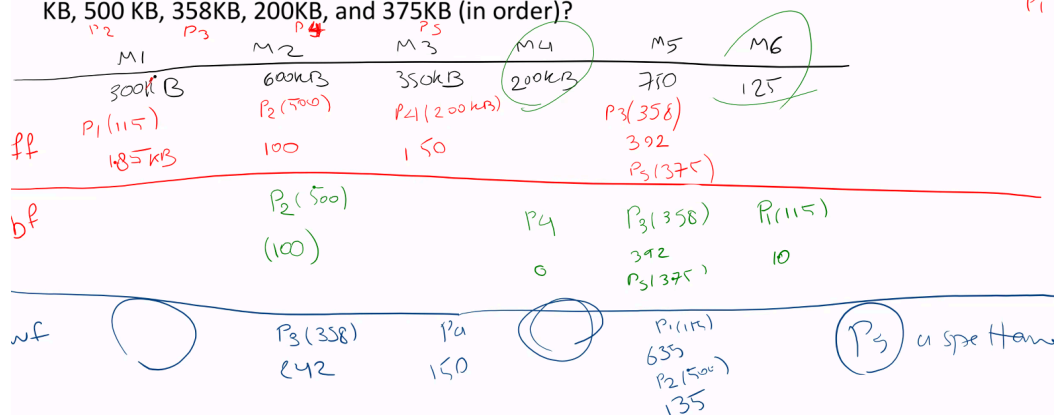
La **memoria** viene **suddivisa in varie partizioni con lunghezza fissa**. Si basa **sempre su memoria contigua**, se un processo occupa piu' partizioni esso deve essere posto in due partizioni consecutive.

1.8 Problemi partizioni

Per **gestire** l'allocazione di processi nei **buchi della memoria** ci sono vari metodi:

- **First-fit** $\Rightarrow$ Alloco il primo blocco abbastanza grande.
- **Best-fit** $\Rightarrow$ Alloco il blocco piu' piccolo abbastanza grande. Migliore efficienza nello storage, ma piu' lento in quanto richiede ricerca.
- **Worst-fit** $\Rightarrow$ Alloco il blocco piu' grande. Il vantaggio potrebbe essere che allocando sempre la partizione più grande disponibile, tende a lasciare partizioni residue sufficientemente grandi da essere riutilizzate successivamente.

• Given six memory partitions of 300 KB, 600 KB, 350 KB, 200 KB, 750 KB, and 125 KB (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of size 115 KB, 500 KB, 358 KB, 200 KB, and 375 KB (in order)?



P5 deve aspettare in quanto non vi e' un blocco contiguo abbastanza grande con worst fit.

1.9 Frammentazione

Come i processi vengono inseriti e tolti dalla memoria si vanno quindi a creare vari buchi. Si va quindi a creare uno stato di **EXTERNAL FRAGMENTATION**, ovvero **si ha spazio sufficiente per altri processi ma tale spazio non e' contiguo**.

Lo stato di **INTERNAL FRAGMENTATION** invece si ha quando un **processo crea un buco in una partizione**.

La frammentazione non permette di sfruttare il 100% della memoria, ci sono varie soluzioni:

- **Compaction**, ogni tot cicli la CPU si dedica a spostare i processi in modo da riallocare lo spazio ed eliminare frammentazione (buchi). In questo modo la CPU spreca tempo ed e' anche necessario aggiornare base, limit e realocation register per ogni processo (nel process control block).

Altre soluzioni riguardano il **paging**.

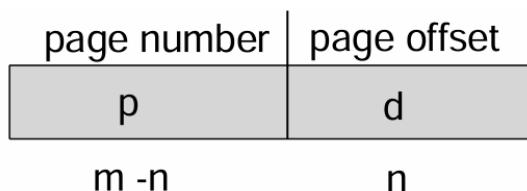
1.10 Paging

Altra **soluzione** alla **frammentazione esterna** che permette di avere spazio di indirizzamento logico non-contiguo. La strategia che si adotta e' il **PAGING**.

Si divide la memoria fisica in FRAMES di spazio statico da 512 bytes a 16Mbytes **e si divide la memoria logica (processo) in blocchi della stessa dimensione chiamati PAGINE**. Ora l'MMU deve mappare ogni pagina in un frame della memoria.

Gli **indirizzi** della CPU sono quindi divisi in:

- **Page number**, indice della page table (numero della pagina relativa al processo).
- **Page offset**, offset che indica la riga della pagina identificata da page number per trovare l'istruzione.

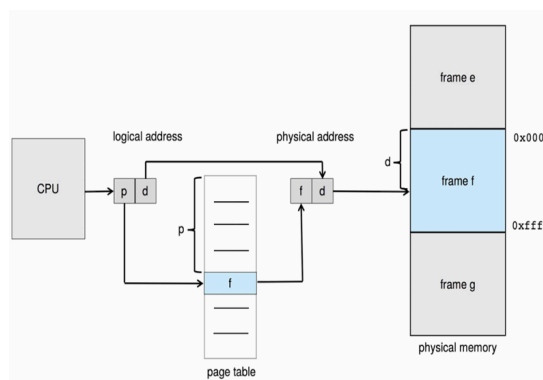


Spazio di indirizzi di 2^m e page size di 2^n .

Page number e' quindi l'indirizzo di memoria della riga della page table dove si trova l'indirizzo del frame e il page offset e l'altra parte di indirizzo che identifica la riga/istruzione.

MMU quindi converte page number in frame number, **l'offset invece rimane lo stesso in quanto la dimensione tra pagine e frame e' la stessa.**

1.11 Paging hardware



Ogni processo ha la sua **page table** che viene usata per mappare indirizzi da logici a fisici. Il **numero di pagina** viene usato come **indice della page table**, nella quale e' presente il frame number usato per accedere al frame corrispondente della memoria fisica.

In questo modo **si risolve quindi il problema dell'external fragmentation**, ma l'**internal fragmentation rimane**. Un processo o parte di esso puo' essere ad esempio piu' piccolo della dimensione delle pagine/frame. **Minore e' la dimensione delle pagine, minore sara' l'internal**

fragmentation, pero' in questo modo la **dimensione della page table aumenta** (per ogni processo, e queste tabelle sono memorizzate in memoria).

1.12 Implementazione della Page Table

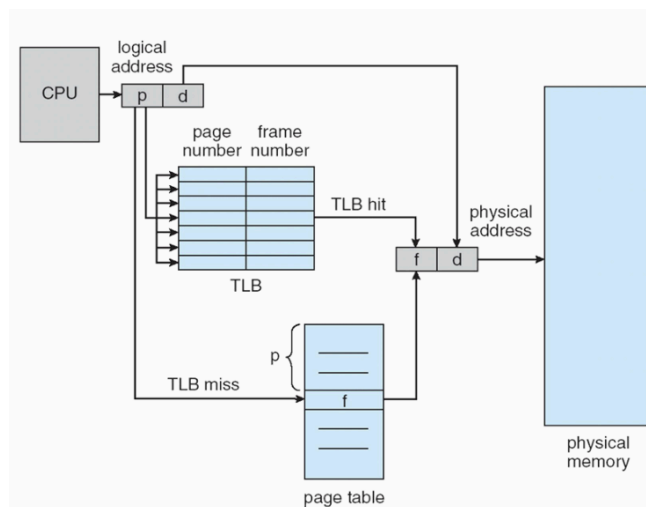
Per evitare di allocare uno spazio di memoria contiguo per salvare le page table ci sono vari modi.

Innanzitutto, ci sono **due registri nel PCB (Process Control Block)** :

- **PTBR (Page Table Base Register)**, punta all'indirizzo di memoria della tabella.
- **PTLR (Page Table Length Register)**, indica la dimensione della page table.

Questo per ogni processo. Il **problema** della page table e' che ogni **processo deve fare sempre due accessi a memoria**, uno sulla page table e l'altro per leggere l'istruzione effettiva. Per questo viene aggiunto il **TLB (Translation look-aside Buffers)**.

1.12.1 TLBs (Translation look-aside buffers)



Il **TLB** e' caratterizzato da poche entry contenenti **[page number e frame number]** dei vari processi e funge da **cache** per **evitare** un **doppio accesso a memoria** (ovvero alla page table) per prelevare un'istruzione. Il TLB ha entry limitate, quindi se c'e' un **TLB miss** bisogna comunque accedere alla page table in memoria (doppio

accesso). Le TLB sono meccanismi hardware, quindi si usa un **ASIDs (address-space identifiers)** in ciascuna entry per identificare il processo associato alla pagina nella entry i-esima (si usa il **PID**). In questo modo e' possibile **dividere il TLB tra piu' processi**.

La percentuale delle volte che la pagina di interesse viene trovata nel TLB si chiama **hit ratio**. L'**Effective Access Time** e' quindi:

$$EAT = hit_ratio * MM + (1 - hit_ratio) * 2 * MM$$

Dove **MM** e' il tempo di accesso a memoria, se ho TLB miss (1-hit_ratio), si fa 2*MM in quanto devo appunto accedere 2 volte a memoria.

Posso anche avere **TLB** che fungono da cache **per un solo processo** e non contengono quindi un campo per il PID. In questo caso e' necessario **azzerare la TLB ad ogni context switching**.

In **caso di cache** si ha:

$$EAT_cache = CACHE_HR * CACHE_ACCESS_TIME * (1 - CACHE_HR) * (MM + CACHE_ACCESS_TIME).$$

Se si vuole combinare alla presenza di una TLB, considerando N = Livelli di page table + 1:

$$EAT = (TLB_HR + N * (1 - TLB_HR)) * EAT_cache.$$

1.12.2 Memory Protection

frame number		valid-invalid bit
0	2	v
1	3	v
2	4	v
3	7	v
4	8	v
5	9	v
6	0	i
7	0	i

page table

Per ogni entry della page di table si puo' aggiungere un **protection bit**. Questo bit viene utilizzato per specificare il **tipo di operazione** (solo lettura o scrittura) che posso fare sul frame associato. Un ulteriore informazione che posso aggiungere e' un **valid-invalid bit**, che indica se la **pagina e' contenuta o meno nella memoria logica** del processo (indica se quindi l'operazione e' legale o meno).

[Nel calcolo della dimensione della pagina, se presente, bisogna anche considerare la dimensione di questo bit per ogni riga].

Si nota quindi come, nel caso di **standard PT**, ogni processo ha una **page table della medesima dimensione** (anche se non la usa tutta) e le **entry non usate avranno quindi valid bit a 0**.

1.12.3 Shared Pages

Un vantaggio della paginazione e' che posso condividere **codice reentrant** (ovvero solo lettura) comune tra processi. Quindi nella **page table di ogni processo** che fa riferimento alla stessa libreria si **mappa lo stesso numero di frame** (dove e'

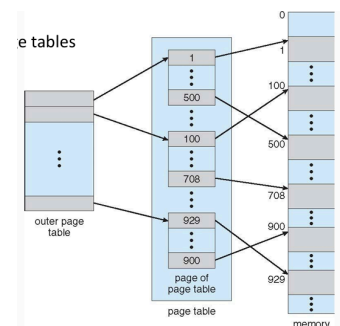
memorizzata la libreria). Questo per evitare di allocare nella memoria fisica 3 frame uguali.

1.12.4 Tipi di page table

Nei sistemi reali le page table sono di grandi dimensioni, quindi per **evitare di allocarle in uno spazio di memoria contiguo** posso **dividere la page table in unita' piu' piccole**. (es. indirizzi a 32 bit e ogni page table da 4kb $\rightarrow 32 - 12 = 20$ bit per indirizzare il numero di pagine, quindi 2 alla 20 pagine).

1.12.4.1 Page Tables Gerarchiche

Meccanismo dove si hanno piu' **livelli di page table**. Nel caso di due livelli, si ha una **page table che ha varie entry**. Ognuna di queste entry contiene un **puntatore alla page table effettiva che contiene la mappatura tra page number e frame number**. Il frame e' quindi scritto nelle entry di queste **inner page table**. In questo modo gli indirizzi logici cambiano:



A logical address (on 32-bit machine with 4K page size) is divided into:

- a page number consisting of 20 bits
- a page offset consisting of 12 bits

Since the page table is paged, the page number is further divided into:

- a 10-bit page number
- a 10-bit page offset

Thus, a logical address is as follows:

page number		page offset
p_1	p_2	d
10	10	12

I primi 10 bit (p_1) identificano le entry della outer page table, mentre i secondi 10 bit (p_2) identificano la riga della inner table associata. L'offset indica sempre la riga del frame della memoria fisica.

Con questo schema avro' massimo 2^{20} frame in memoria. Se non ho bisogno di 2^{20} , potrei aver bisogno di meno page tables, es. se ho un processo di 100mb ha bisogno di meno page tables (questa e' la dimensione effettiva).

If page size is 4 KB (2^{12})

- Then page table has 2^{52} entries
- If two level scheme, inner page tables could be 2^{10} 4-byte entries
- Address would look like

outer page	inner page	offset
p_1	p_2	d
42	10	12

- Outer page table has 2^{42} entries or 2^{44} bytes
- One solution is to add a 2nd outer page table
- But in the following example the 2nd outer page table is still 2^{34} bytes in size
 - And possibly 4 memory access to get to one physical memory location

1.12.4.2 Hashed page tables

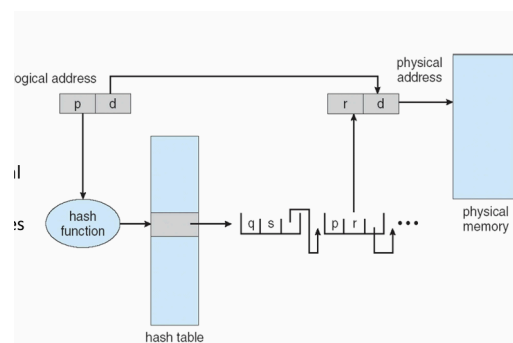
Il valore di **page number** viene **usato come valore in funzione hash**.

La funzione da come **output una lista linkata** che contiene:

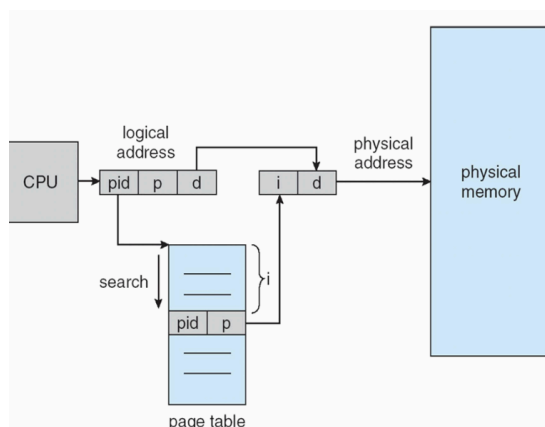
- Numero di pagina.
- Numero di frame.
- Puntatore a prossimo elemento.

Se **c'è un match** tra il numero di **pagina in input** allora il frame **corrispondente è quello giusto**.

Questo provoca **molti accessi alla memoria se non si trova subito il match** (in quanto bisogna scorrere la lista), ma le informazioni sono **ancora meno in posizioni contigue**.



1.12.4.3 Inverted Page Table (IPT)



In questo caso si ha una situazione inversa. Invece di avere per una pagina il frame associato, **si mappa ogni frame ad ogni pagina**. Questo comporta che si abbia **una sola tabella invece di una page table per ogni processo**. Si usa il **pid per identificare** a quale processo quella pagina si riferisce. La ricerca è più lenta, quindi si

Si puo' **accoppiare una IPT con una tabella hash**. In questo modo si evita di effettuare una ricerca lineare del numero di pagina, in quanto vengono inserite direttamente nelle liste di chaining della tabella di HASH le entry della IPT.



- La chiave della tabella HASH e' la coppia (PID, page number), quindi metto in input alla hash table la coppia (PID, n) di interesse.
- La funzione hash ritorna l'indice dell'entry della IPT corrispondente (che corrisponde appunto al frame associato al numero di pagina).
- In alternativa, se ho collisione, quindi la stessa chiave mi da coppie diverse, scorro le coppie fino a quando il valore dell'input non coincide con una di queste coppie.

Differiscono dalla coppia hash table + page table standard, in quanto nella **page table standard** lo stesso frame fisico può essere mappato da pagine virtuali diverse in momenti diversi (allocazione/deallocazione), mentre nella **Inverted Page Table** ogni frame fisico ha al massimo una sola mappatura attiva.

Quindi nella PT standard l'hash page table viene usata per diminuirne la dimensione (in modo che sia dinamica e non abbia invalid bit derivanti da non appartenenza allo spazio di indirizzamento). Mentre con la IPT per migliorarne i tempi di accesso.

Riepilogo comparativo

Tipo	Accesso	Dimensione	Dinamico?	Note principali
Page Table Standard	Diretto	Fissa	No	Entry anche se non usate
Page Table con Hash	Diretto + lista	Dinamica	Sì (entry create solo se usate)	Nessuna entry invalida
Inverted Page Table (IPT)	Lineare	Fissa	No	Una entry per frame, con PID
IPT con Hash	Diretto + lista	Fissa	No (niente nuove entry)	Entry già esistenti collegate

ESERCIZI

- Consider a computer system with a 21-bit virtual address that has only a 16-bit physical address. It also has a 2KB page size. How many entries are there in each of the following?
- A conventional, single-level page table
- An inverted page table

Logical address di 21 bit.
Physical di 16 bit
page size di 2kb.

Standard page table:
2kb = 11 bit di offset
e quindi 10 bit (21-11) = 10 bit di numero di pagine
La page table ha quindi 2^{10} entry.

Inverted page table:
Ho sempre un offset di 11 bit.
Quindi mi rimangono 16-11 = 5 bit per i frame.
Quindi avrò massimo 2^5 entry nella page table.

- Consider a computer system with a 32-bit logical address and 4-KB page size. The system supports up to 512 MB of physical memory. How many entries are there in each of the following?
- A conventional, single-level page table
- An inverted page table

logical address = 32 bit
physical memory = 512mb

Standard:
4kb = 12 bit
quindi page number = $32-12 = 20 \rightarrow 2^{20}$ entry
offset 12 bit

Inverted page table
 $512\text{mb}/4\text{kb} = 128\text{k}$ (dimensione memoria/dimensione frame) ci dà il numero di frame
per puntare a 128k numero di frame (quindi ci sono 128.000 frames)
Il numero di bit per puntare a questi frame $\rightarrow \log_2(128.000)$

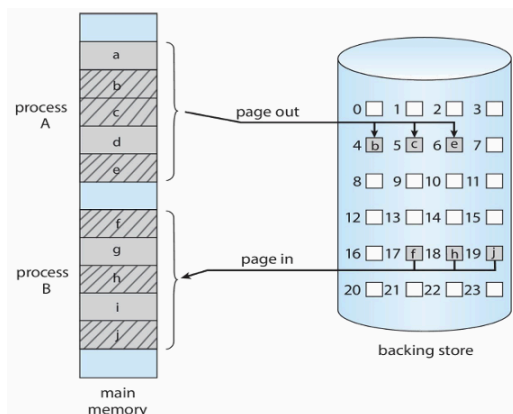
1.13 Swapping

Processo nel quale si toglie un processo dalla memoria (swap out) per fare spazio ad un nuovo processo (swap in). Tradizionalmente dovrei togliere tutte le pagine del processo 1 dalla memoria, questo richiede molto tempo per il context switching. Nei **sistemi moderni** si usa il concetto di:

- Page in e Page out.

Invece di togliere l'intero processo si tolgono solo alcune pagine.

1.13.1 Swapping con Paging



Invece di togliere l'intero processo quindi si tolgono solo alcune pagine (**page out**) e si **inseriscono alcune pagine del nuovo processo (page in)**.

2 OS161

2.0 Nuove opzioni kernel e nuove configurazioni

- ♦ 1. Definire una nuova opzione nel kernel

Per attivare funzionalità opzionali nel kernel (es. una syscall in `project.c`), apri il file:

File da modificare:

`kern/conf/conf.kern`

Aggiungi le seguenti righe:

```
bash
CopiaModifica
defoption project
optfile project syscall/project.c
```

Cosa fa questa modifica:

- `defoption project`: definisce un'opzione chiamata `project`.
- `optfile`: indica al sistema di compilare `syscall/project.c` solo se l'opzione `project` è attiva.

Questa modifica genera automaticamente il file `opt-project.h` durante la configurazione (vedi passo 4).

📌 Anche se l'opzione non è attiva, il file `opt-project.h` viene comunque creato e conterrà:

```
cpp
```

```
CopiaModifica
```

```
#define OPT_PROJECT 0    // oppure 1 se attivata nel file  
PROJECT
```

♦ 2. Creare una nuova configurazione del kernel

Copia una configurazione esistente (es. `DUMBVM`) e rinominala:

```
bash
```

```
CopiaModifica
```

```
cp kern/conf/DUMBVM kern/conf/PROJECT
```

Modifica il nuovo file `PROJECT` e attiva esplicitamente l'opzione:

```
nginx
```

```
CopiaModifica
```

```
options project
```

⚠ Se ometti questa riga, anche se hai fatto tutto correttamente prima, il file `project.c` non verrà compilato.

✅ Risposta alla domanda A: **NO**, non basta modificare `conf.kern` e fare `./config` – bisogna anche scrivere `options project` nel file `PROJECT`.

♦ 3. Configurare il kernel con la nuova versione

Vai nella directory delle configurazioni e genera i file necessari:

```
bash
CopiaModifica
cd kern/conf
./config PROJECT
```

Questo comando crea:

- La directory `kern/compile/PROJECT/`
- Il file `opt-project.h` con `#define OPT_PROJECT 1` se l'opzione è attiva

✓ Risposte B e C: viene creato `opt-project.h`, con una macro `OPT_PROJECT` definita a 1 o 0.

♦ 4. Compilare il nuovo kernel

Spostati nella directory della configurazione:

```
bash
CopiaModifica
cd kern/compile/PROJECT
bmake depend
bmake
bmake install
```

- `bmake depend`: genera le dipendenze
 - `bmake`: compila il kernel
 - `bmake install`: copia il kernel in `os161/root/kernel`
-

♦ 5. Eseguire il kernel

```
bash
CopiaModifica
```

```
cd root
sys161 kernel
```

- ♦ 6. Usare codice condizionato in base all'opzione

Quando usi codice opzionale (es. `project_init()`), proteggilo con una direttiva condizionale:

```
c
CopiaModifica
#include "opt-project.h"

#if OPT_PROJECT
project_init();
#endif
```

Vantaggi:

- Il codice viene compilato solo se l'opzione è attiva.
- Eviti errori se `project.c` non è incluso.

✅ Risposta D: usa `#if OPT_PROJECT` e includi `opt-project.h`.

Le domande erano:

È dato un Sistema OS161. Si supponga di aver aggiunto le istruzioni seguenti a `kern/conf/conf.kern`

```
defoption project
optfile project syscall/project.c
```

e di aver creato il file `PROJECT` in `kern/conf`, copiato dal file `DUMBVM`.

A) Si dica se le azioni descritte nel seguito, più l'esecuzione (in `kern/conf`) di `./config PROJECT`, sono sufficienti affinché il file opzionale `syscall/project.c` sia compilato quando si eseguono (in `kern/compile/PROJECT`) i comandi

```
bmake depend
bmake
```

B) Quale file, tra `project.h` e `opt-project.h` viene generato automaticamente dal comando `./config PROJECT`? Il file viene sempre generato, oppure solo se l'istruzione seguente compare nel file `PROJECT`?

```
options project
```

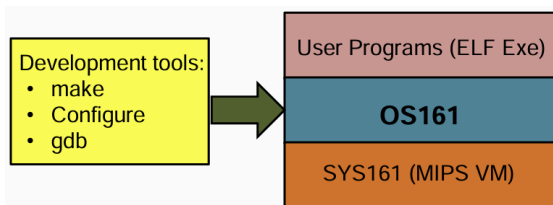
C) Cosa contiene il file (quello generato automaticamente, citato nella domanda precedente)?

D) Si supponga di inserire in `main.c` l'istruzione

```
project_init();
```

Considerando che la funzione `project_init()` è implementata nel file `syscall/project.c`, come si può fare in modo che l'istruzione sia considerata e compilata solo nelle versioni del kernel in cui l'opzione `project` è abilitata?

2.1 Introduzione ambiente



OS161 è un **sistema operativo didattico** basato su unix scritto in C. Questo sistema si basa su un **processore MIPS32**, quindi per farlo girare su altre architetture si usa un

simulatore (**SYS161**) per emulare il processore e poter eseguire il S.O.

- `os161/os161-base-2.0.2/kern`: kernel source
- `os161/os161-base-2.0.2/kern/conf`: kernel configuration
- `os161/os161-base-2.0.2/kern/compile`: kernel compilation/build

Queste tre sono le directory maggiormente utilizzate inizialmente.

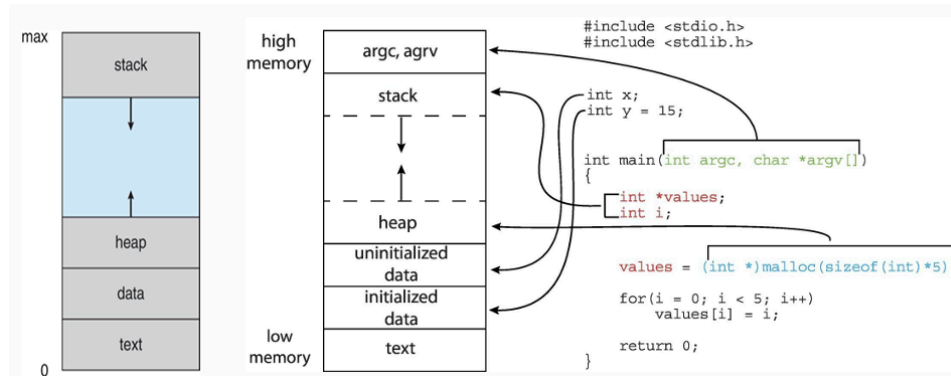
2.2 Kernel Threads

Un thread o processo è un programma in esecuzione. Ogni processo ha un contesto (stato, ovvero ready, execute ec...). Inoltre ogni **processo** ha una **memoria** associata che comprende:

- **Program code** (codice).
- **Program data** (es. variabili globali).
- **Program heap**, per l'allocazione dinamica.

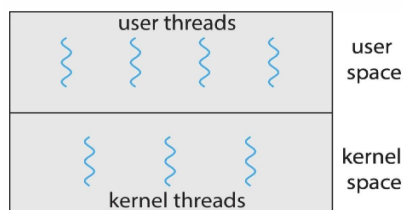
- **Program stack**, per dati temporanei.

Stack ed heap non hanno una dimensione fissa.



Ci sono due tipi di threads:

- **User threads**: Gestiti dalla libreria utente.
- **Kernel Threads**: Supportati direttamente dal kernel.

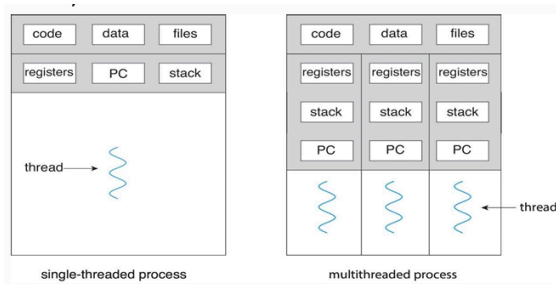


Gli **user threads** per essere eseguiti vengono **mappati ad un kernel thread**. La differenza tra i due e' semplicemente che i primi vengono gestiti dall'utente e sono piu' leggeri, mentre gli altri sono creati dal S.O.

Ci sono tre modi per **mappare user threads in kernel threads**:

- **One-to-One**: Mappa un **user-thread** ad un unico **kernel-thread**. Il problema e' che il numero di kernel thread che possono essere creati e' limitato.
- **Many-to-One**: Mappa **tanti user-thread** ad un unico **kernel-thread**. Questo significa che multipli threads **non** possono essere eseguiti in **parallelo**.
- **Many-to-Many**: Mappa **N user threads** a **M kernel threads** dove **N > M**. Questo garantisce parallelismo e al contempo si possono creare i threads che si vogliono.

2.3 Processi a singolo e multipli threads

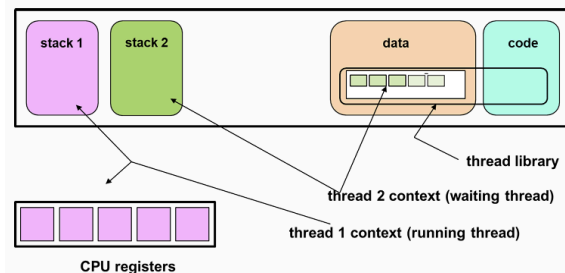


Se si ha un processo con piu' threads, **ogni thread condivide la parte code, data e files, ma ognuno ha PC, stack e registri**. Questo per permettere il context switching tra threads, che viene fatto tramite la libreria POSIX.

2.4 Thread context in OS161

OS161 gestisce **kernel threads** tramite una libreria.

Thread Control Block (TCB) ⇒ Struttura dati usata per memorizzare il contesto di un thread.



Se si hanno **piu' thread associati ad un processo** la memoria viene gestita in maniera diversa:

- Si hanno **due stack (uno per thread)**.

- Data e codice sono

condivisi.

- Se uno dei due thread e' attivo e si vuole fare context switching, **salvo il contesto del thread attivo nel suo stack e sostituisco i registri nella CPU con il contesto del nuovo.**

Queste sono le funzioni per gestire threads in OS161:

```
Interface for bootstrap/shutdown (or panic)
thread_bootstrap
thread_start_cpus
thread_panic
thread_shutdown

Interface for thread handling
thread_fork
thread_exit
thread_yield
thread_consider_migration

Internal functions
thread_create
thread_destroy
thread_make_runnable
thread_switch
```

2.5 Codice e SWITCHFRAME

La **struct "thread"** rappresenta il **TCB**. Questa struttura contiene informazioni come il **nome** del thread e lo **stato** di quest'ultimo (wait, sleeping, running). Il **TCB** ha inoltre un **puntatore allo stack** della memoria e alla zona dello stack chiamata **switchframe** dove **vengono salvati i registri per il context switching**. Quindi il contesto di un thread kernel fa riferimento al contenuto dello switchframe **memorizzato nello stack di kernel** (data e codice sono globali e condivisi tra i vari thread di un processo, quindi non fanno parte strettamente del contesto di un thread).

Alcune delle funzioni principali sono:

- **thread_fork()** ⇒ Usata per creare un nuovo thread, e viene quindi creata una nuova area di memoria per esso. La funzione ritorna un puntatore a quest'area. Ha un parametro chiamato "proc". Se "proc" e' null allora il nuovo thread appartiene (padre) a quello che lo ha

creato, altrimenti appartiene al processo passato come "proc" appunto.

- **thread_exit()** ⇒ Usata per terminare il thread corrente (curthread) (dealloca la thread struct) e quindi il processo associato al thread si autodistrugge. Non possono piu' essere eseguite operazioni successive a questa istruzione in quanto il thread viene tolto dal sistema di scheduling (e come se si scrivessero dati al di fuori del main() in c, il programma viene terminato prima).
- **thread_yield()** ⇒ Usata per forzare un context switching ed eseguire un altro thread, se presente nella ready queue.
- **thread_destroy()** ⇒ Si occupa di liberare l'area di memoria relativa ad un determinato thread, anche la TCB. Quindi di deallocare la struttura dati del thread che viene passato come parametro.

```
thread_destroy(struct thread *thread) {
    KASSERT(thread != curthread);
    KASSERT(thread->t_state != S_RUN);

    /* If you add things to struct thread,
       be sure to clean them up
       either here or in thread_exit().
       (And not both...) */

    /* Thread subsystem fields */
    KASSERT(thread->t_proc == NULL);
    if (thread->t_stack != NULL) {
        kfree(thread->t_stack);
    }
    ...
    kfree(thread->t_name);
    kfree(thread);
}
```

- Si nota come c'e' l'istruzione:
 - KASSERT(thread->state != S_RUN) in quanto il thread per poter essere distrutto deve essere nello stato S_ZOMBIE.
- **thread_switch()** ⇒ Viene utilizzata per effettuare il context switching (processore passa all'esecuzione di un altro processo).

La **thread_fork()** contiene inoltre tre funzioni e vari parametri:

- **entrypoint** ⇒ Rappresenta il puntatore alla prima funzione che il nuovo thread dovra' eseguire.
- **thread_create()** ⇒ per creare nuovo thread ed allocare memoria per esso.
- **switchframe_init()** ⇒ Per memorizzare il valore dei registri per il context switching.

- **thread_make_runnable()** ⇒ Per andare in ready list.

```
thread_fork(const char *name, struct proc *proc,
            void (*entrypoint)(void *data1, unsigned long data2),
            void *data1, unsigned long data2) {
    struct thread *newthread;
    int result;
    newthread = thread_create(name);
    /* Allocate a stack */
    newthread->t_stack = kmalloc(STACK_SIZE);
    /* Thread subsystem fields */
    newthread->t_cpu = curthread->t_cpu;
    /* Attach the new thread to its process */
    result = proc_addthread(proc, newthread);
    /* Set up the switchframe so entrypoint() gets called */
    switchframe_init(newthread, entrypoint, data1, data2);
    /* Lock the current cpu's run queue and make the new thread runnable */
    thread_make_runnable(newthread, false);

    return 0;
}
```

2.6 Gestione processi

Ogni volta che un programma viene eseguito OS161 istanzia una struct "proc" che rappresenta il PCB (Process Control Block). Se un processo ha un thread, quest'ultimo memorizza il PID del processo a cui fa riferimento.

All'esecuzione di un programma quindi:

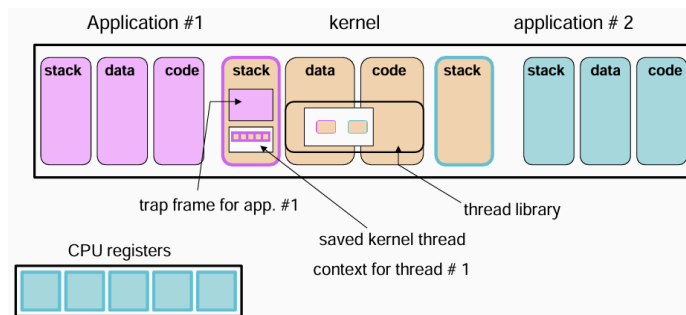
- OS161 chiama la funzione **proc_create_runprogram** dove crea PCB e alloca memoria.
- Si chiama **thread_fork()** (ogni processo deve avere almeno un thread!).
- Si esegue **run_program**.

2.7 System Calls

L'unico modo per comunicare con il sistema operativo e' mediante **system calls** (chiamate a sistema), che permettono di effettuare operazioni di I/O oppure operazioni come **read_file** o **write_file**. Vengono quindi **utilizzate nei programmi utente per chiedere al kernel di eseguire operazioni privilegiate**.

Non vi e' differenza tra system calls, interrupt o eccezione a livello di interruzione per il S.O.

L'handler delle system calls e' gestito da uno switch, per capire le operazioni da fare.



Ogni applicazione ha la sua memoria (stack, data e code). Essendo OS161 basato su MIPS 32 lo spazio di indirizzamento massimo e quindi la memoria e' di 4GB:

- 2GB per utente.
- 2GB per kernel.

TRAPFRAME

Quando un programma in user mode tenta di eseguire un'operazione privilegiata, come una system call, la CPU genera **un'eccezione*** chiamata **TRAP**. Questo evento trasferisce il controllo dal programma utente al **kernel** del sistema operativo (**QUINDI LA SYSTEM CALL NON VIENE ESEGUITA DAL PROCESSO UTENTE MA DAL KERNEL**).

Durante questo passaggio, la **CPU salva lo stato corrente del processo** all'interno di una struttura chiamata **trapframe**. Il trapframe **contiene tutti i registri della CPU** nel momento in cui è avvenuto il trap, inclusi:

- **v0** ⇒ usato per indicare il **numero della system call**. Il suo valore viene copiato dalla variabile `callno` durante la TRAP.
- **{a0, a1, a2}** ⇒ che contengono i **parametri** della chiamata, altri registri necessari per ripristinare l'esecuzione al ritorno dal kernel.

A questo punto viene eseguito lato kernel la funzione:

```
void syscall(struct trapframe *tf) {
    /* ... */
    retval = 0;
    switch (callno) {
    case SYS_reboot:
        err = sys_reboot(tf->tf_a0);
        break;
    case SYS__time:
        err = sys__time((userptr_t)tf->tf_a0, (userptr_t)tf->tf_a1);
        break;
```

```

/* HERE GOES THE MISSING IMPLEMENTATIONS */
default:
kprintf("Unknown syscall %d\n", callno);
1
err = ENOSYS;
break;
}
/* ... */
}

```

Questa funzione legge il valore del registro **tf->tf_v0** in modo da **capire quale systemcall eseguire**. A questo punto viene richiamata la funzione corretta mediante **controllo sul parametro callno** (che viene assegnato **TRAMITE tf->v0 in seguito alla TRAP**) e vengono passati i parametri giusti utilizzando appunto i registri {a0, a1, a2}

```
retval = sys_write(tf->tf_a0, tf->tf_a1, tf->tf_a2);
```

In questo caso sys_write richiedeva 3 parametri, quindi si utilizzando tutti e 3 i registri.

```

if (err) {
    tf->tf_v0 = err;
    tf->tf_a3 = 1;
}
else {
    tf->tf_v0 = retval;
    tf->tf_a3 = 0;
}

```

Completata l'esecuzione della system call il **valore di ritorno viene posto in tf->tf_v0** e il registro tf->tf_a3 viene settato a 0 o ad 1 in base alla correttezza nell'esecuzione della system call.

* → N.B. TRAP e' un eccezione e non interruzione, questo perche e' generata da un processo utente (e non da device esterno come I/O). L'interrupt e' un meccanismo HW, mentre l'eccezione fa riferimento ad eventi interni al S.O.

SYS_WRITE

```

int
sys_write(int fd, userptr_t buf_ptr, size_t size)
{
    int i;
    char *p = (char *)buf_ptr;

    if (fd!=STDOUT_FILENO && fd!=STDERR_FILENO) {
        kprintf("sys_write supported only to stdout\n");
        return -1;
    }

    for (i=0; i<(int)size; i++) {
        putchar(p[i]);
    }

    return (int)size;
}

```

La funzione prede come parametri:

- fd ⇒ file descriptor, in modo da identificare dove scrivere. In questo caso si accetta la scrittura solo sullo STDOUT = (ovvero console) e STDERR.
- buf_ptr ⇒ puntatore dallo spazio utente contenente i dati che la system call dovrà scrivere su console. Dallo spazio utente in quanto appunto la system call viene chiamata a partire da un programma utente che voleva scrivere su riga di comando qualcosa.
- size ⇒ Dimensione dei dati da scrivere.

Nel corpo della funzione si controlla che il **file descriptor** sia riferito allo **STDOUT**. Successivamente, per ogni byte da scrivere, si richiama la **funzione putchar(byte[i])**.

N.B ⇒ La putchar() può essere sostituita con una kprintf("%c", p[i]) in quanto essa stessa utilizza la putchar. Non si può invece scrivere kprintf("%s", p) direttamente in quanto non è detto che con la SYS_WRITE sia garantito che la stringa finisca con '\0'.

In generale quindi, se un programma utente vuole scrivere sullo STDOUT (o anche su file ecc...) è **NECESSARIO** l'utilizzo della chiamata a sistema SYS_WRITE, che interfaccia lo user space al kernel.

SYS_READ

```

int
sys_read(int fd, userptr_t buf_ptr, size_t size)
{
    int i;
    char *p = (char *)buf_ptr;

    if (fd!=STDIN_FILENO) {
        kprintf("sys_read supported only to stdin\n");
        return -1;
    }

    for (i=0; i<(int)size; i++) {
        p[i] = getch();
        if (p[i] < 0)
            return i;
    }

    return (int)size;
}

```

Il funzionamento e' molto simile alla SYS_Write. In questo caso, il **buf_ptr** viene aggiornato con i singoli caratteri che vengono letti dallo STDIN (riga di comando) tramite la funzione getch().

Successivamente, la funzione ritorna il numero di caratteri letti nel registro tf->tf_v0, per poi tornare allo user space. Oltre al controllo sul file descriptor, si dovrebbe anche controllare che buf_ptr non punti a indirizzi relativi a kseg0 (kernel space) e lanciare un errore eventualmente.

Quindi BUF_PTR fa riferimento alla regione di memoria utente nella quale si vogliono copiare i byte letti da riga di comando (o nell'esempio sottostante da file)

Questa e' la versione della read() dove fd fa riferimento ad un file:

```

int sys_read(int fd, userptr_t buf, size_t size) {
    struct vnode *v;
    int result = 0;
    char *p = (char *)buf_ptr;

    if (fd == STDIN_FILENO){
        int i = 0;
        for (i=0; i < (int)size; i++){
            p[i] = getch();
            if(p[i] < 0){
                return i;
            }
        }
    }else{
        v = getVNodeFromDescriptor(fd); // dato il file // descriptor ottengo il vnode (rappresentazione del file)
        struct iovec iov;
        struct uio u;
        uio_uinit(&iov, &u, buf, size, 0, UIO_READ);
        result = VOP_READ(v, &u);
    }

    return result;
}

```

Altro esempio di sys_read con kernel buffer:

```
static int
file_read(int fd, userptr_t buf_ptr, size_t size) {
    ...
    kbuf = kmalloc(size);
    uio_kinit(&iiov, &ku, kbuf, size, of->offset, UIO_READ);
    result = VOP_READ(vn, &ku);
    if (result) {
        return result;
    }
    of->offset = ku.uio_offset;
    nread = size - ku.uio_resid;
    copyout(kbuf, buf_ptr, nread);
    kfree(kbuf);
    return (nread);
}
```

SYS_EXIT

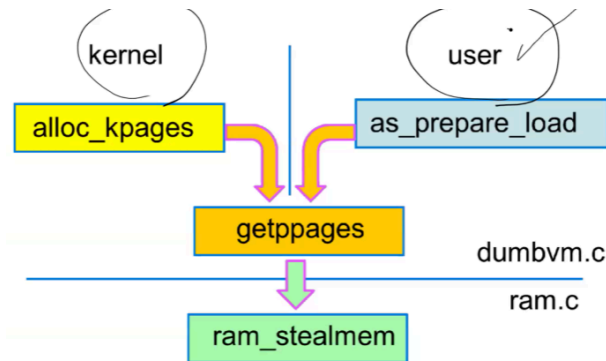
```
void
sys__exit(int status)
{
    /* get address space of current process and destroy */
    struct addrspace *as = proc_getas();
    as_destroy(as);
    /* thread exits. proc data structure will be lost */
    thread_exit();

    panic("thread_exit returned (should not happen)\n");
    (void) status; // TODO: status handling
}
```

System call utilizza per terminare un programma in esecuzione. Nello specifico, si prende il puntatore allo spazio di indirizzamento del processo corrente e si richiama la funzione "as_destroy(as)". Questa funzione libera le zone di memoria associate togliendo quindi il processo dalla memoria. Inoltre si richiama "thread_exit()", la quale termina il thread associato al processo corrente (IN OS161 1 PROCESSO HA UN SOLO THREAD). In questo modo il processo si autodistrugge. Un'ulteriore modifica riguarda l'implementazione della systemcall waitpid(). In tale implementazione, questo codice viene modificato in modo tale che sia il processo padre a distruggere il figlio.

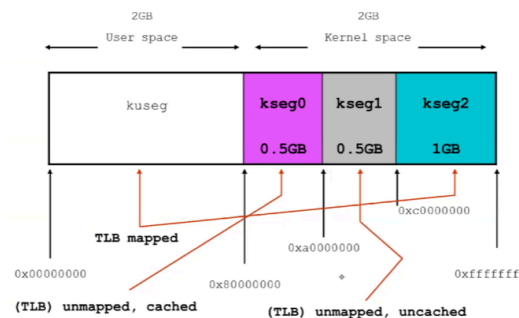
2.8 Address space & Memory Management

OS161 si basa su un tipo di **allocazione contigua** ma implementa anche la **paginazione** con **pagine da 4KB**. Questo significa che se e' necessario allocare, ad esempio, 10 pagine esse devono essere in posizioni contigue (**vedi 2.8.1 con as_define_region**). La richiesta di allocazione della memoria avviene sia lato kernel che lato utente:



Per fare cio' ci sono due metodi:

- dumbvm.c ⇒ Piu' di alto livello.
- ram.c ⇒ Basso livello basato sull'architettura del sistema.



OS161 essendo basato su MIPS ha uno **spazio di indirizzamento logico di 4GB** con indirizzi di 32 bit:

- 2GB user-space.
- Kseg0, riguarda data, codice e stack a livello di kernel.
- Kseg1, I/O devices. I device vengono mappati qui in quanto kseg1 non e' associato ad una cache e un device I/O non puo' essere letto/scritto utilizzando la cache.
- Kseg2, non utilizzato.

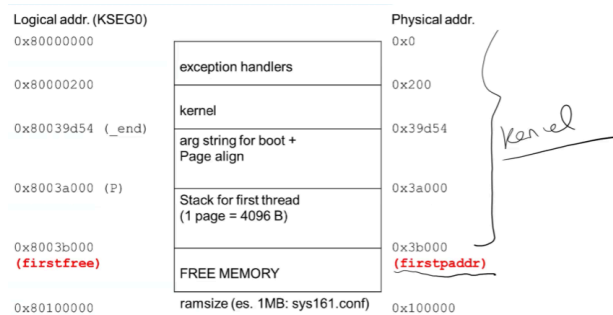
Per mappare questo in memoria fisica, si parte dall'indirizzo 0 con l'allocazione del kernel. Dopo questo inizia lo spazio di memoria libero usato per l'allocazione di processi.

Quindi:

- Il **processore genera indirizzi logici user che vanno da 0x0000 0000 a 0x7fff ffff**. Questi indirizzi fanno riferimento alla sezione di ram **KUSEG**, e vengono mappati in indirizzi fisici mediante una TLB sempre però in KUSEG.

- Gli indirizzi logici kernel vanno da 0x8000 0000 a 0xffff ffff, e per **passare da logici a fisici si sottrae 0x8000 0000 (MIPS_KSEG0)** (viceversa si somma).

Quando un processo utente viene creato (vedi 2.11.1), visto che si parte da processi kernel (quindi memorizzati in KSEG0), il sistema operativo, con il passaggio da kernel a user mode, vedrà da quel momento in poi il processo memorizzato in KUSEG.



Il primo indirizzo per l'allocazione dei processi è "firstpaddr (0x3b000)" e va fino al size della RAM (0x100000). Questa memoria libera viene gestita tramite due livelli (DUMBVM e ram_stealmem). Si nota inoltre come OS161 utilizza un sistema di indirizzamento logico (virtuale) in modo da evitare che la memoria assegnata ad un processo venga assegnata ad un altro. La mappatura tra memoria fisica e logica viene fatta sommando un valore costante ovvero 0x80000000.

Quindi in OS161 la mappatura da indirizzi logici KERNEL a fisici avviene sommando 0x80000000 (MIPS_KSEG0), compito svolto dalla macro PADRR_TO_KVADDR.

Qui ad esempio ho a sinistra indirizzi fisici, i corrispettivi logici si ottengono sommando MIPS_KSEG0.

```
0x440A0 -> 0x800440A0
0x45200 -> 0x80045200
0x48100 -> 0x80048100
0x51018 -> 0x80051018
```

Per quanto riguarda gli indirizzi utente invece devo fare:

Da virtuale → fisico:

$$\text{phys} = (\text{virt} - \text{vbase}) + \text{pbase}$$

Da fisico → virtuale:

$$\text{virt} = (\text{phys} - \text{pbase}) + \text{vbase}$$

Ad esempio:

```
as->as_pbase1, as->as_pbase2, as->as_vbase1, as->as_vbase2,
as->as_npages1, as->as_npages2, as->as_stackpbase, having the
following values: 0x200000, 0x300000, 0x3000, 0x7000, 3, 4,
0x400000
```

0x8110 -> (in
0x6500 -> inv
0x7FFFE010
0x805000B0

0x8110 e' dopo `as_vbase2`, quindi potrebbe fare riferimento al segmento codice:

Verifico:

$0x8110 - 0x7000 = 0x1110$.

$0x1110 / 0x1000$ (ovvero 4096) = 1+1=2 (sommo 1 in quanto si parte da 0, quindi 1 sarebbe pagina 2), quindi $e' < 4$, quindi **sta nelle 4 pagine** di grandezza del segmento codice.

Ora converto 0x8110 in fisico, e per farlo sommo l'offset tra 0x7000 e 0x8110 (ovvero appunto 0x1110) a `pbase2`:

- $0x300000 + 0x1110 = \mathbf{0x301110}$.

0x6500 e' tra `vbase1` e `vbase2`. Vediamo se rientra in `vbase1`:

- $0x6500 - 0x3000 = 0x3500$.

- $0x3500 / 0x1000 = 3,3 \rightarrow$ quindi supera le 3 pagine massime del segmento dati e quindi non appartiene allo spazio di indirizzamento del processo.

0x7FFFE010 e' dopo 0x400000 quindi fa probabilmente riferimento allo stack. Lo stack parte da 0x80000000 a scendere, ed e' di 18 pagine, quindi controllo che questo ci rientri:

- $0x80000000 - 0x12000$ (ovvero 4kb*18 appunto) = 0x7FFE E000 (ovvero **inizio dello stack logico**).

- $0x7FFFE010 - 0x7FFE E000 = 0x10010$ che $e' > 0$ quindi ci rientra.

- Allora per la conversione in fisica faccio:

- $0x400000 + 0x10010 = 0x410010$

0x805000B0 $\Rightarrow$ e' maggiore di 0x8000 0000 quindi e' logico **kernel**. In questo caso per convertirlo in fisico basta sottrarre per `MIPS_KSEG0`:

- $0x805000B0 - 0x8000 0000 = 0x5000B0$.

4 An oS161 system is given, running on a sys161 MIPS simulator with 4MB of RAM memory.

For each of the following addresses, say if they can be a logical user address, a logical kernel address, a physical address (explain/motivate answers):

- 0x80803005: it cannot be a user logical address because it is $> 2G$. it cannot be a physical address because it is $> 4M$, it could be a logical kernel address, but it is not compatible with the RAM (it should be $< 0x80400000$)
- 0x312010: it can be both a logical user and a physical address because it is $< 2G$ and $< 4M$, respectively. It cannot be a kernel logical address
- 0x532100: similar to the previous one, but it cannot be a physical address as it is $> 4M$

Da qui inoltre si nota come quando devo controllare se un indirizzo e' logico o fisico devo:

- Controllare se sia $< 2G$ o $> 2G$ (ovvero sotto o sopra $0X8000\ 0000$ rispettivamente). Se rientra nel primo caso allora puo' essere logico user o fisico. Se rientra nel secondo allora potrebbe essere logico kernel ma:
 - Devo controllare che $0x80803005 - 0x80000000$ sia $< 4M$ (che in questo caso e' la dimensione della RAM), per capire se puo' essere effettivamente logico kernel.
- Se devo verificare che sia logico user o fisico, allora:
 - Se $< 2G$ appunto e $> 4M$ allora e' sicuramente logico user e non puo' essere fisico.
 - Se minore di $4M$ (e anche di $2G$ per condizione di partenza) allora puo' essere sia logico user che fisico.

Quindi l'**allocazione di memoria avviene a due livelli**:

- **getppages()** $\Rightarrow$ richiama ramstealmem per decidere in quale zona di KSEG0 allocare il nuovo processo (utente o kernel che sia).
- **ram_stealmem()** $\Rightarrow$ Alloca memoria continua per un nuovo processo a partire da firstpaddr (primo indirizzo fisico disponibile che viene calcolato a seguito del bootstrap).

Il sistema di allocazione è quindi COMUNE TRA KERNEL E USER SPACE. La differenza è che:

- Un processo utente richiama **as_prepare_load()**, che richiama a sua volta **getppages()** per allocare cio' che il processo utente ha bisogno (codice e dati + stack).
- Un processo kernel, a seguito di una **kmalloc**, richiama **alloc_kpages()**, che richiama **getppages()**. **In kmalloc() posso quindi allocare memoria che non e' multiplo di pagina**, sara' poi **alloc_kpages()** a far rientrare tutto in modo corretto in multipli di pagina.

```
void
ram_bootstrap(void) {
    /* Get size of RAM. */
    size_t ramsize = mainbus_ramsize(); if
    (ramsize > 512*1024*1024) {
        ramsize = 512*1024*1024;
    }
    lastpaddr = ramsize;
    /* Get first free virtual address from where start.S saved
    it. Convert to physical address. */
    firstpaddr = firstfree - MIPS_KSEG0;
}
```

ram_bootstrap (ram.c) $\Rightarrow$ S.O. chiede al controller di RAM la dimensione della ram [mainbus_ramsize()]. **Da qui calcolo il firstpaddr** sulla base di quanto spazio ha richiesto il kernel

(MIPS_KSEG0). Questa informazione viene data a dumbvm e ram.c.

```
paddr_t ram_stealmem(unsigned long npages) {
    paddr_t paddr;
    size_t size = npages * PAGE_SIZE;
    if(firstpaddr + size > lastpaddr) {
        return 0;
    }
    paddr = firstpaddr;
```

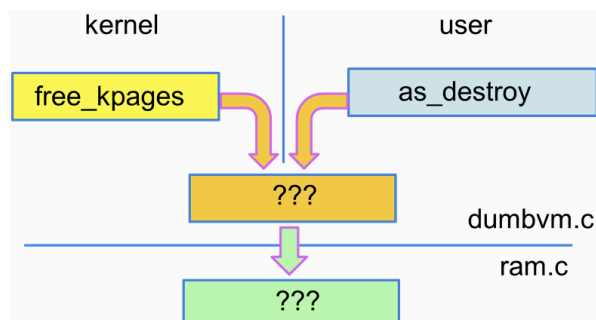
ram_stealmem (ram.c) ⇒ Viene utilizzata per allocare memoria (anche alla fine di bootstrap). Specifico un certo numero di pagine e in questo modo calcolo la dimensione totale. Se la dimensione e' minore rispetto a quella ancora disponibile, allora avanzo il puntatore **firstpaddr** di conseguenza.

Firstpaddr e' quindi il puntatore al primo indirizzo libero in memoria.

```
static paddr_t
getppages(unsigned long npages) {
    paddr_t addr;
    spinlock_acquire(&stealmem_lock);
    addr = ram_stealmem(npages);
    spinlock_release(&stealmem_lock);
    return addr;
}
```

getppages (dumbvm.c) ⇒ Chiama ram_stealmem aggiungendo uno spinlock_acquire(). Permette di fare un lock su ram_stealmem(), in modo da eseguire in modo mutuamente esclusivo questa funzione (**un solo processo puo' allocare memoria alla volta**).

Questa prima parte di allocazione della memoria era gia' implementata. Quello non presente e' la gestione del rilascio della memoria.



Ci sono varie soluzioni:

- A1 ⇒ si realizzano funzioni parallele a getppages (freeppages) e ram_stealmem (ram_freemem).
- A2 ⇒ Si lascia ram_stealmem e si lavora solamente ad alto livello (dumbvm.c) e si implementa solo freeppages basandosi su getppages.
- B ⇒ Si parte da 0, implementando un sistema di paging completo.

2.8.1 Implementazione dumbvm.c ⇒ Per liberare memoria

OS161 di base implementa solamente funzioni per allocare la memoria, ma non gestisce la deallocazione di quest'ultima.

Le funzioni di base di dumbvm sono:

- **as_create** -> Utilizzata per inizializzare e creare la struttura **addrspace**. Addrspace e' quindi la rappresentazione della memoria virtuale del processo:

- Descrive quali zone di memoria ha.
- Dove iniziano queste regioni.
- Quante pagine occupano le singole regioni.

Qui sotto ad esempio si ha che:

- **vbasel** $\Rightarrow$ Indirizzo virtuale iniziale della regione codice.
- **pbasel** $\Rightarrow$ Indirizzo fisico iniziale della regione codice del processo.
- **npages1** $\Rightarrow$ Numero di pagine/frame utilizzati per allocare la regione codice.
- **vbase2/pbase2/npages2** lo stesso di prima ma per regione dati.
- **stackpbase** $\Rightarrow$ Indirizzo fisico iniziale dello stack del processo.

```
struct addrspace *
as_create(void)
{
    struct addrspace *as = kmalloc(sizeof(struct addrspace));
    if (as==NULL) {
        return NULL;
    }

    as->as_vbase1 = 0;
    as->as_pbase1 = 0;
    as->as_npages1 = 0;
    as->as_vbase2 = 0;
    as->as_pbase2 = 0;
    as->as_npages2 = 0;
    as->as_stackpbase = 0;

    return as;
}
```

- **as_define_region** -> Definisce una **nuova regione di memoria virtuale** all'interno dello spazio degli indirizzi di un processo. **Allinea l'indirizzo virtuale vaddr all'inizio di una pagina e arrotonda la dimensione sz a un multiplo di PAGE_SIZE**, così da calcolare correttamente il numero di pagine (npages) necessarie. Questa **regione viene poi registrata nella struttura addrspace** (tipicamente in **as_vbase1** e **as_npages1** se è la prima regione).

Vaddr rappresenta quindi l'indirizzo virtuale di partenza del segmento da definire all'interno dello spazio degli indirizzi virtuali di un processo. Se caratterizzato da un **offset rispetto all'indirizzo di inizio di pagina**, e' possibile che, anche con **sz < PAGE_SIZE**, sia **richiesto allocare 2 pagine per il processo** (frammentazione interna).

Quindi **DUMBVM (DI BASE!!)** utilizza un **approccio di allocazione di tipo contiguo per multiplo di pagina**. Con multiplo di pagina si intende che appunto **vaddr** deve essere allineato, quindi c'e' un offset che viene sprecato per avere questo allineamento.

```

int
as_define_region(struct addrspace *as, vaddr_t vaddr, size_t sz,
                 int readable, int writable, int executable)
{
    size_t npages;

    dumbvm_can_sleep();

    /* Align the region. First, the base... */
    sz += vaddr & ~(vaddr_t)PAGE_FRAME;
    vaddr &= PAGE_FRAME;

    /* ...and now the length. */
    sz = (sz + PAGE_SIZE - 1) & PAGE_FRAME;

    npages = sz / PAGE_SIZE;

    /* We don't use these - all pages are read-write */
    (void)readable;
    (void)writable;
    (void)executable;

    if (as->as_vbase1 == 0) {
        as->as_vbase1 = vaddr;
        as->as_npages1 = npages;
        return 0;
    }

    if (as->as_vbase2 == 0) {
        as->as_vbase2 = vaddr;
        as->as_npages2 = npages;
        return 0;
    }

    /*
     * Support for more than two regions is not available.
     */
    kprintf("dumbvm: Warning: too many regions\n");
    return ENOSYS;
}

```

- **as_prepare_load()** -> utilizza la funzione **getppages()** per **allocare la memoria fisica delle singole regioni**. La quantità di memoria allocata dipende dal numero di pagine impostato con la funzione **as_define_region()**. Alloca anche la memoria per lo stack ed inizializza la memoria fisica. Al fine di implementare un sistema di **liberazione della memoria** e' possibile seguire una serie di passaggi:

```

int
as_prepare_load(struct addrspace *as)
{
    KASSERT(as->as_pbase1 == 0);
    KASSERT(as->as_pbase2 == 0);
    KASSERT(as->as_stackpbase == 0);

    dumbvm_can_sleep();

    as->as_pbase1 = getppages(as->as_npages1);
    if (as->as_pbase1 == 0) {
        return ENOMEM;
    }

    as->as_pbase2 = getppages(as->as_npages2);
    if (as->as_pbase2 == 0) {
        return ENOMEM;
    }

    as->as_stackpbase = getppages(DUMBVM_STACKPAGES);
    if (as->as_stackpbase == 0) {
        return ENOMEM;
    }

    as_zero_region(as->as_pbase1, as->as_npages1);
    as_zero_region(as->as_pbase2, as->as_npages2);
    as_zero_region(as->as_stackpbase, DUMBVM_STACKPAGES);

    return 0;
}

```

DEFINIZIONE STRUTTURE

```

static unsigned char *freeRamFrames = NULL; // vettore contenente per ogni frame in memoria se esso e' libero o meno (1 libera, 0 occupata)
static unsigned long *allocSize = NULL; // numero di pagine contigue allocate partendo da indirizzo i-esimo
static int nRamFrames = 0; // numero di frames in RAM
static int allocTableActive = 0; // settato ad 1 dopo la fase di bootstrap, per capire se essa e' andata a buon fine

```

Si inizializzano queste strutture dati utili per gestire il rilascio della memoria:

- **freeRamFrames** ⇒ vettore/bitmap che contiene un numero di elementi pari al numero di frame (da 4kb) di memoria. Ogni elemento contiene un valore 0 (se frame i-esimo occupato) oppure 1 (se libero).

- **allocSize** ⇒ Vettore parallelo a **freeRamFrames**, che indica quanti **frame contigui sono stati allocati a partire dall'indice i-esimo**. Ad esempio, se **allocSize[10] = 2**, allora ci sono 2 frame allocati a partire dal frame associato all'indice 10. Quindi **freeRamFrames[10, 11] = 0**. Questo è utile nel rilascio:
 - Quando si vuole rilasciare a partire da un frame **i**, si guarda **allocSize[i]**
 - Si rilasciano **allocSize[i]** frame consecutivi, mettendo **freeRamFrames[i, .. allocSize[i]-1] = 1**.
- **nRamFrames** ⇒ contiene numero di frames in RAM = **RAM_SIZE/4KB**.

INIZIALIZZAZIONE

```
void
vm_bootstrap(void)
{
    /* Do nothing. */

    // inizializzazione struttura dati per gestione memoria
    int i = 0;

    nRamFrames = ((int)ram_getsize() / PAGE_SIZE); // il numero di frame = size_ram / page_size

    freeRamFrames = kmalloc(sizeof(unsigned char) * nRamFrames); // alloco freeRamFrames in modo che abbia una dimensione pari al numero di frames
    allocSize = kmalloc(sizeof(unsigned long) * nRamFrames);

    if (freeRamFrames==NULL || allocSize==NULL) { // si controlla se kmalloc è andata a buon fine
        freeRamFrames = NULL;
        allocSize = NULL;
        return;
    }

    for (i=0; i<nRamFrames; i++) { // per ogni frame
        freeRamFrames[i] = (unsigned char)0; // si imposta la pagina i-esima come occupata?
        allocSize[i] = 0;
    }
    //spinlock_acquire(&freemem_lock);
    allocTableActive = 1;
    //spinlock_release(&freemem_lock);
}
```

Si inizializzano le strutture definite in precedenza al bootstrap. Nello specifico si allocano **freeRamFrames** e **allocSize**. Successivamente, **tutti i frame della memoria vengono segnati come occupati (freeRamFrames[i] = 0)**. Vengono poi liberati man mano che i vari processi ne hanno bisogno con le funzioni successive.

GET_PPGAES

```

static
paddr_t
getppages(unsigned long npages)
{
    paddr_t addr;
    addr = getfreeppages(npages); // si allocano eventualmente pagine per il processo attuale

    if(addr == 0) { // Se non c'e' memoria disponibile, allora chiama stealmem
        spinlock_acquire(&stealmem_lock);
        addr = ram_stealmem(npages);
        spinlock_release(&stealmem_lock);
    }
    if(addr != 0 && isTableActive()){
        // se si trova memoria libera, si salva quante pagine sono state allocate
        //spinlock_acquire(&freemem_lock);
        allocSize[addr/PAGE_SIZE] = npages;
        //spinlock_release(&freemem_lock);
    }
    return addr;
}

```

Funzione chiamata quando si crea un processo ed e' necessario allocare memoria per quest'ultimo. Si specifica il numero di pagine che il processo ha bisogno con il parametro **"npages"**. Inizialmente **si controlla se ci sono pagine libere tra quelle precedentemente allocate** in memoria tramite la funzione **getfreeppages()** [vedi dopo].

Se non ci sono pagine libere allora si richiama la funzione **ram_stealmem(npages)** che **alloca memoria a partire dal primo indirizzo fisico disponibile** (non guardando se si e' liberata qualche frame prima).

GET_FREE_PPAGES

```

// restituisce blocco di npages continue liberate per l'allocazione del nuovo processo
static paddr_t getfreeppages(unsigned long npages){
    paddr_t addr;
    long i, first, found, np = (long)npages;

    if (!isTableActive()) return 0; // se le strutture non sono state allocate correttamente in fase di bootstrap non va avanti
    //spinlock_acquire(&freemem_lock); // attiva un lock in modo che un solo processo alla volta effettui le operazioni sottostanti

    first = found = -1;

    for(i=0; i < nRamFrames; i++){
        if (freeRamFrames[i] == 1) { // se la pagina i-esima e' libera
            if (i == 0 || !freeRamFrames[i - 1]) // se e' l'inizio di un nuovo blocco libero (first)
                first = i; // Primo frame libero di un nuovo intervallo

            if (i - first + 1 >= np) // Se trovo abbastanza pagine libere consecutive, maggiore di np (npages appunto)
                found = first;
        }
    }

    if (found >= 0) { // Se e' stato trovato un intervallo di dimensione abbastanza grande libero
        for (i = found; i < found + np; i++) { // segno tutte le pagine all'interno di quell'intervallo come occupate
            freeRamFrames[i] = (unsigned char)0; // Segna come OCCUPATE
        }
        allocSize[found] = np; // Salva quante pagine sono state allocate
        addr = (paddr_t) found * PAGE_SIZE; // Calcola l'indirizzo fisico di partenza
    } else {
        addr = 0; // Nessun blocco trovato
    }

    //spinlock_release(&freemem_lock);
    return addr; // restituisce indirizzo fisica di partenza della zona appena occupata
}

```

Si parte **dall'inizio della memoria**, controllando tramite **freeRamFrames** se c'e' un frame libero. In tal caso, se trovo **abbastanza pagine libere consecutive** a partire dal frame

libero trovato allora **ritorno l'indirizzo di memoria a partire dal quale iniziare l'allocazione** del nuovo processo.

N.B ⇒ L'algoritmo non implementa una politica di tipo FIRST-FIT in quanto NON esce dal loop appena trova un intervallo abbastanza lungo di pagine libere! Anzi, il programma trova l'ultimo intervallo abbastanza lungo (con una politica di tipo last-fit). Se si volesse modificare in modo da avere una politica di tipo **first-fit**:

```
static paddr_t
getfreepages(unsigned long npages) {
    paddr_t addr = 0;
    long i, first, found, np = (long)npages;

    if (!isTableActive()) return 0;
    spinlock_acquire(&freemem_lock);
    for (i=0, first=found=-1; i<nRamFrames && found < 0; i++) {
        if (freeRamFrames[i]) {
            if (i==0 || !freeRamFrames[i-1])
                /* set first free in an interval */
                first = i;
            if (i-first+1 >= np){
                found = first;
                break;
            }
        }
    }
    if (found>=0) {
        for (i=found; i<found+np; i++) {
            freeRamFrames[i] = (unsigned char)0;
        }
        allocSize[found] = np;
        addr = (paddr_t) found*PAGE_SIZE;
    }
    spinlock_release(&freemem_lock);
    return addr;
}
```

Appena trovo abbastanza pagine libere consecutive aggiorno found e faccio break.

Se invece volessi modificare il codice in modo da avere una politica di tipo **best-fit**:

```
static paddr_t
getfreepages(unsigned long npages) {
    paddr_t addr = 0;
    long i, first, found, np = (long)npages;

    long min = MAX_INT;

    if (!isTableActive()) return 0;
    spinlock_acquire(&freemem_lock);
    for (i=0, first=found=-1; i<nRamFrames; i++) {
        if (freeRamFrames[i]) {
            if (i == nRamFrames-1 || !freeRamFrames[i+1])
                /* set first free in an interval */
                first = i;
            if (i-first+1 >= np){
                if (found < 0 || i - first + 1 < min) {
                    found = first;
                    min = i - first + 1;
                }
            }
        }
    }
    if (found>=0) {
```

```

    for (i=found; i<found+np; i++) {
        freeRamFrames[i] = (unsigned char)0;
    }
    allocSize[found] = np;
    addr = (paddr_t) found*PAGE_SIZE;
}
spinlock_release(&freemem_lock);
return addr;
}

```

Invece di fare il controllo all'inizio del blocco **si fa alla fine direttamente**, ad esempio:

- Controlliamo solo quando `i==nRamFrames-1 || !freeRamFrames[i+1]`:
 - `i=2`: Fine del primo blocco → dimensione = $2-0+1 = 3$ (OK! `found = 0`, `min = 3`)
 - `i=8`: Fine del secondo blocco → dimensione = $8-5+1 = 4$ (peggiore di `min=3`, ignorato)

FREE_KPAGES

```

void free_kpages(vaddr_t addr)
{
    /* nothing - leak the memory. */

    if(isTableActive()){
        paddr_t paddr = addr - MIPS_KSEG0; // Conversione da indirizzo virtuale a fisico
        long first = paddr / PAGE_SIZE;    // Calcolo dell'indice della prima pagina in RAM
        KASSERT(nRamFrames > first);       // Controllo: l'indice non deve superare la RAM
        freeppages(paddr, allocSize[first]); // Libera le pagine, usando allocSize per sapere quante
    }
    (void)addr;
}

```

Funzione usata per **liberare pagine lato kernel**. Si ricava **l'indirizzo fisico a partire dal logico passato come parametro sottraendo MIPS_KSEG0**.

Successivamente si richiama `freeppages` sull'indirizzo trovato.

AS_DESTROY (AS)

```

// funzione richiamata quando un processo termina
void as_destroy(struct addrspace *as)
{
    dumbvm_can_sleep();
    freeppages(as->as_pbase1, as->as_npages1); // libera pagine per parte dati
    freeppages(as->as_pbase2, as->as_npages2); // libera pagine per parte codice
    freeppages(as->as_stackbase, DUMBVM_STACKPAGES); // libera pagine relative allo stack
    kfree(as);
}

```

Funzione richiamata per **liberare lo spazio di indirizzamento di un processo utente**. In particolare, si richiama la funzione `freeppages` 3 volte in modo da eliminare parte dati, codice e stack del processo.

as_pbase1,2/stackbase -> sono puntatori a queste aree di memoria rispettivamente. Si nota inoltre come le **pagine allocate per lo stack sono fisse**.

Quindi:

- Quando un processo termina si richiama la funzione `free_kpages` se processo kernel o `as_destroy()` se processo utente. In generale, si utilizza la funzione `freeppages` sull'indirizzo di memoria del suo spazio di indirizzamento, in combinazione con il numero di pagine allocate.
- Quando un processo viene creato, se il processo e' utente si:
 - **Crea l'address space** mediante `as_create()`.
 - Dato l'address space si **definiscono le regioni** di memoria (**codice, data, stack**) e quante pagine occuperanno mediante `as_define_region()`.
 - Dato l'address space si **allocano in KSEG0 le pagine necessarie** alle singole sezioni mediante `as_prepare_load()` che **richiama** per ogni sezione `getppages()`. A questo punto si richiama la funzione `getfreeppages()` che controlla se si sono liberati almeno N frame consecutivi nel mentre. Altrimenti, si richiama `ram_stealmem()` che fa avanzare il puntatore al prossimo frame libero di memoria, in modo da allocare nuove pagine in sequenza.
- Se il processo e' **kernel**:
 - Si richiama direttamente `alloc_kpages()` che a sua volta **richiama** `getppages()` e utilizza eventuali pagine libere oppure si alloca tramite `ram_stealmem()`.
 - C'e' **questa distinzione** tra processi kernel e utente in quanti quelli **utente utilizzano un meccanismo di mapping mediante TLB** ed e' quindi necessario definire le regioni in questo modo.

In generale pero', visto che utilizzo `ram_stealmem()` per sia processi utente che processi kernel le **partizioni per questi processi sono interlacciate**.

2.9 Sincronizzazione primitive

2.9.1 Sezione critica

La parte di codice che e' **condivisa tra vari processi** prende il nome di **sezione critica**.

La sezione critica puo' quindi avere **variabili**, o file che vengono modificati da **piu' processi contemporaneamente**:

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (true);
```

Prima di entrare nella **critical section** un processo deve quindi fare richiesta di entrare nella **entry section**. Al contempo deve anche informare che sta uscendo dalla critical section (**exit section**), in modo che altri processi possano andare in entry section.

Bisogna quindi implementare dei **protocolli** che **gestiscano l'entrata e l'uscita** nella critical section. Questi protocolli devono avere caratteristiche come:

- **Mutua esclusione** ⇒ Un **solo processo alla volta** puo' eseguire critical section.
- **Progress** ⇒ Se la critical section e' **libera**, allora un processo deve poterla eseguire e **non essere bloccato**.
- **Bounded Waiting** ⇒ Tutti i processi devono avere il loro turno per eseguire la critical section (**EVITARE STARVATION**).

Gli algoritmi per la gestione della critical section dipendono dal kernel:

- **Preemptive** ⇒ **Processo** prende controllo di CPU per l'esecuzione del codice, ma la sua **esecuzione puo' essere fermata in ogni momento**.
- **Non-preemptive** ⇒ Quando il processo prende il controllo della CPU, il **kernel non puo' fermare** la sua esecuzione fino al rilascio (piu' semplice da gestire, in quanto non ho stop dell'esecuzione).

2.9.2 Soluzione di Peterson

Soluzione che ha bisogno di **due variabili**:

- **int turn** ⇒ **turno del processo** per entrare in sezione critica.
- **flag[2]** ⇒ Numero di elementi pari a numero di processi. Indica se **processo i-esimo e' pronto** per entrare in critical section.

```
while (true){  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j)  
        ;  
    /* critical section */  
}
```

Se P1 vuole entrare in critical section, setta il suo flag ad 1 (flag[0]=1). Da il turno a j (che rappresenta il processo successivo). Se l'altro processo (j) e' pronto per entrare viene eseguito, quando esce mettera turn=i e flag[j]=false quindi P1 viene poi eseguito.

Questa soluzione **non e' detto che funzioni nelle architetture moderne:**

- Two threads share the data:

```
boolean flag = false;  
int x = 0;
```

- Thread 1 performs

```
while (!flag);  
print x
```

- Thread 2 performs

```
x = 100;  
flag = true
```

- What is the expected output? 100

Se x=100 viene eseguito dopo di flag=true perche il **compilatore** vede che esse sono istruzioni indipendenti allora non si stampa piu' 100 ma 0.

2.9.3 Sincronizzazione hardware

Il software non basta, sono necessari **meccanismi hardware per garantire un corretto accesso alla sezione critica** e sincronizzare correttamente i processi. Sono fondamentali **3 tecniche:**

2.9.4 Memory Barriers

Ogni architettura ha il suo modello della memoria:

- **Strongly ordered** ⇒ Una modifica della memoria da parte di un processo e' subito visibile anche agli altri processi.
- **Weakly ordered** ⇒ Non e' garantito che un processo avvisi i cambiamenti che ha fatto in memoria.

Il **memory barrier** e' un'istruzione che forza ogni cambiamento in memoria di essere propagato.

Thread 1 now performs

```
while (!flag)
memory_barrier();
print x
```

Thread 2 now performs

```
x = 100;
memory_barrier();
flag = true
```

In questo caso quindi, **tutto quello prima di memory_barrier() deve essere eseguito prima di andare avanti.**

2.9.5 Hardware instructions

Le architetture moderne ci danno la possibilita' di modificare il contenuto di una variabile in due modi:

- **Test-and-Set** $\Rightarrow$ Si performa un test su una condizioe, se condizione e' true si setta un valore.

```
do {
    while (test_and_set(lock)); /* do nothing */

    /* critical section */ set lock to true

    lock = false;

    /* remainder section */
} while (true);
```

Si usa una variabile booleana condivisa lock che puo' essere false/true. **Il primo processo che arriva prende il controllo di lock. Esso entra nella sezione critica e setta lock da false va a true.** Quando finisce setta il lock a false. L'altro processo vede lock a false e poi entra nella critical section.

- **Compare and swap** $\Rightarrow$ Comparando due variabili e se true allora si scambiano i valori.

```
int compare_and_swap(int *value, int expected, int new_value) {
    int temp = *value;

    if (*value == expected)
        *value = new_value;
    return temp;
}
```

Le istruzioni dentro la funzione sono atomiche, quindi devono essere eseguite tutte e non possono essere fermate. Il funzionamento e' quindi:

```

while (true){
    while(compare_and_swap(lock, 0 (expected), 1(new value)) != 0)
        ; /* do nothing */

    /* critical section */ lock = 0;

    /* remainder section */
}

```

Queste istruzioni hanno la caratteristica di essere atomiche.

2.9.6 Variabili atomiche

I metodi hardware visti in precedenza vengono usati come blocchi di base per altri metodi di sincronizzazioni, tipo per incrementare variabili.

2.9.7 Mutex Locks

I metodi software che usano soluzioni hardware sono ad esempio mutex e semafori.

Mutex Lock :

- Un processo **acquisisce il lock** prima di entrare in critical section con **acquire()**.
- Esegue la critical section e poi chiama **release()**.

```

while (true) {
    acquire lock

    critical section

    release lock

    remainder section
}

```

Queste due **chiamate** devono essere **atomiche** (implementate via hardware con compare-and-swap appunto)

Uno degli **svantaggi** e' che la **soluzione richiede busy waiting**. Mentre P1 e' in critical section l'altro e' bloccato nell'**attesa di acquire()** senza andare in sleep (spreca tempo di CPU). Questo tipo di

lock si chiama **SPINLOCK**.

I semafori risolvono questo problema, ma se la parte di **critical section e' molto corta**, fare il context switching non conviene, quindi **spinlock va bene**.

Gli spinlock in OS161 sono **implementati nel seguente modo**:

```

struct spinlock {
    int lock; // campo lock contiene un intero da ritornare dalle funzioni test_and_set
    struct thread *owner; //l'owner attuale del thread, serve a garantire che chi acquisisce lo
    spinlock lo rilasci
}

```

```

void spinlock_acquire(struct spinlock *splk) {
    // ...
    while (1) {
        if (spinlock_data_get(&splk->splk_lock) != 0) {
            continue;
        }
        if (spinlock_data_testandset(&splk->splk_lock) != 0) {
            continue;
        }
        break;
    }
    // ...
}

```

```

void
spinlock_release(struct spinlock *lk)
{
    /* this must work before curcpu initialization */
    if (CURCPU_EXISTS()) {
        KASSERT(lk->lk_holder == curcpu->c_self);
    }

    lk->lk_holder = NULL;
    spinlock_data_set(&lk->lk_lock, 0);
    spllower(IPL_HIGH, IPL_NONE);
}

```

Si nota come nel caso dell'acquire:

- Viene fatto un if sia su data_get che testandset in modo da poter:
 - **Fare un test non atomico "leggero"** ovvero data_get() che **guarda solo il valore del lock**, se vede che e' **conteso** si fa **ripartire** il ciclo.
 - Un test **atomico** vero e proprio **testandset()** quando il **lock sembra libero** per **provare ad acquisire il lock**.
- Viene messo un if invece che un while per data_get(), e' indifferente.

2.9.8 Semafori

Si basano su **variabile S**, che viene controllato da:

Definition of the wait()	• Definition of the signal()
<pre> wait(S) { while (S <= 0); // busy wait S--; } </pre>	<pre> signal(S) { S++; } </pre>

Quando si usa **wait(S)** il processo aspetta che **S** sia diverso da **0**. Quando **S > 0** **entra nella critical section e decrementa S**. Quando esce dalla critical section incrementa **S** (mediante **signal(S)**) e altro processo in attesa viene svegliato e puo' eventualmente prendere il controllo della critical section. Il **semaforo** puo' essere:

- **Counting semaphore** ⇒ Per sincronizzazione tra piu' processi.
- **Binary semaphore** ⇒ Coincide con il mutex, S puo' assumere valore tra 0 ed 1. Ma appunto NON fa busy waiting.

2.9.9 OS161 Sincronizzazione

Dobbiamo fare attenzione che quando un **thread esegue la sua critical section non debba essere fermato**. La **preemption** in OS161 viene garantita mediante **timer interrupts, disabilitando timer interrupts mentre si e' in critical section**.

Il **Timer Interrupts** e' un meccanismo periodico che passa il controllo al kernel il quale puo' ad esempio passare il controllo della CPU ad un altro thread

Si possono usare 3 funzioni per emulare la disabilitazione dei timer:

- **splhigh()** .
- **spl0()** .
- **splx()** .

Essi **cambiano la priorita' dei thread**. Quando un thread e' in critical section si mette ad alta priorita', in modo che non venga interrotto da altri.

```

struct semaphore {
    char *name;
    int count;
};

struct semaphore *sem_create(const char *name, int
    initial count);
void P(struct semaphore *);
void V(struct semaphore *);
void sem_destroy(struct semaphore *);
see
    • kern/include/synch.h
    • kern/thread/synch.c

```

Structure of data defining semaphore: name of the semaphore and count variable

Four functions for creating, signaling, waiting and destroying semaphore.

count sarebbe S del semaforo. **P()** e' wait, mentre **V()** e' signal.

```

void P(struct semaphore *sem) {
    int spl;
    assert(sem != NULL);

    /* May not block in an interrupt handler.
     * For robustness, always check, even if we can actually
     * complete the P without blocking. */
    assert(in_interrupt==0);

    spl = splhigh();
    while (sem->count==0) {
        thread_sleep(sem);
    }
    assert(sem->count>0);
    sem->count--;
    splx(spl);
}

void V(struct semaphore *sem)
{
    int spl;
    assert(sem != NULL);
    spl = splhigh();
    sem->count++;
    assert(sem->count>0);
    thread_wakeup(sem);
    splx(spl);
}

```

Saving the current priority of the process to recover later and Increasing the priority of the process.
 Considering the semaphore value, if the value of semaphore is 0, go to sleep.
 If sem is bigger than 0, increasing sem.
 Changing the priority to the original one.

Saving the current priority of the process to recover later and Increasing the priority of the process.
 Increasing the sem
 Waking up one of the processors in the waiting list.

In **P()**, il thread disabilita gli interrupt con **splhigh()** per accedere in sicurezza al valore del semaforo.

Se `count` è 0, va in sleep (si blocca) finché un altro thread non chiama `V()` e lo sveglia.

Quando il thread entra nella sezione critica, non viene modificata la sua priorità.

Dopo aver decrementato `count`, gli interrupt vengono riabilitati con `splx()`.

- `thread_sleep(addr)` $\Rightarrow$ blocca i thread con indirizzo `addr`.
- `thread_wakeup(addr)` $\Rightarrow$ Sblocca thread che stanno dormendo sull'indirizzo specificato.

```
struct lock *mylock = lock_create("LockName");

lock_acquire(mylock);
    critical section /* e.g., call to list_remove head */
lock_release(mylock);
```

Un thread che prende il controllo di un lock deve essere lo stesso che lo rilascia. Questa è la differenza con i semafori in OS161.

2.9.10 Condition variables

Un thread che usa **condition variables** come sincronizzazione, controlla il valore di una variabile di condizione. Se questo valore non è quello che sta aspettando si sospende e rilascia il lock(MUTEX). Quando il valore è corretto diventa ready nuovamente e può acquisire il lock.

Si ha quindi:

- **Condition variable**, se false si lascia il lock, se true allora si cerca di acquisire il lock:

P_i	P_j
<code>lock_acquire(lock);</code>	<code>lock_acquire(lock);</code>
<code>while (condition not true)</code>	<code>... // modify condition</code>
<code>{</code>	<code>cv_signal(cond);</code>
<code>cv_wait(cond, lock);</code>	<code>lock_release(lock);</code>
<code>}</code>	
<code>... // do stuff</code>	
<code>lock_release(lock);</code>	

cv_broadcast(cond) $\Rightarrow$ fa wake up di tutti i processi che stanno aspettando sulla condition variable.

Usando le condition variable e spinlock (busy waiting) dobbiamo fare sincronizzazione a livello di kernel.

2.9.11 Implementazione

Di base in OS161 è presente l'implementazione di semafori, spinlock e waitchannels. I semafori e gli spinlock vengono utilizzati per la sincronizzazione di thread, in modo da

evitare l'accesso ad una sezione critica in contemporanea tra due thread.

SPINLOCK VS LOCK

Gli **spinlock** e i **lock** sono come **semafori binari**, con la differenza che quando si fa P() nel caso del **lock il thread va in sleep**, mentre nel caso dello **spinlock entra in busy waiting**, occupando tempo di CPU. Si preferisce utilizzare la spinlock() quando la sezione critica e' molto breve. Il lock inoltre, a differenza anche di un semaforo, introduce il **concetto di ownership**, quindi **solo il thread che ha acquisito il lock puo' rilasciarlo**.

SEMAFORI IN OS161

```
void P(struct semaphore *sem) {
    spinlock_acquire(&sem->sem_lock);
    while (sem->sem_count == 0) {
        wchan_sleep(sem->sem_wchan,
                    &sem->sem_lock);
    }
    sem->sem_count--;
    spinlock_release(&sem->sem_lock);
}
```

La P() viene implementata mediante l'uso di wait_channel. Nello specifico lo spinlock() viene utilizzato per gestire la mutua esclusione su sem->sem_count == 0 e per accedere alla wait_channel. Si utilizza un while invece che un if in quanto wchan_sleep() e wchan_wakeone() non garantiscono che la condizione vera durante la wakeone() lo sia al ritorno da sleep().

La wchan_sleep deve rilasciare lo spinlock, prima di mettere il thread in stato "wait", per poi riprenderlo al risveglio (prima di ritornare al chiamante) (per questo si passa lo spinlock).

LOCKS CON SEMAFORI

```

struct lock {
    char *lk_name;
    HANGMAN_LOCKABLE(lk_hangman); /* Deadlock detector hook. */
    // add what you need here

    struct semaphore *sem; // semaforo associato al lock, dovra essere binario (inizializzato ad 1)
    struct thread *holder; // thread che ha acquisito il semaforo, sara' lo stesso a doverlo rilasciare

    // (don't forget to mark things volatile as needed)
};

```

Il lock e' quindi caratterizzato da un semaforo e dall'holder. L'holder in particolare, permette di implementare l'ownership.

```

struct lock *
lock_create(const char *name)
{
    struct lock *lock;

    lock = kmalloc(sizeof(*lock));
    if (lock == NULL) {
        return NULL;
    }

    lock->lk_name = kstrdup(name);
    if (lock->lk_name == NULL) {
        kfree(lock);
        return NULL;
    }

    HANGMAN_LOCKABLEINIT(&lock->lk_hangman, lock->lk_name);

    // add stuff here as needed
    // allocazione di semafori

    lock->holder = NULL; // holder inizialmente nessuno
    int initial_value = 1; // 1 per avere semaforo binario
    lock->sem = sem_create("sem_lock", initial_value);

    return lock;
}

```

Nel **lock create** si inizializza la struttura dati, si da un nome al lock. Successivamente, si **imposta l'holder a NULL** e viene creato un **semaforo con initial_value=1** in quanto deve essere appunto binario.

```

void
lock_acquire(struct lock *lock)
{
    /* Call this (atomically) before waiting for a lock */
    HANGMAN_WAIT(&curthread->t_hangman, &lock->lk_hangman);

    // Write this
    KASSERT(!lock_do_i_hold(lock)); // ovvero per acquisire non ci deve essere nessun processo ad avere gia' il lock
    P(lock->sem); // eseguo P() del semaforo associato al lock [decremento S se non ci sono altri thread che hanno gia' acceduto]
    lock->holder = curthread; // associo il thread corrente come quello che ha acquisito il lock

    (void)lock; // suppress warning until code gets written

    /* Call this (atomically) once the lock is acquired */
    HANGMAN_ACQUIRE(&curthread->t_hangman, &lock->lk_hangman);
}

```

Nella **lock acquire** si esegue il **metodo P()**, ovvero wait(), del semaforo associato al lock. Inoltre si pone come holder il curthread in esecuzione sul core.

Il metodo **P()** **decrementa il contatore se esso e' > 0 e permette di entrare nella sezione critica**. Altrimenti, il **processo entra in uno stato di sleep** tramite :

wchan_sleep(sem->sem_wchan, &sem->sem_lock); Questa linea di codice mette il curthread in sleep.

Si nota inoltre come per acquisire un lock non ci devono essere altri thread che hanno l'ownership di quest'ultimo.

```

void
lock_release(struct lock *lock)
{
    /* Call this (atomically) when the lock is released */
    HANGMAN_RELEASE(&curthread->t_hangman, &lock->lk_hangman);

    // Write this
    KASSERT(lock_do_i_hold(lock)); // solamente se qui ci ha acceduto lo stesso thread che ha acquisito il lock inizialmente
    V(lock->sem); // eseguo V() del semaforo associato al lock [incremento S e sveglio altri thread]
    // resetto holder
    lock->holder = NULL;

    (void)lock; // suppress warning until code gets written
}

```

Nella **lock release** si controlla inizialmente che **ad effettuare la release sia lo stesso thread che ha fatto l'acquire**, controllando il campo holder del lock.

Successivamente, si esegue il **metodo V()** del semaforo associato, in modo da incrementare il contatore e permettere ad un altro processo in attesa di essere svegliato e acquisire poi il lock.

LOCKS CON WCHAN

lock create

```

struct lock *
lock_create(const char *name)
{
    struct lock *lock;

    lock = kmalloc(sizeof(*lock));
    if (lock == NULL) {
        return NULL;
    }

    lock->lk_name = kstrdup(name);
    if (lock->lk_name == NULL) {
        kfree(lock);
        return NULL;
    }

    // add stuff here as needed
    lock->lk_wchan = wchan_create(lock->lk_name);
    if (lock->lk_wchan == NULL) {
        kfree(lock->lk_name);
        kfree(lock);
        return NULL;
    }
    lock->lk_owner = NULL;
    spinlock_init(&lock->lk_lock);
    return lock;
}

```

A differenza del caso precedente con i semafori, viene creato il wait_channel che verra' utilizzato come coda di attesa dei thread.

lock acquire

```

void
lock_acquire(struct lock *lock)
{
    // Write this
    KASSERT(lock != NULL);
    if (lock_do_i_hold(lock)) {
        kprintf("AAACK!\n");
    }
    KASSERT(!lock_do_i_hold(lock));

    KASSERT(curthread->t_in_interrupt == false);

    spinlock_acquire(&lock->lk_lock);
    while (lock->lk_owner != NULL) {
        wchan_sleep(lock->lk_wchan, &lock->lk_lock);
    }
    KASSERT(lock->lk_owner == NULL);
    lock->lk_owner = curthread;
    spinlock_release(&lock->lk_lock);
    (void)lock; // suppress warning until code gets written
}

```

Durante la `lock_acquire()` in questo caso, si controlla innanzitutto che colui che sta eseguendo la funzione non abbia già il possesso del lock (mediante `lock_do_i_hold()` NEGATO), in quanto dovrebbe prima fare la `release()`! Fatto questo si aggiunge uno `spinlock()` in modo da accedere in modo mutuamente esclusivo al `wait_channel()`. In particolare, si aspetta nella coda finché c'è qualcuno che ha possesso del lock (`lock->lk_owner` deve essere `!= NULL` affinché possa essere acquistato da un altro thread).

lock release

```

void
lock_release(struct lock *lock)
{
    KASSERT(lock != NULL);
    KASSERT(lock_do_i_hold(lock));
    spinlock_acquire(&lock->lk_lock);
    lock->lk_owner = NULL;
    wchan_wakeone(lock->lk_wchan, &lock->lk_lock);
    spinlock_release(&lock->lk_lock);

    (void)lock; // suppress warning until code gets written
}

```

Nella `lock_release()` si nota come:

- Il thread che la esegue deve essere necessariamente il possessore del lock.
- Si effettua una `spinlock()` in modo da accedere in modo esclusivo sul lock, resettarne l'ownership e svegliare uno dei thread a caso che stava dormendo sul `wait_channel` tramite `wakeone()`.

LOCKS OWNERSHIP

Il thread che ha l'ownership di un lock e' colui che ha chiamato la `lock_acquire()` sul lock ed ha completato l'attesa. Quindi l'ownership non si ha con la creazione del lock e non basta chiamare `lock_acquire()`, bisogna aver superato l'attesa! Questa ownership viene controllata mediante la funzione:

```
bool
lock_do_i_hold(struct lock *lock)
{
    // Write this
#ifdef OPT_SYNCH
    bool res;
    spinlock_acquire(&lock->lk_lock);
    res = lock->lk_owner == curthread;
    spinlock_release(&lock->lk_lock);
    return res;
#endif

    (void)lock; // suppress warning until code gets written

    return true; // dummy until code gets written
}
```

CONDITION VARIABLES

```
struct cv {
    struct wchan *cv_wchan; // la wait channel (coda) su cui i thread possono addormentarsi
    struct spinlock cv_lock; // uno spinlock che protegge l'accesso alla coda interna
};
```

Le condition variable vengono utilizzate in modo da far aspettare un thread finche' non si verifica una certa condizione.

- **wchan (wait channel)** $\Rightarrow$ Rappresenta la coda interna alla condition variable che contiene i thread in attesa. Rappresenta in particolare una primitiva di sincronizzazione del kernel, che consente operazioni di wait e signal connesse a una condizione osservata.
- **spinlock** $\Rightarrow$ Utilizzato per proteggere l'accesso concorrente alla coda wchan.

Le condition variables vengono implementate utilizzando 3 funzioni principali:

- **cv_wait(struct cv *cv, struct lock *lock)** $\Rightarrow$ Il thread deve aver gia' acquisito il lock sulla condition variable **cv**. A questo punto acquisisce mediante spinlock un lock sulla coda, rilascia il lock sulla condition variable e si mette in attesa che un altro thread lo svegli in wchan. E' fondamentale rilasciare il lock prima di entrare in attesa in modo che un altro thread possa acquisire la condition variable, modificarne lo stato e svegliare altri thread. Quando il thread esce dall'attesa (un altro thread ha cambiato la condizione e lo ha svegliato), allora si riacquisisce il lock sulla condition variable.

- **cv_signal(struct cv *cv, struct lock *lock)** ⇒ Risveglia un thread (o piu' con **cv broadcast**) che era in attesa in wchan dopo aver cambiato lo stato della variabile di interesse.

Questo e' quindi un esempio:

```
lock_acquire(mio_lock);
while (!condizione_che_mi_interessa) {
    cv_wait(mia_cv, mio_lock);
}
// qui condizione_che_mi_interessa == true, e il lock è ancora acquisito
/* eseguo operazioni protette dal lock */
lock_release(mio_lock);
```

```
lock_acquire(mio_lock);
/* modifica lo stato globale che fa cambiare la condizione */
cv_signal(mia_cv, mio_lock); // oppure cv_broadcast
lock_release(mio_lock);
```

```
lock_acquire(my_lock);
```

```
while (buffer_is_empty()) {
    cv_wait(my_cv, my_lock); // rilascio lock, attesa
}
```

```
// uso il buffer...
```

```
cv_signal(my_cv, my_lock); // risveglio chi sta aspettando
lock_release(my_lock);
```

Questo e' un **esempio di condition variable**. In particolare, si **entra nella sezione critica solo dopo il verificarsi di una certa condizione**, in questo caso il buffer deve essere vuoto.

In OS161 si implementano:

```
void
cv_wait(struct cv *cv, struct lock *lock)
{
    // Write this
    KASSERT(cv != NULL);
    KASSERT(lock != NULL);
    KASSERT(lock_do_i_hold(lock)); // il thread in esecuzione deve avere il possesso del lock

    struct spinlock *splk = &lock->sem->sem_lock; // ???

    spinlock_acquire(splk); // lock per protezione accesso a coda

    lock_release(lock); // dopo che si ha l'accesso sicuro alla coda si rilascia il lock al curthread (che andra in sleep)

    wchan_sleep(cv->wchan, splk); // si mette in sleep

    spinlock_release(splk); // quando si sveglia si esce dalla coda

    lock_acquire(lock); // riacquisizione del lock

    (void)cv; // suppress warning until code gets written
    (void)lock; // suppress warning until code gets written
}
```

In questo caso, all'**entrata in cv_wait** si acquisisce uno **spinlock** per garantire l'**accesso sicuro alla coda** di attesa (wchan), dove il thread verrà sospeso.

Successivamente, si **rilascia il lock della sezione critica**, in modo da permettere ad altri thread di accedere allo stato

condiviso e potenzialmente modificare la condizione per cui si è in attesa (es. `buffer_is_empty() == false`).

Il thread viene quindi messo in sleep sulla condition variable (wchan). Quando viene risvegliato (es. da un `cv_signal`), **riacquisisce il lock prima di continuare l'esecuzione e accedere nuovamente alla sezione critica.**

```
void
cv_signal(struct cv *cv, struct lock *lock)
{
    // Write this
    KASSERT(cv != NULL);
    KASSERT(lock != NULL);
    KASSERT(lock_do_i_hold(lock)); // il thread in esecuzione deve avere il possesso del lock

    struct spinlock *splk = &lock->sem->sem_lock; // ???????

    // lock_release(lock);
    spinlock_acquire(splk);

    wchan_wakeone(cv->cv_wchan, splk); // si sveglia uno dei thread in sleep (cima alla FIFO)

    spinlock_release(splk);

    // lock_acquire(lock); // riacquisizione del lock

    (void)cv; // suppress warning until code gets written
    (void)lock; // suppress warning until code gets written
}
```

Nella signal invece si sveglia semplicemente uno dei thread in sleep (accedendo sempre a wchan tramite spinlock).

Quindi il **lock** viene utilizzato per acquisire il controllo la **condition variable** e fare un controllo/modifica su quest'ultima. Mentre lo **spinlock** viene utilizzato per accedere in mutua esclusione alla coda dei processi in attesa.

Nella `cwait()` e' fondamentale che lo **spinlock** su wchan avvenga **prima del rilascio del lock** sulla CV di interesse in quanto altrimenti **potrei perdere il `cv_signal()`** eseguito da un altro thread fra quelle due istruzioni, questo perche:

- T1 rilascia il lock sulla CV senza essere ancora in coda.
- T2 nel mentre si trova la CV libera, modifica il suo stato e fa `cv_signal`. T1 pero', non essendo in coda di attesa, non puo' essere svegliato da questa signal.
- T1 entra in attesa inutilmente.

Alla fine l'unica differenza tra wait channel e condition variable è che: nel caso di wait channel la condizione che deve essere osservata viene osservata tramite spinlocks, e nel caso di condition variable tramite locks.

```

- /
void
wchan_sleep(struct wchan *wc, struct spinlock *lk)
{
    /* may not sleep in an interrupt handler */
    KASSERT(!curthread->t_in_interrupt);

    /* must hold the spinlock */
    KASSERT(spinlock_do_i_hold(lk));

    /* must not hold other spinlocks */
    KASSERT(curcpu->c_spinlocks == 1);

    thread_switch(S_SLEEP, wc, lk);
    spinlock_acquire(lk);
}

```

La `wchan_sleep` riceve come parametro lo `spinlock` in quanto la `wchan_sleep` deve rilasciare lo `spinlock` prima di mettere il thread in wait, per poi riacquisirlo subito dopo.

```

void
wchan_wakeone(struct wchan *wc, struct spinlock *lk)
{
    struct thread *target;

    KASSERT(spinlock_do_i_hold(lk));

    /* Grab a thread from the channel */
    target = threadlist_renhead(&wc->threads);

    if (target == NULL) {
        /* Nobody was sleeping. */
        return;
    }

    /*
     * Note that thread_make_runnable acquires a runqueue lock
     * while we're holding LK. This is ok; all spinlocks
     * associated with wchans must come before the runqueue locks,
     * as we also bridge from the wchan lock to the runqueue lock
     * in thread_switch.
     */
    thread_make_runnable(target, false);
}

```

La `wchan_wakeone` invece non ha necessita' del lock, viene utilizzato solo per verificare che colui che ha chiamato tale funzione sia effettivamente proprietario del lock sul wait channel.

N.B ⇒ La `wakeone()` risveglia uno e un solo thread, in alternativa c'è la `wakeall()`.

ESEMPIO DI PRODUTTORE-CONSUMATORE

```

struct prodConsBuf { data_t data;
int dataReady;
struct lock *pc_lk;
struct cv *pc_cv;
};

void producerWrite(struct prodConsBuf *pc, data_t *dp) {
    /* lock for mutual exclusion */
    lock_acquire(pc->pc_lk);
    /* properly handle here the fact that the buffer could be full
     (filled by the other producer and not yet read by the

```

```

consumer) */
    while(pc->dataReady == 1){ // finche il buffer e' pieno
        cv_wait(pc->pc_cv, pc->pc_lk); // entro nella coda di
        attesa
    }
    /* copy data: done here by simple assignment -
       a proper function could be used instead */
    pc->data = *dp;
    pc->dataReady = 1; /* set flag */
    /* signal the consumer */
    cv_signal(pc->pc_cv, pc->pc_lk); // ho un solo consumatore
    quindi signal
    /* just release the lock and return */
    lock_release(pc->pc_lk);
}

void consumerRead(struct prodConsBuf *pc, data_t *dp) {
    /* lock for mutual exclusion */
    lock_acquire(pc->pc_lk);
    while (!pc->dataReady) {
        cv_wait(pc->pc_cv, pc->pc_lk);
    }
    /* copy data: done here by simple assignment -
       a proper function could be used instead */
    *dp = pc->data;
    pc->dataReady = 0; /* reset set flag */
    /* signal a producer waiting, if any */
    cv_broadcast(pc->pc_cv, pc->pc_lk); // ho due produttori quindi
    broadcast
    /* release the lock and return */
    lock_release(pc->pc_lk);
}

```

2.10 Waitpid system call

Quando un processo termina la sua esecuzione e richiama la **systemcall "sys_exit"** con la normale implementazione accade

questo:

```
void
sys__exit(int status)
{
    /* get address space of current process and destroy */
    struct addrspace *as = proc_getas();
    as_destroy(as);
    /* thread exits. proc data structure will be lost */
    thread_exit();

    panic("thread_exit returned (should not happen)\n");
    (void) status; // TODO: status handling
}
```

In questo caso la **struct proc** del processo viene persa in quando l'**address space** rilasciato. Quello che si vuole fare è modificare questa `sys_exit()` in modo tale da segnalare la fine del processo, terminare il thread, ma senza distruggere la `struct proc`.

La `waitpid()` viene utilizzata per far sì che un **processo padre** **attenda il cambio di stato** (es. **terminazione**) di un **processo figlio** dato il **PID**.

L'**obiettivo** è quello di, dopo che un figlio chiama `thread_exit()` e resta in stato **zombie** (in quanto la **struct proc non distrutta**), far sì che sia il **processo padre** a **distruggere struct proc** con `waitpid()`. Quindi:

- processo figlio con `PID=123` in esecuzione.
- padre chiama `waitpid(123)` e attende la terminazione del figlio.
- figlio termina e segnala tramite semaforo o `cv_signal()` la fine della sua esecuzione.
- padre distrugge il figlio ed esegue quello che deve fare.

Per **implementare la waitpid** bisogna seguire una serie di passaggi:

- **Creazione di una process table** → Struttura globale per memorizzare e gestire i processi attivi tramite `PID`.
- **Funzione `proc_search_pid(pid)`** → Restituisce il puntatore al processo corrispondente a un dato `PID`.
- **Funzione `proc_init_waitpid(struct proc *p)`** → Inserisce il processo nella process table e assegna un `PID` univoco.
- **Modifica di `proc_create`** → Integra `proc_init_waitpid` per inizializzare correttamente il nuovo processo.
- **Funzione `proc_end_waitpid(struct proc *p)`** → Rimuove il processo dalla process table e libera le risorse di sincronizzazione.
- **Modifica di `proc_destroy`** → Chiama `proc_end_waitpid` per pulire correttamente alla terminazione di un processo.

- Implementazione `proc_wait(proc)` →
- Implementazione della syscall `sys_getpid` → Restituisce al processo utente il proprio PID.
- Implementazione della syscall `sys_waitpid(pid, status_ptr, options)` → Attende che il processo con PID specificato termini, e restituisce il suo codice di uscita.
- Gestione doppia di `proc_remthread` → Risolve il problema della doppia rimozione del thread dal processo che causava kernel panic.

PROCESS TABLE

```

55  #define MAX_PROC 100
56  static struct _processTable {
57      int active;           /* initial value 0 */
58      struct proc *proc[MAX_PROC+1]; /* [0] not used. pids are >= 1 */
59      int last_i;          /* index of last allocated pid */
60      struct spinlock lk;  /* Lock for this table */
61  } processTable;

```

Obiettivo ⇒ Innanzitutto e' necessario **associare un processo al suo PID** e **capire se c'e' un processo attualmente attivo con quell'identificatore**. Per fare ciò si crea una **struct processTable** accessibile ovunque e STATICA.

How To ⇒ Questa processTable contiene innanzitutto una variabile spinlock, in modo tale che le **modifiche** a questa struttura statica siano eseguite da un solo processo in modo **mutualmente esclusivo**.

Successivamente si crea un array di processi (struct proc) che vengono acceduti tramite il loro PID (ovvero l'indice dell'array). **Questo significa che se l'elemento i-esimo dell'array e' NULL allora non c'e' nessun processo ATTIVO con il PID = i.**

Inoltre si salva l'ultimo pid assegnato ad un processo (in variabile last_i) in modo da capire quale PID assegnare ad un nuovo processo attivo che viene inserito nella tabella.

PROC_SEARCH_PID(PID) -> (proc)

```

struct proc *
proc_search_pid(pid_t pid) {
#ifdef OPT_WAITPID
    struct proc *p;
    KASSERT(pid>=0&&pid<MAX_PROC);
    p = processTable.proc[pid];
    KASSERT(p->p_pid==pid);
    return p;
#else
    (void)pid;
    return NULL;
#endif
}

```

Obiettivo ⇒ Si vuole capire se il processo con PID = pid e' attualmente attivo. **Ritorna il puntatore al processo se lo e', altrimenti ritorna NULL.**

How to ⇒ Dato un PID che viene passato come parametro, restituisce il processo (puntatore struct proc) associato. Per fare cio' si **restituisce appunto l'elemento in indice=PID dell'array processTable.proc[PID]** mostrato in precedenza.

PROC_INIT_WAITPID(PROC, NAME_SEMAPHORE)

```

89  * Initialize support for pid/waitpid.
90  */
91  static void
92  proc_init_waitpid(struct proc *proc, const char *name) {
93  #if OPT_WAITPID
94      /* search a free index in table using a circular strategy */
95      int i;
96      spinlock_acquire(&processTable.lk);
97      i = processTable.last_i+1;
98      proc->p_pid = 0;
99      if (i>MAX_PROC) i=1;
100     while (i!=processTable.last_i) {
101         if (processTable.proc[i] == NULL) {
102             processTable.proc[i] = proc;
103             processTable.last_i = i;
104             proc->p_pid = i;
105             break;
106         }
107         i++;
108         if (i>MAX_PROC) i=1;
109     }
110     spinlock_release(&processTable.lk);
111     if (proc->p_pid==0) {
112         panic("too many processes. proc table is full\n");
113     }
114     proc->p_status = 0;
115 #if USE_SEMAPHORE_FOR_WAITPID
116     proc->p_sem = sem_create(name, 0);
117 #else
118     proc->p_cv = cv_create(name);
119     proc->p_lock = lock_create(name);
120 #endif
121 #else
122     (void)proc;
123     (void)name;
124 #endif
125 }

```

Obiettivo ⇒ **Assegnare un PID univoco ad un processo** appena creato in modo che, fintantoche' esso risulta essere attivo, sia **memorizzato in processTable.proc[...]**.

How to ⇒ Innanzitutto si **acquisisce il lock** sulla processTable (per accesso mutualmente esclusivo su di essa). Fatto ciò si **ricava il PID da assegnare** al nuovo processo incrementando il contatore relativo al PID assegnato al processo appena precedente.

Successivamente si **scorre processTable.proc[] e si inserisce** il nuovo processo nella posizione relativa al PID che gli e' stato assegnato.

Si rilascia il lock e si **crea un semaforo** per il nuovo processo.

Questo semaforo viene utilizzato più avanti nella funzione `proc_wait()`; in ogni caso, tale funzione fa `P()` sul semaforo di tale processo e il padre rimane bloccato fino a quando non avviene il signal `S()` (ovvero in `sys_exit`, alla terminazione del processo).

`PROC_END_WAITPID(PROC)`

```
131 static void
132 proc_end_waitpid(struct proc *proc) {
133     #if OPT_WAITPID
134         /* remove the process from the table */
135         int i;
136         spinlock_acquire(&processTable.lk);
137         i = proc->p_pid;
138         KASSERT(i>0 && i<=MAX_PROC);
139         processTable.proc[i] = NULL;
140         spinlock_release(&processTable.lk);
141     #if USE_SEMAPHORE_FOR_WAITPID
142         sem_destroy(proc->p_sem);
143     #else
144         cv_destroy(proc->p_cv);
145         lock_destroy(proc->p_lock);
146     #endif
147     #else
148         (void)proc;
149     #endif
150 }
151 }
```

Obiettivo ⇒ Rimuove un processo dalla `processTable` e ne distrugge il semaforo relativo. Viene richiamata alla terminazione del processo.

How to ⇒ Si acquisisce il lock sulla `processTable`, si ricava il PID associato al processo e si mette `processTable.proc[PID] = NULL`. Si rilascia il lock e si cancella il semaforo associato al processo.

`PROC_CREATE(NAME)`

```

153  /*
154  * Create a proc structure.
155  */
156  static
157  struct proc *
158  proc_create(const char *name)
159  {
160      struct proc *proc;
161
162      proc = kmalloc(sizeof(*proc));
163      if (proc == NULL) {
164          return NULL;
165      }
166      proc->p_name = kstrdup(name);
167      if (proc->p_name == NULL) {
168          kfree(proc);
169          return NULL;
170      }
171
172      proc->p_numthreads = 0;
173      spinlock_init(&proc->p_lock);
174
175      /* VM fields */
176      proc->p_addrspace = NULL;
177
178      /* VFS fields */
179      proc->p_cwd = NULL;
180
181      ⚡ proc_init_waitpid(proc, name);
182
183      return proc;
184  }

```

Obiettivo ⇒ Funzione già presente, si modifica in modo tale che, alla creazione di un nuovo processo, essa venga inserito nella processTable richiamando la funzione definita in precedenza `PROC_INIT_WAITPID(PROC, NAME_SEMAPHORE)`.

How to ⇒ Si aggiunge semplicemente alla funzione `proc_create()` la chiamata a `proc_init_waitpid(proc, name)`.

PROC_DESTROY(PROC)

Analogamente al caso precedente, si aggiunge a tale funzione già presente una chiamata alla funzione definita in precedenza

`PROC_END_WAITPID(PROC)`

SYSCALL.C

Al file `syscall.c` si aggiungono le system call che dovranno essere implementate, ovvero `sys_waitpid(PID)` e `sys_getpid()`.

```

case SYS_exit:
    /* TODO: just avoid crash */
    sys_exit((int)tf->tf_a0);
    break;
case SYS_waitpid:
    retval = sys_waitpid((pid_t)tf->tf_a0,
        (userptr_t)tf->tf_a1,
        (int)tf->tf_a2);
    if (retval<0) err = ENOSYS;
else err = 0;
    break;
case SYS_getpid:
    retval = sys_getpid();
    if (retval<0) err = ENOSYS;
else err = 0;
    break;

```

Si nota come `sys_waitpid()` necessiti di tre parametri (che vengono come al solito passati tramite i vari argomenti del `trapframe`) che verranno discussi a breve.

SYS_GETPID() -> (pid_t)

```

70  pid_t
71  sys_getpid(void)
72  {
73  #if OPT_WAITPID
74      KASSERT(curproc != NULL);
75      return curproc->p_pid;
76  #else
77      return -1;
78  #endif
79  }

```

Systemcall che si occupa semplicemente di restituire il PID del processo attualmente in esecuzione tra quelli attivi (un solo processo può essere in RUN per volta).

PROC_WAIT(PROC) -> (int)

```

431 int
432 proc_wait(struct proc *proc)
433 {
434     #if OPT_WAITPID
435         int return_status;
436         /* NULL and kernel proc forbidden */
437         KASSERT(proc != NULL);
438         KASSERT(proc != kproc);
439
440         /* wait on semaphore or condition variable */
441         #if USE_SEMAPHORE_FOR_WAITPID
442             P(proc->p_sem);
443         #else
444             lock_acquire(proc->p_lock);
445             cv_wait(proc->p_cv);
446             lock_release(proc->p_lock);
447         #endif
448         return_status = proc->p_status;
449         proc_destroy(proc);
450         return return_status;
451     #else
452         /* this doesn't synchronize */
453         (void)proc;
454         return 0;
455     #endif
456 }

```

Funzione che, dato un processo in input, effettua una wait sul valore del semaforo del processo in input. Questo significa che il chiamante (processo padre) rimane in attesa fino a quando il processo in input non esegue Signal()!

Questo accade quando il processo termina ad esempio.

Successivamente, quando il processo termina e la sys_exit() esegue appunto Signal() il processo chiamante che ha eseguito la proc_wait() legge lo stato del processo figlio e lo distrugge!. Alla fine restituisce lo stato.

SYS_WAITPID(PID, STATUS, OPTIONS) -> (int)

```

50 int
51 sys_waitpid(pid_t pid, userptr_t statusp, int options)
52 {
53     #if OPT_WAITPID
54         struct proc *p = proc_search_pid(pid);
55         int s;
56         (void)options; /* not handled */
57         if (p==NULL) return -1;
58         s = proc_wait(p);
59         if (statusp!=NULL)
60             *(int*)statusp = s;
61         return pid;
62     #else
63         (void)options; /* not handled */
64         (void)pid;
65         (void)statusp;
66         return -1;
67     #endif
68 }

```

Si occupa di richiamare la funzione `PROC_SEARCH_PID(PID)` illustrata in precedenza, in modo tale da ricavare il puntatore al processo associato al PID ricevuto in input (pippo).

Successivamente richiama la funzione `proc_wait(pippo)` in modo che, il chiamante attenda la terminazione del processo pippo prima di distruggerlo (in modo che non rimanga zombie).

`SYS_EXIT(status)`

```
22  /*
23  * system calls for process management
24  */
25  void
26  sys__exit(int status)
27  {
28  #if OPT_WAITPID
29      struct proc *p = curproc;
30      p->p_status = status & 0xff; /* just lower 8 bits returned */
31      proc_remthread(curthread);
32  #if USE_SEMAPHORE_FOR_WAITPID
33      V(p->p_sem);
34  #else
35      lock_acquire(p->p_lock);
36      cv_signal(p->p_cv);
37      lock_release(p->p_lock);
38  #endif
39  #else
40      /* get address space of current process and destroy */
41      struct addrspace *as = proc_getas();
42      as_destroy(as);
43  #endif
44      thread_exit();
45
46      panic("thread_exit returned (should not happen)\n");
47      (void) status; // TODO: status handling
48  }
```

La `SYS_exit` va appunto modificata, in modo che non distrugga il processo.

In particolare, **si ricava l'attuale processo in esecuzione e se ne modifica lo stato** (viene passato come parametro alla `exit`) memorizzando solo il byte meno significativo.

Successivamente **si fa la `Signal()`** in modo che, come detto sopra, **il processo che ha chiamato `proc_wait()` sappia che il processo figlio sta terminando**. Si richiama successivamente `thread_exit`, la quale viene modificata in modo che `proc_remthread()` sia coerente con il `proc_remthread` aggiunto

nella `sys_exit()`. `proc_remthread()` e' stato aggiunto in `sys_exit()` in modo da evitare che il processo padre distrugga il figlio prima che esso chiami `thread_exit()`.

Infatti si aggiunge questo a `thread_exit`:

```

778 void
779 thread_exit(void)
780 {
781     struct thread *cur;
782
783     cur = curthread;
784
785     /*
786      * Detach from our process. You might need to move this action
787      * around, depending on how your wait/exit works.
788      */
789     if (cur->t_proc != NULL) {
790         proc_remthread(cur);
791     }
792
793     /* Make sure we *are* detached (move this only if you're sure!) */
794     KASSERT(cur->t_proc == NULL);
795
796     /* Check the stack guard band. */
797     thread_checkstack(cur);
798
799     /* Interrupts off on this processor */
800     splhigh();
801     thread_switch(S_ZOMBIE, NULL, NULL);
802     panic("braaaaaa!!!!!!!!!!!!!!!!sssss\n");
803 }

```

Quindi ora la `thread_exit()` necessita del puntatore al **processo** per accedere all'address space e rilasciarlo. Quindi non posso richiamare `proc_destroy()` nella `sys_exit`.

Rappresentazione semplificata della `sys_exit()`

```

my_sys_exit(int status) { // lo stato viene ricevuto come parametro
    struct proc *p = curproc; // serve per poter accedere al processo dopo averlo
                                // staccato da curthread (curproc non più valido)

    p->p_status = status; // salva lo stato di ritorno per la waitpid

    proc_remthread(curthread); // stacca il processo dal thread

    // segnala al processo che fa la waitpid (per usare cv_signal è
    // necessario possedere il relativo lock)
    lock_acquire(p->p_lock);
    cv_signal(p->p_cv, p->p_lock);
    lock_release(p->p_lock);

    // meglio NON fare as_destroy qui (lo farà la proc_destroy)
    // il thread finisce qui (diventa zombie)
    thread_exit();
}

```

La chiamata dovrà essere del tipo:

```
my_sys_exit((int)tf->tf_a0);
```

Funzione `my_sys_exit(int status)`:

- Lo status viene ricevuto come parametro della system call.

2.11 USER PROCESS (FILE ELF)

Un **file ELF (Executable and Linkable Format)** e' un tipo di file utilizzato per rappresentare un programma compilato (contiene codice e dati in formato binario). Il file ELF relativo ad un programma viene **creato al momento della compilazione del programma**. L'interno di un file ELF e' **organizzato in segmenti** (pezzi separati del codice o dati globali) che il kernel utilizza per creare lo **spazio di indirizzamento del processo**. OS/161 assume che ogni file ELF contenga due segmenti principali detti **"LOAD"**:

- **Text segment** ⇒ codice eseguibile (sezioni .text, .rodata).
- **Data segment** ⇒ dati globali inizializzati e non (sezioni .data, .bss).

OS/161 non si aspetta che l'ELF descriva lo spazio di stack; l'implementazione "dumbvm" creerà autonomamente per ogni processo uno stack di lunghezza fissa (12 pagine, terminante all'indirizzo virtuale 0x7fffffff).

OS/161 utilizza una struttura **struct uio** per effettuare **trasferimenti di dati ("User I/O") tra file e memoria** (kernel o user), astratta tramite chiamate VOP.

```
struct uio {
    struct iovec *uio_iov;
    unsigned uio_iovcnt;
    off_t uio_offset;
    size_t uio_resid;
    enum uio_seg uio_segflg;
    enum uio_rw uio_rw;
    struct addrspace *uio_space;
};
struct iovec {
    void *iov_base;
    size_t iov_len;
};
```

La iovec rappresenta un singolo buffer usato in operazioni di I/O dove:

- **iov_base**: indirizzo di memoria dove leggere o scrivere i dati.
- **iov_len**: numero di byte da leggere/scrivere in quel buffer.

La struct uio viene usata per passare tutti i parametri a funzioni di basso livello di lettura come VOP_READ. Questa struct contiene:

- **struct iovec* uio_iov** ⇒ array di buffer per la scrittura/lettura di dati su file.

- **uio_iovcnt** ⇒ Numero di buffer che si stanno utilizzando.
- **offset** ⇒ Offset nel file dove si vuole iniziare a leggere/scrivere.
- **resid** ⇒ Numero di elementi rimanenti da leggere/scrivere.
- **uio_segflg** ⇒ Indica se i dati di interesse all'IO sono nello spazio kernel o nello spazio user. Nel caso di user si specifica ulteriormente se e' segmento dati o codice. Questo significa che:
 - Se si effettua una **READ()** normale ad esempio allora buf_ptr e' nello spazio utente, quindi segflg = **UIO_USERSPACE**.
 - Se nella READ() vado ad utilizzare un **buffer kernel** per poi effettuare il copyout() allora segflg = **UIO_SYSSPACE**!
- **uio_rw** ⇒ Specifica se l'operazione deve essere lettura o scrittura.
- **uio_space** ⇒ Serve se uio_segflg fa riferimento a segmento user e indica l'address space da usare per la conversione logico-fisica degli indirizzi.

Nello specifico **per leggere parti di un file ELF** il sistema operativo:

- Costruisce un iovec che punta a un buffer in memoria.
- Invoca **uio_kinit(&iov, &ku, <buf>, <size>, <offset>, UIO_READ)** per preparare la struttura struct uio (e l'iovec) in modo che VOP_READ sappia leggere <size> byte da <offset> del file denotato da struct vnode *v. Quindi si creano le struct vuote **iovec e uio**. Si passano come parametro a uio_kinit() per "riempirle". Poi si richiama VOP_READ(v, &uio) per effettuare la lettura effettiva dei dati dal disco.

La funzione **VOP_READ(vnode, &uio)** ⇒ Si occupa di leggere dal file (rappresentato con vnode) in base alle informazioni descritte nella struct uio. Il **vnode** nello specifico rappresenta il **FILE CONTROL BLOCK del file ELF da cui si legge**. (da prova d'esame 2019)

La lettura da file da parte di un processo utente puo' avvenire quindi richiamando una sys_read() fatta in questo modo:

```
int sys_read(int fd, userptr_t buf, size_t size) {
    struct vnode *v;
    int result = 0;
    char *p = (char *)buf_ptr;
```

```

if (fd == STDIN_FILENO){
    int i = 0;
    for (i=0; i < (int)size; i++){
        p[i] = getch();
        if(p[i] < 0){
            return i;
        }
    }
}else{
    v = getVNodeFromDescriptor(fd); // dato il file // descriptor ottengo il vnode
    (rappresentazione del file)
    struct iovec iov;
    struct uio u;
    uio_init(&iov, &u, buf, size, 0, UIO_READ);
    result = VOP_READ(v, &u);
}

return result;
}

```

<pre> void uio_init (struct iovec *iov, struct uio *u, void *kbuf, size_t len, off_t pos, enum uio_rw rw) { iov->iov_kbase = kbuf; iov->iov_len = len; u->uio_iov = iov; u->uio_iovcnt = 1; u->uio_offset = pos; u->uio_resid = len; u->uio_segflg = UIO_SYSSPACE; u->uio_rw = rw; u->uio_space = NULL; } int load_elf (struct vnode *v, vaddr_t *entrypoint) { Elf_Ehdr eh; /* Executable header */ Elf_Phdr ph; /* "Program header" = segment header */ int result; struct iovec iov; struct uio ku; ... uio_init(&iov, &ku, &eh, sizeof(eh), 0, UIO_READ); result = VOP_READ(v, &ku); ... } </pre>	<pre> load_segment (struct addrspace *as, struct vnode *v, off_t offset, vaddr_t vaddr, size_t memsize, size_t filesize, int is_executable) { struct iovec iov; struct uio u; int result; iov.iov_ubase = (userptr_t)vaddr; iov.iov_len = memsize; // length of the memory space u.uio_iov = &iov; u.uio_iovcnt = 1; u.uio_resid = filesize; // amount to read from the file u.uio_offset = offset; u.uio_segflg = is_executable ? UIO_USERISPACE : UIO_USERSPACE; u.uio_rw = UIO_READ; u.uio_space = as; result = VOP_READ(v, &u); ... } </pre>
---	--

La funzione **LOAD_ELF()**, viene utilizzata per **caricare un programma eseguibile sottoforma di file ELF nello spazio di indirizzamento di un processo**. In particolare:

- Legge **ELF HEADER** per ottenere informazioni su quali segmenti caricare e dove.
- Usa ogni **program header**, quindi ogni header di ogni segmento da caricare, per **definire le regioni di memoria virtuale**.
- Allocare memoria fisica e richiamando **LOAD_SEGMENT()** carica i dati dal file ELF in memoria.

Questa funzione utilizza **UIO_SYSSPACE** in quanto vengono letti ELF HEADER e PROGRAM HEADER in variabili kernel, in quanto servono al S.O per capire dove andare a leggere i dati I/O.

LOAD_SEGMENT() invece serve quindi per **caricare un singolo elemento del file ELF in memoria**. Questa utilizza UIO_USERISPACE/UIO_USERSPACE in quanto si occupa di caricare i dati da disco a spazio di indirizzamento del processo utente (in base a quello deciso in LOAD_ELF()).

```

load_elf(struct vnode *v, vaddr_t *entrypoint) {
    Elf_Ehdr eh;    /* Executable header */
    Elf_Phdr ph;    /* "Program header" = segment header */
    int result, i;
    struct iovec iov;
    struct uio ku;
    struct addrspace *as;
    as = proc_getas();
    /* Read the executable header from offset 0 in the file. */
    uio_kinit(&iov, &ku, &eh, sizeof(eh), 0, UIO_READ);
    result = VOP_READ(v, &ku);
    /*Go through the list of segments and set up the address space.*/
    for (i=0; i<eh.e_phnum; i++) {
        off_t offset = eh.e_phoff + i*eh.e_phentsize;
        uio_kinit(&iov, &ku, &ph, sizeof(ph), offset, UIO_READ);
        result = VOP_READ(v, &ku);
        ...
        result = as_define_region(as,
            ph.p_vaddr, ph.p_memsz,
            ph.p_flags & PF_R,
            ph.p_flags & PF_W,
            ph.p_flags & PF_X);
    }
    result = as_prepare_load(as);
    /* Now actually load each segment. */
    for (i=0; i<eh.e_phnum; i++) {
        off_t offset = eh.e_phoff + i*eh.e_phentsize;
        uio_kinit(&iov, &ku, &ph, sizeof(ph), offset, UIO_READ);
        result = VOP_READ(v, &ku);
        result = load_segment(as, v, ph.p_offset, ph.p_vaddr,
            ph.p_memsz, ph.p_filesz,
            ph.p_flags & PF_X);
    }
    result = as_complete_load(as);
    *entrypoint = eh.e_entry;
    return 0;
}

```

2.11.1 Flusso creazione di processi utente

Il flusso di creazione di un programma utente si configura in:

- Si parte dal menu kernel (riga di comando) quando si esegue il comando "p /bin/hello" ad esempio (dove hello e' il file ELF).
- A seguito di cio' viene richiamata la funzione `common_prog()`:

```
int common_prog (int nargs, char **args) {
    struct proc *proc;
    int result;

    /* Create a process for the new program to run in. */
    proc = proc_create_runprogram(args[0] /* name */);
    ...
    result = thread_fork(args[0] /* thread name */,
        proc /* new process */, cmd_progthread /* thread function */,
        args /* thread arg */, nargs /* thread arg */);
    ...
    /* TODO: wait for process termination */
    ...
    return 0;
}
```

- Questa funzione:
 - Si occupa della creazione della struct proc che rappresenta il PCB (Process Control Block) e contiene informazioni come PID, struttura base del processo e contenitore per il thread.

```
struct proc {
    char *p_name;                /* Name of this process */
    struct spinlock p_lock;      /* Lock for this structure */
    unsigned p_numthreads;       /* Number of threads in this process */

    /* VM */
    struct addrspace *p_addrspace; /* virtual address space */

    /* VFS */
    struct vnode *p_cwd;         /* current working directory */

    /* add more material here as needed */
    ...
};
```

- Ritorna quindi poi il puntatore a questa struttura.
- Successivamente si crea un thread associato al nuovo processo mediante la `thread_fork()` e si passa il puntatore al PCB in modo da collegare il thread a processo.
- La `thread_fork()` (vedi 2.5) richiama `cmd_progthread()` che a sua volta **richiama RUNPROGRAM**, che effettua:
 - **vfs_open()** ⇒ Apre il file ELF "hello" quindi passato come arg della `common_prog()`.
 - **as_create()** (vedi 2.8.1) ⇒ Crea nuovo spazio di indirizzamento del processo.
 - **proc_setas(as) e as_activate()** in modo da settare il nuovo processo creato creato come processo attivo.
 - **load_elf()** ⇒ Dato il programma "hello", legge l'header ELF principale (`Elf_Ehdr`) e l'elenco dei segmenti (`Elf_Phdr`). Recupera lo spazio di indirizzamento (`addrspace`) del processo attivo tramite `proc_getas()`. Per ciascun segmento del file

ELF, chiama `as_define_region()` per definire nella `addrspace` le regioni di memoria corrispondenti (es. codice, dati). Una volta preparato lo spazio, i dati effettivi del file ELF vengono copiati in memoria tramite `load_segment()`, che scrive fisicamente nei frame associati agli indirizzi definiti (es. `as->vbase1, as->vbase2`).

- **`vfs_close(as)`**
- **`as_define_stack(as, &stackptr)`** $\Rightarrow$ Alloca regione di memoria virtuale per lo stack e imposta lo stack pointer in cima a questa regione.
- **`enter_new_process()`** $\Rightarrow$ Il processo a questo punto e' pronto, bisogno solo "convertirlo da kernel a utente". Questa funzione si occupa di passare da diritti kernel a diritti utente per il nuovo processo creato. Questa funzione crea un trapframe vuoto (mediante `bzero()`) in modo da far finta che il processo sia tornato da un'eccezione. A questo punto si impostano i registri necessari del trapframe e si esegue la funzione assembler `mips_usermode(&tf)` che si occupa di passare da privilegi kernel ad utente. E' in questo punto che c'e' il "passaggio del processo utente da KSEG0 a KUSEG".

```
void enter_new_process(int argc, userptr_t argv, userptr_t env,
                      vaddr_t stack, vaddr_t entry) {
    struct trapframe tf;

    bzero(&tf, sizeof(tf));

    tf.tf_status = CST_IRQMASK | CST_IEp | CST_KUp;
    tf.tf_epc = entry;
    tf.tf_a0 = argc;
    tf.tf_a1 = (vaddr_t)argv;
    tf.tf_a2 = (vaddr_t)env;
    tf.tf_sp = stack;

    mips_usermode(&tf);
}

void mips_usermode(struct trapframe *tf) {
    ...
    /* This actually does it. See exception-*.S. */
    asm_usermode(tf);
}
```

3 VIRTUAL MEMORY

3.1 Introduzione

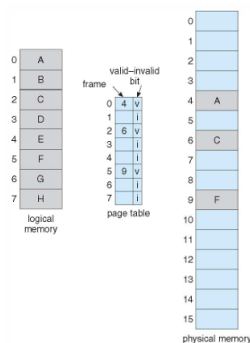
La **memoria virtuale** (o logica) separata dalla fisica permette l'esecuzione di processi che non hanno tutte le pagine in memoria primaria, grazie appunto allo **swapping**. Permette di

eseguire programmi che sono piu' grandi della memoria fisica e di implementare memoria condivisa (librerie e spazio di indirizzi condivisi tra piu' processi).

La memoria virtuale puo' essere **implementata** in due modi:

- **Demand paging.**
- Demand segmentation.

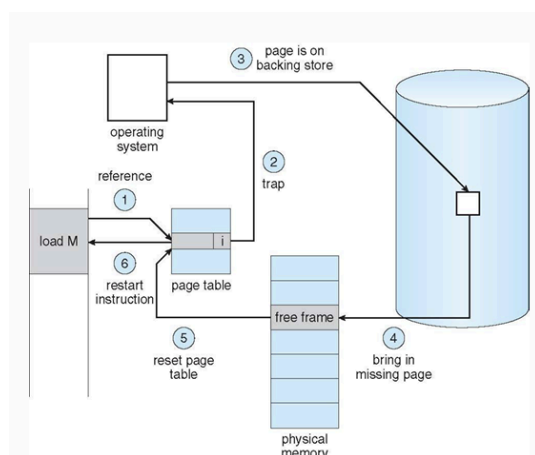
3.2 Demand Paging



Quando la CPU non trova l'istruzione nel TLB o in memoria primaria effettua un **Demand Paging**, per portare la pagina corretta in memoria. Per **controllare se la pagina e' in memoria** si controlla appunto l'indirizzo nella page table. Visto che non e' detto che abbia tutte le pagine in memoria, si **usa un valid-invalid bit nelle page-table** per indicare se la pagina i-esima e' presente in RAM.

Se una **pagina non viene trovata** (invalid) si ha **PAGE FAULT**.

3.2.1 Gestione Page Fault (Best case)



- Si controlla se la pagina e' presente in RAM andando in **Page Table**.

- Se il bit associato alla pagina e' **invalid** e non e' in memoria (trap) allora bisogna fare un **page-in**.

- Se in RAM **c'e' un frame libero** (best case) allora si fa **page-in** per caricare il frame da disco alla RAM.

- Si **aggiorna la page table** del processo con valid e si mette l'indirizzo del frame nel quale la pagina e' stata caricata.

- Si ripristina l'esecuzione dell'istruzione che era stata interrotta.

L'esecuzione di un istruzione di un processo potrebbe **richiedere di accedere a piu' pagine**. Nel caso **PURE DEMAND**

PAGING il processo parte con **zero pagine nella memoria**, ogni volta che non si trova la pagina desiderata si ha un page fault e si usa un meccanismo di **localita' spaziale e temporale** in modo da **evitare una cascata di page fault**.

Con metodo **PURE DEMAND PAGING** e **localita'** si hanno **piu' page fault all'inizio** fino ad arrivare a **regime** dove si avra' un **basso** numero.

Ogni sistema operativo ha una **LISTA DI FRAME LIBERI**, quando c'e' un page fault si trova facilmente un frame libero. I frame liberi sono **allocati scrivendo tutti 0 nella memoria**, in modo da garantire protezione.

Quando il numero di elementi nella lista scende sotto una certa soglia, il S.O. inizia a fare page-out per evitare di dover fare page-out dopo.

Dato p = **probabilita' di avere page fault**

$$\text{Effective Access time} = (1-p) * m_a + p * \text{page fault time}$$

Non si tiene conto dell'accesso alla page table in questo caso.

- $(1-p) \Rightarrow$ Non c'e' stato page fault.
- $p \Rightarrow$ c'e' stato page fault.

3.2.2 Gestione Page Fault (Worst Case)

- Si salva lo **stato dei registri** e lo stato del processo.
- Si **controlla** se l'**invalid bit** faceva riferimento a page fault o operazione illegale.
- Si controlla la locazione della pagina su disco.
- **Aspetta il suo turno** in una coda per iniziare la lettura su disco. Appena arriva il suo turno fa **page-in**.
- **Se non ci sono frame liberi**, bisogna scegliere un **victim-frame** da togliere dalla RAM prima di fare page-in ed entrare eventualmente **in coda** per fare page-out del victim frame scelto.
- Una volta fatto page-in, ripristina lo stato dei registri e del processo.
- Aggiorna il contenuto della page table.

L'**access time** diventa quindi:

$$EAT = (1 - p) \times \text{memory access} + p (\text{page fault overhead} + \text{swap page out} + \text{swap page in})$$

Memory access time = 200 nanoseconds

Average page-fault service time = 8 milliseconds

$$EAT = (1 - p) * 200 + p (8 \text{ milliseconds}) = (1 - p) * 200 + p * 8,000,000 = 200 + p * 7,999,800$$

L'access time e' quindi **proporzionale al page-fault rate**.

3.3 Copy-on-Write -> tecnica per tenere basso page-fault

Utilizzando la chiamata di sistema **fork()**, il processo **padre e figlio** hanno inizialmente accesso alla **stessa zona di memoria** (**Copy-on-Write**) fino a quando uno dei due non la modifica, questo permette di avere **meno page-fault**. A seguito di una **modifica** il **contenuto** della memoria viene **duplicato**, quindi non fanno piu' riferimento a stessa area.

3.4 Page Replacement

Se **non ci sono frame liberi** quando si fa page-in a causa di un page-fault bisogna quindi **capire quale frame liberare** (page-out).

Come detto prima in questo caso devo fare **due operazioni I/O**:

- Se non ci sono frame liberi si sceglie quindi un **victim frame** e si fa page out di quest'ultimo (**si scrive il victim su disco**).
- Si fa page-in e si aggiorna page table (**si scrive su RAM**).

Per **ridurre il numero di operazioni I/O** si aggiunge un **dirty bit** alla page table.

- **Se dirty bit = 0**, allora la pagina non e' stata modificata mentre era in RAM, quindi non ho bisogno di scrivere su disco. **Questo mi permette di fare solo page-in in quanto sovrascivo direttamente il victim frame!**
- **Se dirty bit = 1**, allora la pagina e' stata modificata e devo aggiornare il contenuto su disco.

3.5 Algoritmi di Replacement

Nel **demand paging** ci sono **due problemi**:

- Se ci sono piu' processi in memoria, bisogna capire quante pagine allocare per ogni processo inizialmente.
- Come scegliere il victim frame per avere il **page-fault rate minore**.

Per confrontare due algoritmi di replacement uso una **reference string** e computo il numero di page fault su quella stringa per ogni algoritmo.

7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1

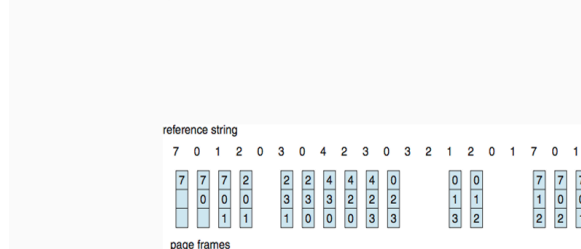
Maggiore e' il numero di frame della memoria, minore sara' il numero di page-faults (di base).

Si utilizzano quindi delle strutture dati (come code) per memorizzare i frame allocati e capire quale scegliere come victim frame. La page table viene aggiornata successivamente.

3.5.1 First-In-First-Out (FIFO)

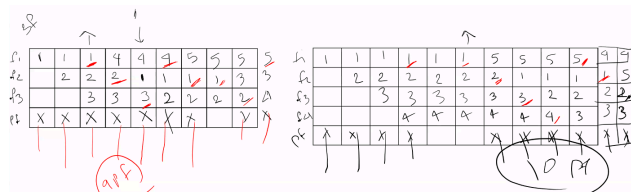
Algoritmo tale che si crea una coda con i frame che entrano in memoria e **la prima pagina che inserisco in memoria sara' la prima ad uscire.**

Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1**



Dall'esempio si nota come tra il quinto e settimo passo faccio page-out di 0 e poi devo subito fare page-in sempre di 0. Questo e' uno dei **limiti dell'algoritmo FIFO.**

Il metodo FIFO e' contraddistinto dall'**ANOMALIA DI BELADY**, in quanto il **numero di page fault per 4 frame** disponibili ad esempio e' **maggiore del numero di page fault di quando si hanno 3 frame** disponibili in memoria.



L'algoritmo **FIFO e' semplice da implementare** e necessita' solo di una coda, ma e' il **peggiore a livello di performance.**

3.5.2 Algoritmo Ottimale

Un **algoritmo di previsione** che **sceglie come victim frame quello che verra' usato il piu'tardi possibile.**

Reference string: 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1 >> 9 page fault

Difficult to implement since it requires future knowledge of the reference string.

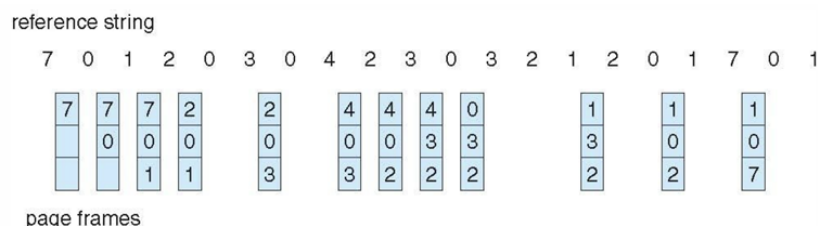
- It is used mainly for comparison study, measuring how well a new algorithm performs.



Questo algoritmo e' il migliore che si puo' avere e viene usato come confronto per gli altri. Non puo' essere implementato in quanto bisognerebbe prevedere il futuro.

3.5.3 Least Recently Used (LRU)

Questo algoritmo **sostituisce la pagina che e' stata usata meno recentemente**. Per fare cio' associa l'istante di tempo in cui ciascuna pagina e' stata usata l'ultima volta.



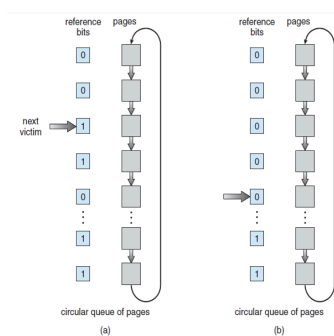
Ci sono due modi per implementarlo:

- **Mediante contatore** ⇒ Alla page table **si aggiunge un'ulteriore entry** nella quale **si memorizza l'ultimo tempo di clock** in cui la pagina i-esima e' stata usata (tempo di ultimo accesso). Quindi ogni volta che devo scegliere un **victim frame** devo scegliere il frame con dentro la pagina con il **tempo minore**.
- **Mediante stack** ⇒ Dove si inseriscono le pagine in uno **stack** e quando una pagina viene usata si mette al top dello stack (**al top ci sara' il piu' recente** e quindi **alla coda dello stack ci sara' il meno recente**).

3.5.4 LRU Approximation

Invece di aggiungere il tempo in cui la pagina e' stata usata si **aggiunge un reference bit**. Inizialmente i bit sono a 0 e quando una **pagina viene riferita** ed e' gia' in memoria si **setta il suo reference bit ad 1**, in questo modo pero' non si sa l'ordine in cui le pagine sono state usate.

3.5.4.1 Second-chance algorithm



Quando si sta **scegliendo il victim frame** si **controlla il reference bit** di ogni pagina:

- **Se a 0** si procede a **sostituirla** e **si scorre la coda** (ultima pagina diventa prima e a seguire). Alla nuova pagina si assegna reference bit a 0.

- **Se ad 1** (usata di recente), **si imposta a 0** e **si passa alla prossima pagina** della coda circolare contenente le varie pagine

(seconda chance). **Inoltre si sposta l'head della FIFO all'elemento successivo!!!**

3.5.4.2 Enhanced Second-Chance

Oltre ad un **reference bit** si aggiunge anche un **modify bit (dirty bit)**, in particolare:

(0, 0) ⇒ Ne usata ne modificata di recente, quindi non devo nemmeno fare page-out (una sola operazione I/O), la migliore da sostituire.

(0, 1) ⇒ Non usata ma modificata di recente, necessario scriverla su disco quando sostituita.

(1, 0) ⇒ Usata di recente ma non modificata.

(1, 1) ⇒ Usata di recente e modificata, sarà usata tra poco ed e' necessario fare page-out per scrivere le modifiche su disco.

Si dà **seconda chance alle pagine con reference bit** (il primo da sx) **ad 1**. Per cercare la pagina migliore potrebbe essere necessario scorrere la coda circolare più volte.

3.5.5 Counting-based algorithms

Least Frequently Used (LFU) ⇒ Si usa un contatore che si aggiorna ogni volta che una pagina viene riferita. Il problema e' quando la pagina viene usata molto all'inizio e poi mai più. Questo algoritmo **sostituisce quella che ha il contatore minore**.

Most frequently Used (MFU) ⇒ Come il precedente ma si sostituisce la pagina con il counter maggiore.

ESERCIZI

7	7	7	1	1	1	3	3	3	7	7	7	7	5	5	5	2	2	2	1
	2	2	2	2	2	2	4	4	4	4	1	1	1	4	4	4	3	3	3
		3	3	3	5	5	5	6	6	6	6	0	0	0	6	6	6	0	0
X	X	X	X		X	X	X	X	X		X	X	X	X	X	X	X	X	X

int A[100][100] - pages of size 200 - For three-page frames, LRU replacement - page frame 1 contains process

```
a. for (int j = 0; j < 100; j++){
    for (int i = 0; i < 100; i++){
        A[i][j] = 0;
    }
}
```

answer = 5,000

```
b. for (int i = 0; i < 100; i++){
    for (int j = 0; j < 100; j++){
        A[i][j] = 0;
    }
}
```

answer = 50

Bisogna capire innanzitutto quante pagine sono allocate per la matrice:

$A[i][j] = A[0][0] - A[0][1] \dots A[0][99] = 100$ elementi, la pagina e' da 200 quindi ancora non la ha occupata tutta $A[1][0] \dots A[1][99]$ occupo la prima pagina quindi la prima pagina va da $A[0][0]$ a $A[1][99]$ -> due righe della matrice per pagina quindi, essendo la matrice 100x100 In totale ho 50 pagine che devo dedicare a questa matrice.

a)
 $A[0][0]$ -> serve pagina 0 -> page fault
 $A[1][0]$ -> pagina 0 -> ho gia' 0, no page fault
 $A[2][0]$ -> pagina 1 -> page fault
 $A[3][0]$ -> pagina 1 -> no page fault

Ogni due elementi devo quindi accedere ad una nuova pagina. Solo il loop interno (100 iterazioni dove ogni 2 mi chiede una pagina) mi richiede accesso a 50 pagine diverse, quindi 50 page fault. Il loop esterno viene eseguito 100 volte, quindi $100 \times 50 = 5000$. Ho 3 frame liberi, 1 sempre occupato, quindi ne ho 2 disponibili.

b)
 Si accede a:
 $A[0][0]$ -> serve pagina 0 (page fault) -> Nel loop interno $A[0][0] \dots A[0][99]$, non mi serve altro, sempre solo pagina 0
 $A[1][0]$ -> sempre pagina 0 quindi niente page fault, nel loop interno arrivo fino a $A[1][99]$, quindi niente page fault
 $A[2][0]$ -> serve pagina 1 (page fault) ma nel loop interno sempre niente page fault.

Quindi ho page fault solo relativamente a loop esterno e, essendo che mi serve una pagina diversa ogni 2 righe, avro' in totale solo $100/2 = 50$ page faults.

Quindi meglio fare il loop come nel secondo caso ooooooooooooli

Si consideri la seguente sequenza di riferimenti in memoria nel caso di un programma di 1000 parole:

261, 409, 985, 311, 584, 746, 632, 323, 470, 915, 858.

page size = 200 parole

quindi pagina 0 va da 0 a 199, la pagina 1 da 200 a 399 e così via.

261 fa quindi riferimento alla pagina 1

409 alla 2, 985 alla 4 e così via come specificato sotto

passi:

- Pagina 1 gia' in memoria. Ha reference bit pari a 0, quindi il suo reference bit si mette ad 1 -> FIFO = 4, 3, 1.
- Pagina 2 non e' in memoria, quindi cerco la pagina da sostituire. 4 e' il primo della fifo, ha reference bit a 0, quindi lo tolgo e metto 2. la fifo si aggiorna: 3(1), 1(1), 2(0) -> l'ultimo messo viene messo come ultimo elemento
- Pagina 4 non in memoria, inizio da 3(1), avendo reference bit ad 1 passo al successivo e metto a 0 reference bit a 0. Lo stesso per 1(1). Poi 2(0) lo sostituisco. FIFO: 3(0), 1(1), 4(0).
- Pagina 1 gia' in memoria, si mette ad 1 il suo reference bit. FIFO: 3(0), 1(1), 4(0).
- Pagina 2 va messa. Sostituisco 3 -> FIFO: 1(1), 4(0), 2(0).
- Pagina 3 va messa. Sostituisco 4(0) e abbasso 1(0). FIFO: 1(0), 2(0), 3(0)
- Per i prossimi 2 ci sono gia, aggiorno solo i reference bit: FIFO 1(1), 2(1), 3(1)
- Per pagina 4 ciclo la fifo la prima volta metto tutti i reference a 0 e quindi al secondo ciclo 1 viene sostituito: FIFO: 2(0), 3(0), 4(0)

fifo ----->	4-3-1	3-1-2	3-1-4	3-1-4	1-4-2	1-2-3	2-3-4				
	1	2	4	1	2	3	3	1	2	4	4
4 ₀	4(0)	2(0)	4(0)	4(0)	4(0)	3(0)	3(1)		3(1)	3(0)	3(1)
3 ₁	3(1)	3(1)	3(0)	3(0)	2(0)	2(0)			2(1)	2(0)	2(1)
1 ₀	1(1)	1(1)	1(0)	1(1)	1(1)	1(0)			1(1)	4(0)	4(1)
		x	x vanno a 0 in quanto 2 era ultimo in FIFO		x	x				x	

3.6 Allocazione dei frame

Bisogna capire, dati due processi, quanti frame allocare a ciascuno dei due processi. Con **pure demand paging ogni**

processo ha 0 frame allocati il primo processo che fa 93 page fault prende tutti i frame e mentre il secondo processo deve per forza fare page fault per poter essere eseguito. Un altro modo per **allocare i processi** e':

- Far si che **ogni processo** abbia un **numero minimo di frame allocati dall'inizio** e se ne vuole di piu' fa page fault. Il numero minimo e' definito dall'architettura. **Questi frame allocati vengono liberati in seguito al rilascio del processo relativo oppure quando un altro processo necessita di un frame e gli algoritmi di replacement scelgono questi frame.**
- Il numero massimo di frame per ogni processo e' pari al numero totale di frame nel sistema.

Ci sono due **schemi di allocazione**:

- **Equal allocation** $\Rightarrow$ A prescindere dalla grandezza dei processi, ad **ogni processo do lo stesso numero minimo di pagine iniziali** allocate. Dati **m** frame e **n** processi, per ogni processo alloco inizialmente **m/n** frame. Inutile se un processo ha dimensione molto piccola.
- **Proportional allocation** $\Rightarrow$ Alloco frame iniziali ad ogni processo **in base alla loro dimensione (memoria logica).**

S_i = size of virtual memory for process p_i

$S = \sum S_i$

m = total number of available frames

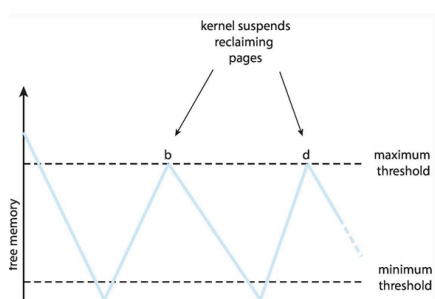
Allocating a_i frames to the process p_i , where a_i is approximately $\gg a_i = S_i / S * m$

Oltre alla loro dimensione, posso valutare il numero di frame da allocare per un processo in base al suo valore di **priorita'**.

3.7 Global vs Local allocation

Global replacement $\Rightarrow$ Un processo sceglie un **victim frame** qualunque, anche se quel frame e' momentaneamente allocato per un altro processo. Questo comporta che il tempo di esecuzione di un processo varia ogni volta, in quanto dipende dagli altri processi in esecuzione.

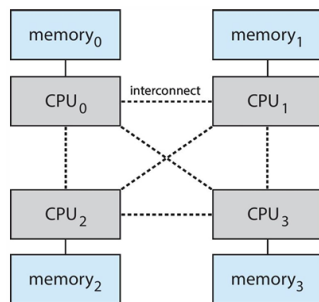
Local replacement $\Rightarrow$ Ogni processo sceglie come victim frame solo uno tra quelli che gli sono stati allocati inizialmente. Questo fa si che il range di performance sia piu' stabile (dipende solo da code di I/O per accesso a disco durante page-in o page-out).



Se si implementa il **global replacement**, invece di aspettare che

la **lista di frame liberi** del S.O. sia vuota per fare page-out, si inizia ad effettuare **page-out una volta superato un certo threshold**. Quando si scende sotto questa soglia parte una routine del sistema operativo (**reapers**) che inizia a liberare i frames. Quando si raggiunge un **massimo threshold** questa routine viene **interrotta**.

3.8 Non-Uniform Memory Access (NUMA)



I sistemi moderni sono multi-CPU. Questo significa che ogni CPU ha una memoria locale dedicata, ma può anche accedere alle memorie delle altre CPU. Lo svantaggio in quest'ultimo caso è che il **tempo di accesso è più lento**. Questi sistemi (**NUMA**) non trattano quindi l'accesso alla memoria in modo uniforme.

3.9 Trashing

Se un processo **non ha abbastanza frame allocati** per la sua esecuzione si ha un **alto numero di page-fault** in quanto dopo aver liberato un frame è necessario riallocarne subito un altro (magari è necessario che siano in memoria in contemporanea). Un processo è in **TRASHING** se il processo spende **più tempo a richiedere pagine invece di essere eseguito**. Questo comporta:

- **Basso tempo di utilizzo di CPU.**
- Il S.O. essendoci basso tempo di utilizzo va ad aumentare il numero di processi in esecuzione (grado di **multiprogrammazione**), che causa un ulteriore peggioramento del trashing e **più processi saranno in questo stato**.

Per **diminuire l'effetto del trashing** si può passare ad un algoritmo di tipo **LOCAL REPLACEMENT** (vedi 3.7). Questo diminuisce il trashing ma **non lo elimina**. In quanto anche se solo un processo è in trashing la **coda di I/O per accesso al disco** è comunque elevata.

Per prevenire il trashing allora ci si sposta verso un sistema chiamato **LOCALITY MODEL**.

3.9.1 Locality Model

Il **Locality Model** si basa sul concetto di **località di riferimento**, ovvero l'insieme di **pagine che vengono utilizzate frequentemente in uno stesso intervallo** di tempo (**locality**).

Ogni processo, durante la sua esecuzione, attraversa **diverse** fasi in cui utilizza specifiche **localities**.

Se un processo dispone di un **numero sufficiente di frame** per ospitare tutte le pagine della sua **locality corrente**, i **page faults** saranno limitati principalmente alla **fase iniziale** della locality.

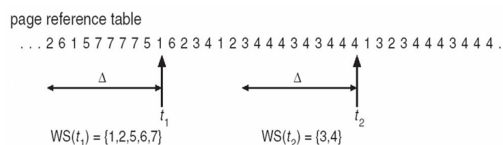
Tuttavia, **se un processo non dispone di abbastanza frame per la locality attuale**, si verificherà un numero elevato di **page faults**, causando **trashing**. Ad esempio, se un processo necessita di 10 frame contemporaneamente ma ne ha solo 8 disponibili, dovrà continuamente sostituire pagine, con un effetto a cascata che riduce drasticamente le prestazioni.

3.9.2 Working-Set Model

Implementazione del **locality model** che introduce il concetto di **working-set window (Δ , delta)**, che rappresenta un **intervallo di riferimento (misurato in numero di riferimenti a memoria) entro cui vengono considerate le pagine attivamente utilizzate** da un processo.

In particolare, una pagina viene considerata appartenente al working-set se riferenziata all'interno della finestra delta. Se quindi una **pagina non viene riferenziata per delta riferimenti consecutivi allora non fa probabilmente piu' parte del locality corrente** e viene **rimossa (page-out)**.

Esempio: se $\Delta = 2$ e il processo accede in sequenza alle pagine **p1, p2, p3**, allora la **working-set window** sarà {p2, p3} e la pagina **p1** verrà espulsa dalla memoria.



Bisogna scegliere la **dimensione della finestra** in maniera corretta in quanto se **delta e' troppo piccolo** non si riesce a racchiudere nella working-set window tutto il locality (**trashing**), mentre se **troppo grande** si prendono pagine che non appartengono al locality (**meno efficienza**).

Con questa politica quindi la **dimensione del resident set** del processo **varia** in base al numero di pagine che ci sono nella working set window attuale.

Quindi **alcune caratteristiche**:

- La **stima della finestra** va ad intaccare solamente le **performance** della policy.
- Il **resident set deve essere aggiornato** anche se non c'e' stato un page fault (difatti una pagina va rimossa se non viene referenziata per D riferimenti consecutivi).
- La tecnica deve tenere traccia solamente del momento in cui c'e' stato l'ultimo accesso per ogni pagina (non la lista degli accessi).

3.9.2.1 Gestione e controllo working set

Per rilevare il passaggio da una locality all'altra e cambiare i frame nel working set, **invece di effettuare il controllo ad ogni riferimento** in memoria si usa un meccanismo basato su **interrupt e reference bit** (come per LRU). Il sistema operativo aziona quindi un meccanismo di **interrupt periodico** dove si scansionano le pagine del working set e:

- Se il **reference bit** e' ad **1** il bit viene azzerato per il prossimo interrupt.
- Se il **reference bit** e' a **0** probabilmente non fa piu' parte della locality attuale e la pagina rimossa.

Posso anche effettuare piu' interrupt ed utilizzare piu' reference bit, fare piu' interrupt richiede pero' piu' tempo di CPU.

Quindi questa metodoologia puo' essere usata con

3.9.3 Page-Fault Frequency

Per ogni processo si controlla ogni quanto frequentemente ho un page-fault e, invece di attivare l'algoritmo ad ogni riferimento alla pagina, si fa ad ogni page-fault. **Ad ogni page fault** controllo la distanza con il page-fault precedente. Se la **distanza e' minore di una soglia "c"** vuol dire che non ho dedicato abbastanza frame al processo e **ne aggiungo uno**. Se **la distanza invece e' maggiore**, la frequenza di page fault e' ok, quindi **rimuovo le pagine allocate** per quel processo che hanno **reference bit a 0**.

Tempo	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
Stringa	6	4	4	3	2	4	4	4	1	3	3	2	4	4	5	1	2	2	2	6	1	2	5	4
Frame 0	6	6	6	6	6	6	6	6	1	1	1	1	1	1	5	5	5	5	5	6	6	6	6	6
Frame 1		4	4(1)	4	4	4(1)	4(1)	4(1)	4(0)	4	4	4	4(1)	4(1)	4(0)	4	4	4	4		1	1	1	1
Frame 2				3	3	3	3	3		3	3(1)	3(1)	3(1)	3(1)	3(0)	3	3	3	3				5	5
Frame 3					2	2	2	2				2	2	2		1	1	1	1					4
Frame 4																	2	2(1)	2(1)	2(0)	2	2(1)	2(1)	2(1)
Fault	x	x		x	x				x	x	x	x			x	x	x			x	x		x	x
Ref Bit									x						x					x				

$C = 3$
 $d = 1$
 $d < C$,
quindi 4 si aggiunge
 $d = 2$
 $d < C$
3 si aggiunge
 $d = 4$
 $d > C$
rimuovo
pagine che
hanno ref bit
a 0
 $d = 3$
rimuovo
pagine
con ref bit
a 0 e metto
a quelle con
ref bit a 1 = 0

d e' la distanza tra il page fault attuale e quello precedente.

Se tutti hanno reference bit ad 1 e una pagina non e' tra i frame -> si mettono tutti i reference bit a 0 e si sceglie il primo da togliere (come second chance)

Page replacement strategies

Page Fault Frequency (empirical probability)

$$f(A, m) = \sum_w p(w) \frac{F(A, m, w)}{\text{len}(w)}$$

• A page replacement algorithm under evaluation

• w a given reference string

• $p(w)$ probability of reference string w

• $\text{len}(w)$ length of reference string w

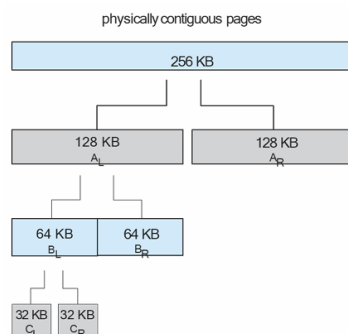
• m number of available page frames

• $F(A, m, w)$ number of page faults generated with the given reference string (w) using algorithm A on a system with m page frames.

3.10 Gestione Kernel Memory

Per adesso abbiamo fatto riferimento all'allocazione di processi utente. Gli **oggetti** che devono essere allocati nella **memoria kernel devono essere spesso allocati in modo contiguo**. Per cercare di diminuire la fragmentation si usano tecniche come:

- **Buddy system** $\Rightarrow$ Si alloca la memoria utilizzando potenze di 2.



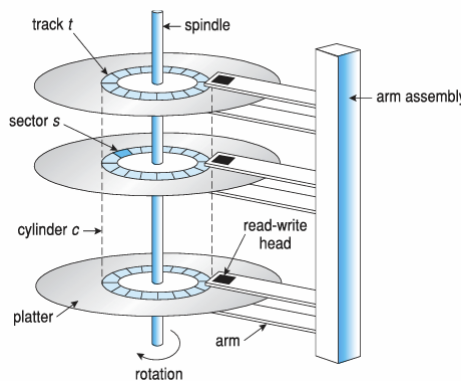
Considerando blocchi di 256Kb e un dato richiede 21kb, il **blocco viene suddiviso in potenze di 2 in modo da ottenere il blocco piu' piccolo che puo' contenere 21kb**. Si ottengono quindi blocchi di memoria di questo tipo e, quando il blocco da 32kb si libera, si puo' effettuare merging.

- **Slab allocation** $\Rightarrow$ La memoria viene **divisa in blocchi chiamati slab raggruppati in caches**. Gli oggetti del kernel hanno **dimensioni specifiche che fanno riferimento a caches che contengono blocchi della medesima dimensione specifica**. Questo permette di diminuire la frammentazione, in quanto **ogni tipo di oggetto** (con una specifica dimensione) ha la sua **cache associata e verra' allocato unicamente in quelle slab**.

4 MASS STORAGE

La **memoria secondaria** viene generalmente implementata mediante **hard disk drives (HDDs)**, **memoria non volatile (NVM)** o anche **DRAM**. Quest'ultimo caso principalmente in sistemi embedded se e' richiesta alta velocita' di accesso

4.1 HDDs



I dischi (**platter**) sono caratterizzati da **due superfici magnetiche**. Ogni disco e' caratterizzato da una **testina** e da una serie di **sezioni circolari** chiamate **track**. Ogni track viene **divisa a sua volta in settori**. L'**insieme delle tracce** che hanno la medesima distanza dal centro tra i vari dischi prendono il nome di **cilindro**. Quindi **ci sono tanti**

cilindri quante le tracce, il cilindro indica semplicemente l'insieme delle tracce tra i vari platter.

- **Positioning time** $\Rightarrow$ Calcolato tramite:
 - **Rotational Latency** $\Rightarrow$ Tempo che il disco ci mette a girare in modo da allineare la testina al settore corretto. **Latenza media** calcolata come $\frac{1}{2} * \text{latency}$
 - **Seek time** $\Rightarrow$ Tempo impiegato dalla testina per raggiungere il cilindro desiderato (scegliere la track corretta).
- **Transfer rate** $\Rightarrow$ Tempo impiegato dalle informazioni per passare **da disco a computer**.

Quindi:

- **Average latency = Average access time**, ovvero il tempo necessario per raggiungere la posizione nella quale vi e' il dato cercato si calcola come:

average seek time (cilindro o track) + average latency
(settore)

- **Average I/O time:**

average access time + (amount / transfer rate) + controller
overhead.

Quindi:

Lat. Rot. media + ricerca media + Tempo di trasferimento + overhead

Quindi se mi chiede Average I/O time per 2 blocchi, moltiplico
x 2 il tempo di trasferimento.

Il **controller** si occupa di **gestire la comunicazione tra device
I/O e sistema operativo.**

For example, to transfer a **4KB block** on a **7200 RPM** disk with a **5ms** average
seek time, 1Gb/sec transfer rate with a .1ms controller overhead =

- 5ms + 4.17ms + 0.1ms + transfer time

- Time per rotation = 60 seconds / 7200 RP = 8.333 ms

- Transfer time = (1Gb/sec) / 0.032768 = 0.031 ms

- 4KB = 4*8*1024 = 32768bit = 0.032768Gb

- Transfer time = 4KB / 1Gb/s * 8Gb / GB * 1GB / 1024*KB = 32 / (1024²) = 0.031 ms

- Average I/O time for 4KB block = 9.27ms + .031ms = 9.301ms

seek time + rotational time + transfer time + controller =

5ms + ((60s / 7200) / 2) + 0.031ms = 9.27ms + 0.031ms = 9.301ms

Quindi il controller
overhead e' di 0.1ms (e'
scritta male la slide)

4.2 NVM

Invece di dispositivi meccanici si hanno sistemi (**Solid-state
Disk**) caratterizzati dalla presenza di **transistor**. Questi
dispositivi sono solitamente **piu' affidabili** in quanto non
hanno parti in movimento, questo fa si che **non ci sia seek
time o rotational latency**. I contro sono gli effetti
elettronici che impattano la vita dei transistor, come **aging o
effetto ambientale** (radiazione).

Le **informazioni** vengono scritte in **blocchi** di dati che
prendono il nome di **pagine**, quindi, non potendo modificare il
singolo bit **se devo modificare una singola informazione devo
riscrivere l'intera pagina**. Qui sorge un ulteriore problema,
ovvero vi e' un **limite nel numero di scritture** che possono
essere effettuate sulle **stesse pagine**. Il life span viene
misurato in **Drive Writes Per Day (DWPD)**.

Per **distribuire equamente le varie pagine** del SSD utilizzo una
flash translation layer (FTL) per tenere traccia delle valid e
invalid page. Le pagine invalid vengono poi liberate
periodicamente mediante una routine chiamata garbage collector
e le informazioni scritte copiate in altre locazioni.

La durata di un disco SSD è quindi limitata dal numero massimo di cancellazioni. Le cancellazioni sono effettuate per blocchi/pagine e hanno impatto anche sul numero massimo di scritture, in quanto una scrittura (ri-scrittura) deve essere preceduta da cancellazione.

4.4 HDD Scheduling

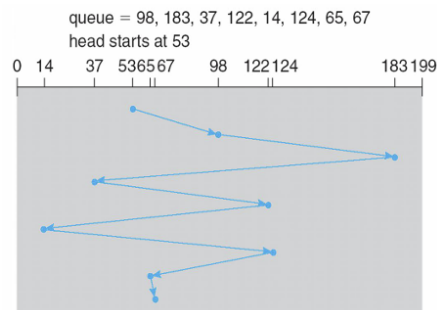
L'obiettivo è quello di **ridurre il più possibile il tempo di accesso** e massimizzare la larghezza di banda. Il **controller I/O** riceve varie **richieste di I/O**, ognuna che punta ad un **settore diverso** magari, dal S.O le quali vengono memorizzate in un **buffer di I/O**. Il controller usa quindi un **algoritmo** in modo da ottimizzare le richieste nel buffer **scegliendo i settori da raggiungere nel giusto ordine** ad esempio.

number of cylinder: 98, 183, 37, 122, 14, 124, 65, 67

Head pointer 53

Ad esempio, data la testina su cilindro (traccia 53), che ordine scelgo?

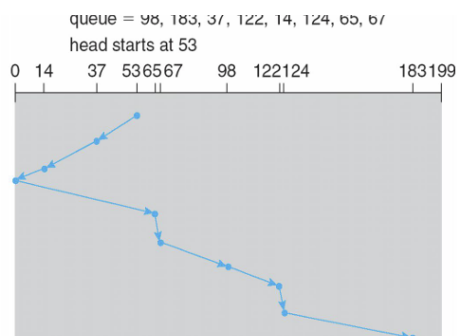
- **First Come First Served (FIFO)**, ovvero **seguo l'ordine delle tracce così come sono nel buffer**.



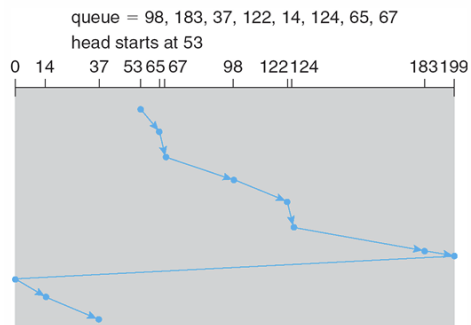
- **SCAN (Elevator algorithm)**

⇒ Parto dalla posizione iniziale della testina, in base alla direzione in cui mi sto muovendo soddisfo prima le richieste lungo quella direzione, per poi tornare

indietro e soddisfare quelle all'altro estremo. In questo esempio avevo letto la traccia 57, ora la testina è in 53 quindi mi sto muovendo verso il "basso". Quindi prima soddisfo tutte quelle fino a giungere a 0, per poi cambiare direzione e soddisfare le altre.



- **CSCAN** ⇒ Invece di tornare alla posizione 0 subito, vado prima a **soddisfare le richieste piu' lontane**, per poi **tornare alla posizione iniziale** e infine rispondo alle richieste mancanti tra 0 e la head start.



In sistemi come **LINUX** invece di avere una sola coda (buffer) si hanno **varie code separate** anche per letture o scritture e vengono implementati algoritmi diversi per le varie code in base a necessita'.

4.5 NVM Scheduling

Di solito non si implementano algoritmi particolari, si segue una tecnica basata semplicemente sull'**ordine in cui le richieste arrivano (NOOP)**. Questo perche' gli SSD non sono penalizzati dalla posizione del blocco dati in quanto l'accesso ha il medesimo ritardo a prescindere dalla posizione.

E' possibile inoltre che un disco rigido e un SSD siano organizzati in modo da formare un volume unico in quanto la tecnologia del disco viene vista al livello basso dei driver dei dispositivi.

4.6 Storage Management and Error Detection

Nella **gestione dei file** e' necessario implementare algoritmi di **Error Detection** come parity bit oppure strategie di **Error-correction code (ECC)** che permettono di correggere errori semplici e notare errori piu' complessi. Il miglior modo per **gestire gli errori** e' mediante la **ridondanza**, mediante ad esempio **sistemi TMR**, che **triplicano l'informazione** e verificano la **presenza di errori mediante majority voting**. Il sistema operativo effettua operazione di **low-level formatting** quando un nuovo disco viene inserito:

- **Partizionamento**, dove il disco viene suddiviso in uno o piu' **gruppi di pagine**.
- **Formattazione logica**, in modo da dedicare una parte della memoria all' **bootstrap loader** e individuare la partizione di boot.

4.7 RAID Structure

Invece di avere sistemi con singolo disco, si hanno **gruppi di dischi che lavorano insieme**, questo permette di:

- Migliorare l'**affidabilita'** del sistema (**ridondanza**).
- Effettuare **stripping di data**.

Poiché ogni singolo disco ha un proprio **Mean Time To Failure (MTTF)**, cioè la **vita operativa media prima del guasto**, è necessario introdurre **meccanismi di protezione dei dati** mediante ridondanza.

L'approccio piu' semplice per fare cio' e' il **Mirroring**, dove l'informazione del disco 1 viene copiata nel disco 2 (mirrored volume). Si introduce poi il **Mean Time To Repair (MTTR)**, ovvero il **tempo medio che impiego a sostituire un disco e far ripartire il normale funzionamento del sistema**.

I RAID, avendo alto MTTF e basso MTTR, tendono quindi a massimizzare il **Mean Time To Data Loss (MTTDL)** (**tempo medio per un dato per essere perso dopo un failure**), ad esempio con **RAID di 2 dischi**:

Mirrored RAID: 2 disks

$$\text{MTTF}_{(1\text{st_fail})} = \text{MTTF}/2 = 100,000/2 = 50,000 \text{ hours}$$

$$\text{Prob}_{(2\text{nd_fail_during_repair})} = \text{MTTR}/\text{MTTF} = 10/100,000 = 10^{-4}$$

$$\text{Mean time to data loss} = \text{MTTF}_{(1\text{st_fail}+2\text{nd_fail_during_repair})}$$

$$\text{MTTF}_{(\text{fail}+\text{fail_during_repair})} = \text{MTTF}_{(1\text{st_fail})} / \text{Prob}_{(2\text{nd_fail_during_repair})}$$

$$= \text{MTTF}^2 / (2 * \text{MTTR}) = 10^{10} / (2 * 10)$$

$$= 5 * 10^8 \text{ hours}$$

Se $\text{MTTF} = 10^5$, la probabilita' che un disco fallisca e' $1/10^5$.

$\text{MTTR} = 10 \text{ hours}$.

Allora $\text{MTTDL} = 1 / \text{probabilita' di avere data loss (pdl)}$.

Allora $\text{pdl} = (P(f1) + P(f2)) * (P(f1 \text{ or } f2) * \text{MTTR}) = (2 * 10^{-5}) * (10^{-5} * 10) = 2 * 10^{-9}$.

Allora $\text{MTTDL} = \frac{1}{2} * 10^{-9} = \frac{1}{2} * \text{pdl}$

Dove $P(f1)$ = probabilita' che fallisca disco 1.

NECESSARIO SAPER ESTENDERE LA FORMULA A PIU' RAID.

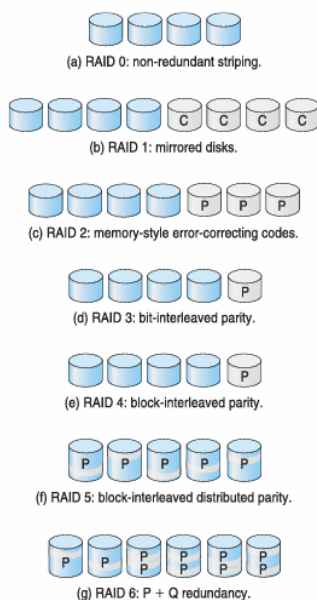
$$\text{MTTDL}_{\text{RAID5}} = (\text{MTTF}_{\text{DISCO}})^2 / (N \times (N-1) \times \text{MTTR})$$

4.7.1 Stripping data

Usando gruppi di dischi mirrored ho una maggior ridondanza e affidabilità nel sistema. Questo però significa anche che posso effettuare **operazioni di lettura in parallelo**, oppure migliorare il **transfer rate** mediante scritture in parallelo:

- **Bit level stripping**, Divido i bit di ciascun byte sui vari dischi (es. scrivo 1 bit su ogni disco se ho 8 dischi).
- **Block-level stripping**, Dati **N dischi**, il **blocco I** del file va nel blocco **$(I \bmod N) + 1$** .

4.7.2 RAID Levels



Ci sono diversi **livelli di RAID** che **implementano varie funzioni**:

- **RAID 0**, alta performance ma bassa affidabilità in quanto non ho ridondanza e visto che ogni disco dipende dal successivo, se ho un guasto perdo tutto.
- **RAID 1**, Meta' dei dischi vengono usati come mirrored.
- **RAID 2**, Una parte della memoria viene dedicata esclusivamente per l'error detection e correction.
- **RAID 5**, algoritmo di ECC suddiviso nei vari dischi.

Consider a Hard Disk Drive (HDD) with standard technology and geometry. The disk has

- sector size of 512 Bytes,
- 2480 tracks per surface, **2048 tracks**
- 50 sectors per track,
- 5 double-sided platters,
- average seek time of 10 msec.

1. What is the capacity of a track in bytes? What is the capacity of each **platter/2** surface? What is the capacity of the disk? How many cylinders does the disk have?
2. The CHR (Cylinder-Head-Sector) disk addressing method allows a disk block to span over multiple sectors of a given track (identified by a (cylinder,head) pair. Are the following valid block sizes: 256 B, 2048 B, 51200 B?
3. If the disk platters rotate at 5400 rpm (revolutions per minute), what is the maximum rotational delay? What is the average rotational delay? If one track of data can be transferred per revolution, what is the transfer rate

1. Ogni traccia = $512 \text{ B} \cdot 50$ (in quanto ogni traccia ha 50 settori da 512 bytes) = 25KB.
Capacità superficie = Ogni superficie ha 2048 tracks, allora $2048 \cdot 25 \text{ KB} = 50 \text{ MB}$.
Capacità del disco = $50 \text{ MB} \cdot 2$ per platter, avendo 5 platter = $100 \text{ MB} \cdot 5 = 500 \text{ MB}$.
Il numero di cilindri è pari al numero di tracks.

2. La dimensione dei blocchi che posso leggere deve essere multiplo del settore, il quale è la dimensione MINIMA a cui posso accedere. Inoltre il massimo corrisponde alla dimensione della traccia.

3. $60 \text{ s} / 5400 = 11 \text{ ms}$ e tempo medio = 5.5ms
transfer rate per un track di data -> un track di data sono 25kb -> $25 \text{ kb} / \text{revolution} = 25 \text{ KB} / (60 / 5400) = 90 \cdot 25 \text{ KB} / \text{s} = 2.25 \cdot 8 \text{ Mbit} / \text{s}$. Una traccia è circolare, quindi serve l'intero giro di rivoluzione, c'è scritto nella consegna della traccia.

5 I/O SYSTEMS

5.1 Overview

Per la comunicazione tra un dispositivo I/O e Sistema Operativo e' fondamentale la presenza di **moduli device driver** che permettono al kernel di parlare con diversi dispositivi. Il **kernel genera dei comandi** che vengono **tradotti in istruzioni specifiche** dal **device driver** per la comunicazione con i device I/O. I dispositivi I/O comunicano con il **device driver mediante un Controller I/O** (o adapter) tramite **bus**. Per inviare comandi al controller I/O la **CPU** scrive in vari **registri del controller**:

- **Data-in register**, usato dalla **CPU** per **leggere** i dati provenienti dal **device I/O**.
- **Data-out register**, **CPU scrive** i dati che poi il controller invia al device.
- **Status register**, contiene bit di stato per la condizione del device. Ci sono i bit:
 - ready bit.
 - busy bit.
 - error bit.
 - bit di completamento.
- **Control register**, per modificare la modalita' di funzionamento del dispositivo.

Questi **registri** del controllore vengono **mappati** nella memoria del processore mediante **Memory-mapped I/O**. Ovvero fanno direttamente parte dello spazio di indirizzamento del processore. Questo permette alla CPU di accedervi come fossero **normali locazioni di memoria**.

5.2 Polling e Interrupts

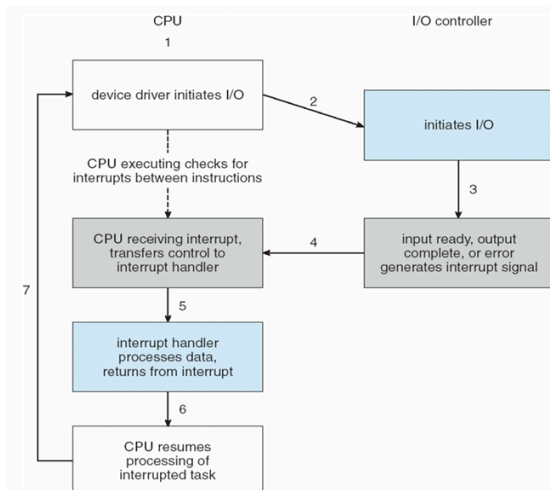
Quando la CPU vuole **comunicare con un device I/O** lo fa scrivendo sui registri data-out e control del controller I/O e controllando lo status register:

- **CPU controlla il busy bit** del controller I/O. Se esso e' a 0 allora puo' scrivere su **data-out** register.
- **CPU setta command-ready = 1** (del Control Register) e il controller imposta busy bit ad 1.
- Controller **legge data-out** ed effettua l'operazione I/O.

- Controller azzerà il command-ready bit e l'error bit nello status register e imposta busy = 0 (**operazione I/O completata**).

Seguendo un approccio **Polling**, la **CPU rimane in attesa** controllando il busy bit in maniera periodica, sprecando cicli macchina.

Un approccio migliore si basa su **Interrupt**, invece di far attendere la CPU, il **controller I/O genera un interrupt al completamento dell'operazione I/O**. L'interrupt è un



meccanismo hardware in quanto vi è un cavo dedicato (**interrupt-request line**). Quando la CPU riceve un interrupt sospende l'esecuzione corrente (**context switching**) e identifica l'interrupt ricevuto guardando la **Interrupt Vector Table (IVT)**. Esegue l'**interrupt handler** corrispondente e poi riprende l'esecuzione normale.

La differenza tra **eccezione**,

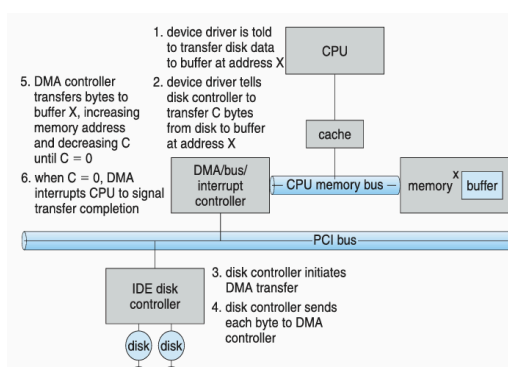
trap e interrupt è che:

- **Interrupt** ⇒ segnale hardware mandato da un device esterno per soddisfare una richiesta I/O.
- **Eccezione** ⇒ evento interno al processore quando si hanno errori come accesso a parti di memoria non autorizzati.
- **Trap** ⇒ Interrupt generati via software, generati a seguito di una chiamata a sistema.

Dal sistema operativo vengono gestiti ugual modo ma cambia appunto l'origine di tali eventi.

5.3 Direct Memory Access (DMA)

Per evitare di utilizzare la CPU per effettuare trasferimento di dati tra I/O device e RAM, utilizzando quindi una tecnica di **I/O programmato dove la CPU controlla i registri di stato e si deve bloccare per soddisfare la richiesta**, si possono utilizzare **DMA CONTROLLER I/O**.



La CPU comunica mediante system calls al device driver l'inizio dell'operazione I/O. Il **device driver inoltra indirizzo sorgente**

(buffer I/O), indirizzo destinazione (RAM) e dimensione dati al controller I/O. Questi dati vengono inoltrati al controller DMA che gestisce il trasferimento dei dati su memoria, mentre la CPU puo' continuare l'esecuzione. Il DMA avvisa mediante **interrupt** la CPU al completamento dell'operazione. Nel trasferimento dei dati alla memoria da parte del DMA la CPU non puo' ovviamente utilizzare il BUS, si adotta quindi la strategia **cycle stealing**:

- Alla CPU si lasciano dei **cicli disponibili**, mentre negli altri il DMA accede al bus di sistema per il trasferimento dei dati. In questo modo la CPU puo' operare anche se piu' lentamente. Essa risulta essere **poco colpita da cio'** se deve effettuare calcoli che riguardano **operandi nei registri** o in cache, mentre se sta eseguendo un processo **memory-bounded** ne e' **piu' colpita**.

5.4 Buffering (Kernel buffer)

Un buffer e' un area di memoria che viene utilizzata per memorizzare informazioni che vengono scambiate tra due dispositivi. Le ragioni per cui si utilizzano i buffer sono molteplici una delle quali riguarda:

- La **differenza di velocita' tra produttore e consumatore**. Se un produttore e' troppo lento nella scrittura di dati (o ad esempio **trasmette un byte alla volta e il consumatore ne necessita di 8 insieme**), utilizzare un buffer permette al consumatore di leggere i dati raggiunta una certa condizione del buffer (ad esempio buffer pieno). Si introduce inoltre un sistema **double buffering**, che garantisce parallelismo, in quanto il consumatore non effettua l'operazione istantaneamente. Per questo, nel mentre che il consumatore consuma il primo buffer, il produttore puo' iniziare a scrivere sul secondo. Al termine di cio', il consumatore avra' presumibilmente terminato di lavorare sul primo buffer, e quindi il produttore riprendera' a scrivere su quest'ultimo.

Un buffer kernel puo' essere utilizzato per il passaggio dei blocchi di file in transito tra disco e memoria utente. In particolare puo' essere utile per:

- **Disaccoppiare accesso a disco e memoria utente** ⇒ Quando un processo esegue una system call "read(fd, addr, size)" si vogliono spostare "size" byte da disco alla locazione di

memoria RAM indicata dall'indirizzo addr. Senza un buffer kernel bisognerebbe bloccare la pagina utente in RAM fino a quando l'operazione non e' completata. In sistema basati su paginazione porta vari svantaggi:

- Fino a quando l'I/O non e' completato la pagina utente relativa all'addr deve rimanere in RAM.

Con un buffer kernel si hanno vari vantaggi come:

- Quando arriva la richiesta `read(fd, addr, size)` il kernel esegue immediatamente un'operazione di I/O dal disco verso il buffer kernel.
- Durante l'attesa del completamento dell'I/O le pagine utente relative a "addr" possono essere soggette a swap-out. Verranno poi reinserite a seguito di un page-fault al completamento dell'I/O.
- **Pipelining mediante doppio buffer** $\Rightarrow$ Buffer A è in uso per ricevere dati dal disco (I/O Dal Disco $\rightarrow$ Buffer A). Contemporaneamente, Buffer B, già pieno di una porzione precedente di dati, viene copiato in memoria utente (Buffer B $\rightarrow$ Memoria Utente). Una volta che A è pieno, si inizia a copiare A verso utente, mentre B viene marcato libero e può essere riutilizzato per un'altra lettura disco.

Dato un sistema con doppio buffer kernel e supponendi di utilizzare DMA per le letture da disco, si ha che il DMA viene utilizzato per il trasferimeneto dei dati da disco al primo buffer kernel (invece che venire fatto dalla CPU).

5.5 I/O sincrono e bloccante

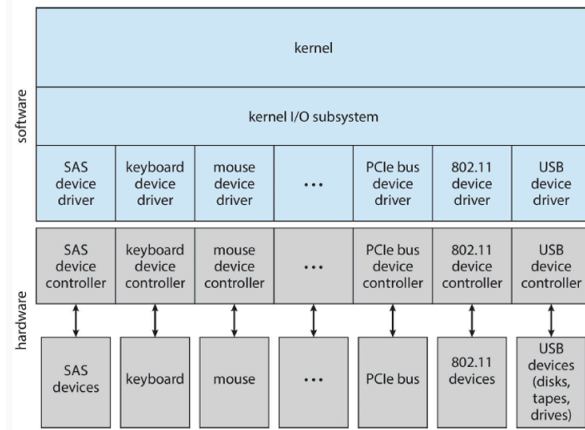
Un dispositivo di input/output puo' trasferire dati in due modalita':

- Sincrona $\Rightarrow$ Il trasferimento dei dati risulta essere temporizzato con altri eventi del sistema e ha tempi di risposta quindi prevedibili.
- Asincrona $\Rightarrow$ Il trasferimento dei dati risulta avere tempi di risposta non predicibili, quindi il trasferimento verra' completato in futuro.
- Bloccante $\Rightarrow$ Il processo che ha acceduto alla risorsa di I/O deve attendere il completamento dell'operazione di I/O.
- Non bloccante $\Rightarrow$ La chiamata di I/O ritorna senza attenderne il completamento.

La differenza tra asincrono e non bloccante e' che una `read()` non bloccante ad esempio ritorna subito anche se i dati non

sono completamente disponibili, mentre una `read()` asincrona fa un trasferimento completo ma che finira' in futuro.

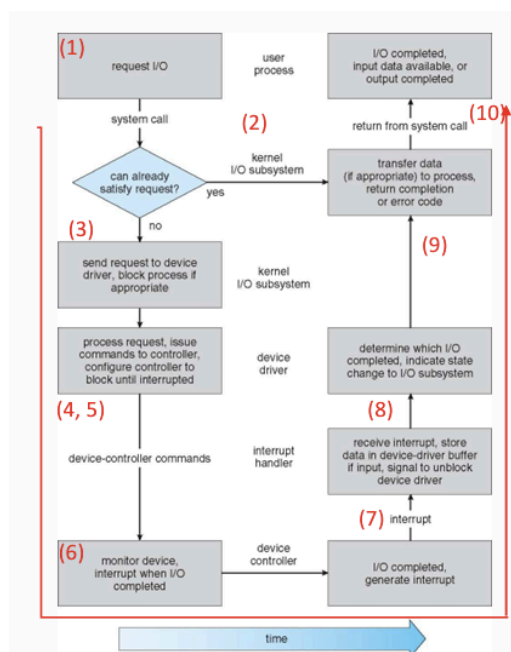
5.6 Kernel I/O Structure



Ogni **controller I/O** ha quindi un **device driver associato**, che comunica con il kernel mediante chiamate a sistema (**Kernel I/O subsystem**). I **device controller** sono quindi caratterizzati da **code I/O** che usano diverse strategie per gestire le richieste (vedi 4.4). Inoltre vi e' un ulteriore ottimizzazione a livello di Kernel I/O

subsystem.

Processo di I/O:



- Il processo chiede i dati (`read()`):

Il programma fa una chiamata di sistema (`read()`), bloccandosi in attesa dei dati.

- Il kernel controlla se i dati sono già in memoria:

Se i dati richiesti sono già nella **cache** (buffer del kernel), li restituisce subito e il processo riprende.

- Se i dati non sono in cache: Serve accedere fisicamente al dispositivo (es. disco). Il processo viene **messo in attesa** (non usa più la CPU).

- Il driver prepara il

trasferimento:

Il **device driver** (software che parla col dispositivo) riserva memoria nel kernel e invia il comando al **controller** del dispositivo.

- **Il dispositivo esegue il trasferimento:**
Il **controller** del dispositivo legge o scrive i dati fisicamente sull'hardware.
- **Il trasferimento avviene tramite DMA:**
Il **DMA (Direct Memory Access)** sposta i dati direttamente nella memoria del kernel, senza usare la CPU. Alla fine, invia un **interrupt** per dire "ho finito".
- **Il sistema riceve l'interrupt:**
L'**interrupt handler** del kernel intercetta il segnale, salva info utili e notifica il driver.
- **Il driver completa la richiesta:**
Capisce quale operazione è finita, controlla se è andata bene, e lo segnala al kernel.
- **Il kernel prepara il ritorno al processo:**
I dati letti vengono copiati nello spazio del processo. Il processo viene **spostato nella coda dei pronti**.
- **Il processo riprende l'esecuzione:**
Il **process scheduler** gli assegna la CPU, e il processo riprende da dove si era fermato nella chiamata `read()`.

capire il concetto es. di DMA, trasferimento, system call ecc...

6 FILE SYSTEM INTERFACE

Il **file system** consiste in:

- **Insieme di file.**
- Struttura di **directory che organizza** i file.

Il sistema operativo vede un **file** come una **uniform logical view** di un'informazione memorizzata.

Un file nel file system può essere organizzato in tre modi:

- **Sequenza di byte** ⇒ File visto come "striscia" continua di byte.
- **Struttura a record** ⇒ Il file è suddiviso in blocchi omogenei (record) di dimensione fissa o variabile, ciascuno contenente un insieme di campi. Un file viene quindi visto come sequenza di record.
- **File con indice** ⇒ Si parte dalla struttura a record, e si aggiungono indici per accelerare l'accesso. Questi indici mantengono puntatori ai record in funzione di uno o più campi chiave.

Tutte queste organizzazioni **permettono sia accesso sequenziale che diretto**:

- Se si sa a che byte accedere e' possibile, mediante offset, riferirsi direttamente a quello.
- Si possono scorrere tutti i record del file o accedere mediante indice direttamente al blocco di interesse.
- Posso scorrere record per record o utilizzare l'indice (es. B+Tree) per accedere direttamente al record di interesse.

6.1 Files

Gli **attributi** dei file sono:

- Nome, identificatore, tipo, location (puntatore), size, protezione (W, R, X...), time, data e utente.
- Gli **attributi** prendono il nome di **metadata** e devono essere **memorizzati insieme al file** stesso in memoria fisica.

Le **operazioni** che posso effettuare su un file sono:

- creazione, scrittura, lettura, file seek, cancellazione; implementate mediante system calls.

All'apertura di un file ci sono **varie informazioni associate**:

- **Open-file table** ⇒ Unica tabella condivisa tra i vari processi **contenente i file aperti**. Ogni file e' indetificato da un indice in questa tabella.
- **File-open count** ⇒ Contatore del **numero di volte che il file e' stato aperto** (decrementa alla chiusura). Questo permette di capire se e' **possibile eliminare il file**.
- **Location del file**.
- **Diritti di accesso**.

6.1.1 File locking

Si associa un lock ad un file per indicare se esso puo' essere condiviso tra piu' processi:

- **Shared lock**, piu' processi possono acquisire il lock in **contemporanea**.
- **Exclusive lock**, **un solo processo alla volta** puo' acquisire il lock al file.

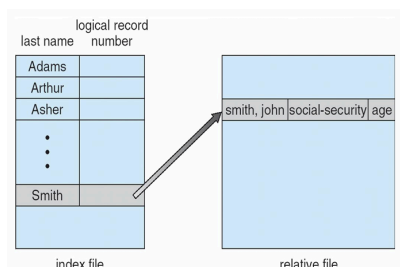
Il lock puo' essere:

- **Mandatory** ⇒ Una volta acquisito un lock esclusivo e' il S.O. che impedisce l'accesso al file.
- **Advisory** ⇒ E' il developer che deve gestire il rilascio o acquisizione del lock.

6.1.2 Accesso a file

L'accesso al contenuto di un file può avvenire in vari modi:

- **Accesso sequenziale** ⇒ Informazione in file **processata in ordine (blocco dopo blocco)**. Le operazioni sono quindi `read_next()` e `write_next()` dove il puntatore viene incrementato in automatico.
- **Accesso diretto** ⇒ File suddiviso in **record logici** di lunghezza fissa che possono essere **letti o scritti in qualsiasi ordine**. Quindi si **specifica direttamente il record che si vuole elaborare**. Le operazioni sono quindi `read(n)` e `write(n)` dove `n` è il **relative block number**.
- **Approccio misto** ⇒ Basato su indici che puntano ad un determinato blocco del file. Gli indici possono essere acceduti in sequenza.



Questi accessi risultano essere importanti nell'**implementazione del meccanismo di allocazione dei file**. Difatti un approccio di allocazione linkata ad esempio non è compatibile con un accesso diretto.

6.2 Struttura Directory

Una **directory** è una struttura dati che **memorizza informazioni sui file che contiene**.

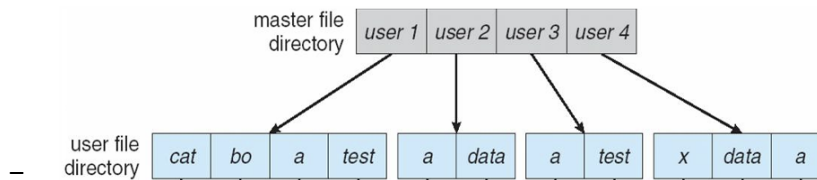
Un disco può essere quindi suddiviso in **partizioni**, che possono essere formattate con un file system. Una **partizione caratterizzata da un file system prende il nome di Volume**. Ciascun volume contiene le informazioni del file system in una **Volume Table of Contents**.

Ci sono diversi tipi di **strutture di directory**:

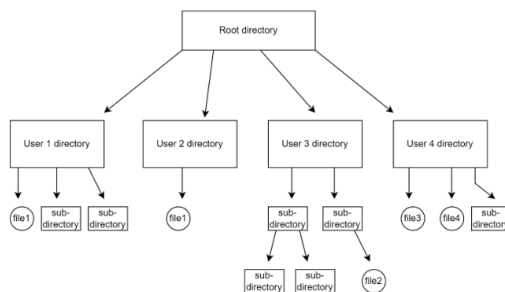
- **Single level directory** ⇒ Tutti i **file contenuti in stessa directory**. Questo comporta che tutti i file abbiano un nome unico e che tutti gli utenti possono accedere agli stessi. Metodo semplice ma non efficiente.



- **Two level directory** ⇒ Per **ogni utente si ha una struttura single level directory**. Facilita file protection.



- **Tree-structure directories** ⇒ Permette all'utente di creare le proprie **subdirectories**. L'albero ha una **root directory** e ogni file ha il suo unico path.
- **Acyclic-Graph directories** ⇒ Permette di **condividere subdirectories o file tra utenti diversi**. Si può fare mediante **creazione di un link**, in questo modo si ha un'unica versione di un file. Se un utente lo modifica l'altro vede il cambiamento. Se si effettua una copia del file si perde questa condivisione.



- **General Graph Directory** ⇒ Per **evitare cicli** e' permesso il **link solamente a file** e non directory. Il **garbage collector** si occupa di trovare loop nel sistema.

6.3 Protezione dei file

Mediante un **Access-Control List (ACL)** per ogni file si specificava per ogni utente la lista degli accessi che esso aveva al file stesso. Questo non risulta essere efficiente. Ora si suddividono gli **utenti in tre gruppi diversi**:

- **Owner**, chi crea il file.
- **Group**, gruppi di utenti che condividono il file e necessitano di accesso simile.
- **Other**.

Ci sono quindi 3 tipi di accesso : read, write, execute e:

a) owner access	7	⇒	111 RWX
b) group access	6	⇒	110 RWX
c) public access	1	⇒	001

7 FILE SYSTEM IMPLEMENTATION

Un **file system** si occupa di gestire file, allocare la memoria ecc... . Esso e' caratterizzato da **diversi livelli**:

- **I/O Kernel subsystem** ⇒ Caratterizzato da **device driver** che fungono da **traduttori** per la comunicazione con i device I/O.
- **Basic file system** ⇒ Responsabile di **creare comandi che vengono inviati al device driver** (che poi traduce per il controller I/O).
- **File-organization module** ⇒ Contiene **informazioni sui singoli file** e sui blocchi di memoria in cui sono memorizzati.
- **Logical file system** ⇒ Usato per **gestire metadata**. Quando si crea un nuovo file questo layer si occupa di allocare un file control block per la memorizzazione dei metadata.

In memoria, il **file system** si basa su **diverse strutture**:

- **Mount table** ⇒ contiene i vari "mount" dei vari file system, i rispettivi mount points e tipologie.
- **System-wide open-file table** ⇒ Tabella condivisa tra tutti i processi del sistema che contiene il **FCB di ogni file aperto**. In questa tabella inoltre si memorizza il **file-open count** (vedi 6.1), in modo da sapere quanti processi hanno aperto quel file.
- **Per-process open-file table** ⇒ Contiene **puntatori alla ipttabella system-wide** corrispondenti alle entry dei file aperti per ogni singolo processo. Oltre che alla **modalita' di accesso** garantita (R o W).

E' necessaria la presenza della **system-wide** e della **per-process** in quanto posso avere piu' processi o thread che aprono lo stesso file in contemporanea in modalita' **differenti**.

Nella **gestione dell'I/O** ci si basa su **tre sezioni**:

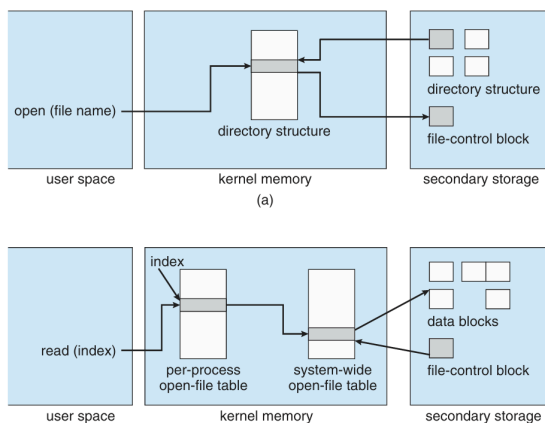
- **User space** ⇒ Processi utente.
- **Kernel memory** ⇒ Sezione di RAM utilizzata per contenere strutture come la tabella dei file aperti di sistema e per processo.
- **Memoria secondaria**.

Ora esempi di processi utente che richiamano funzioni come `open()`, `read()`, `close()` e `write()`:

`Open("path/filename", flags)`

Una volta creato un file per essere usato per effettuare operazioni di I/O deve essere aperto. Il processo utente passa come parametro alla chiamata di sistema il nome del file. A questo punto la `open()`:

- Controlla se il file e' presente nella **system-wide table**. Se si, aggiunge una entry alla **per-process open-file table** del processo che ha chiamato la syscall con un **puntatore alla system-wide**. Altrimenti, si cerca il file nella directory in memoria secondaria utilizzando una **struttura basata sui direttori***, si copia il **FCB** nella **system-wide table** e si aggiunge una entry alla per-process table. La `open()` a questo punto ritorna l'**indice della entry della per-process table** che e' stata aggiunta (**FILE DESCRIPTOR**).



Close(int fd)

Quando un processo utente vuole chiudere un file chiama la `close(fd)`. Essa prende in input il file descriptor, quindi si accede all'**indice della per-process file table** relativo al file e si **elimina la entry**. Successivamente **si decrementa il contatore dei riferimenti corrispondente nella tabella system-wide**, se questo diventa 0, si cancella l'entry anche da questa.

Read() e Write()

La `read/write` usano il file descriptor per accedere alla per-process table, da li si riferiscono alla system-wide table da dove ottengono il **FCB** che contiene ad esempio la **lista dei blocchi dato**, che utilizzano per accedere su disco.

★

La struttura di directory puo' essere implementata mediante:

- Linear list.
- Hash table.

Posso avere appunto piu' file system (uno per ogni volume del disco), i quali sono strutture software. Un sistema basato su **layer fa si che alcuni di essi possono essere condivisi tra diversi file systems.**

Il **File Control Block (FCB)** contiene metadati del file associato come permessi, ownership e locazione in memoria. Il **FCB** viene memorizzato nella struttura a directory su disco, nello specifico:

- Nella entry della directory dove c'e' anche il puntatore al primo blocco della FAT, nel caso di S.O che implementano questo meccanismo.
- Nella inode table nel caso di sistemi basati su inode.

7.1 Metodi di allocazione

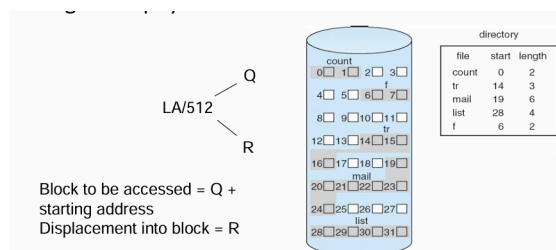
Disco rigido suddiviso in blocchi. Se ho dimensione frame maggiore dei blocchi, allora un frame viene memorizzato su piu' blocchi. Altrimenti in alcune architetture posso mettere piu' frame nello stesso blocco, oppure massimo un frame (aumento frammentazione interna).

Modo in cui i **blocchi (unita' minima in cui possiamo leggere o scrivere su disco)** di memoria sono allocati per i file:

- **Allocazione contigua.**
- **Allocazione linkata** $\Rightarrow$ Ogni blocco del file contiene un puntatore al successivo. Questo non e' adatto per Accesso Diretto (vedi 6.1) in quanto dovrei seguire tutti gli (n-1) puntatori per arrivare ad un certo blocco n.
- **Allocazione indexed.**

7.1.1 Allocazione contigua

Ogni file occupa **insieme di blocchi contigui**. Devo trovare un chunk di blocchi che devono essere uno dopo l'altro.



In questo modo **non devo memorizzare l'indirizzo di ogni blocco**. Mi basta **l'indirizzo al primo blocco** e poi la dimensione del file per capire quanti blocchi esso occupa.

- Nel caso della tabella a destra, ho il nome del file (count) l'inizio (2) e la dimensione in numero di blocchi (2).

I limiti di questo metodo riguardano:

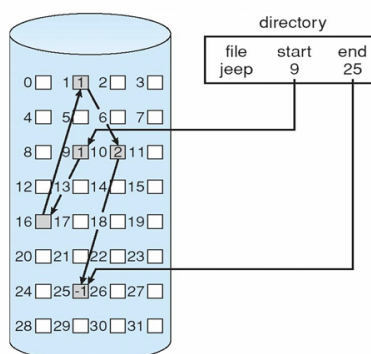
- **External fragmentation**, file allocati e cancellati possono lasciare insieme di blocchi contigui troppo piccoli per futuri file. Per risolvere posso fare **Compacting** ⇒ Riorganizzazione dei file su disco per eliminare gli spazi vuoti.
- Se non sappiamo per certo la dimensione di un file (as esempio ne aumentiamo la dimensione) e' possibile che **non si possa piu' estendere**.

Per migliorare quest'ultimo problema si applica un

Extent-Based Systems:

- Uno spazio contiguo di N blocchi (cluster) viene allocato inizialmente.
- Se lo spazio non basta viene aggiunto un altro spazio contiguo (extent) e ne salvo il puntatore.
- Il limite in questo caso e' che i **cluster devono avere medesima dimensione**.

7.1.2 Linked Allocation



Ogni file ha una lista linkata dei blocchi in cui e' memorizzato. Ogni **blocco contiene quindi un puntatore al prossimo blocco**. Questo comporta che localizzare un blocco puo' richiedere molte operazioni I/O o disk seeks. Questo approccio permette di **risolvere il problema di external fragmentation**.

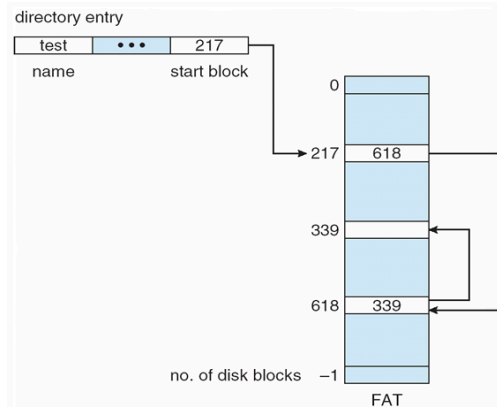
Il problema di questo approccio e' che mentre nell'allocazione contigua se

voglio accedere direttamente al terzo blocco so dove lo posso trovare. Nel caso di quest'altro approccio **non so la posizione del blocco i-esimo senza guardare il puntatore contenuto nel blocco-i-1**.

Invece di avere ogni blocco che contiene il puntatore al successivo, posso suddividere in cluster di blocchi contigui il file. **Invece di avere un blocco che punta al prossimo, avro' un cluster che punta al successivo**.

FILE ALLOCATION TABLE

Variazione sull'allocazione linkata. Una **parte della memoria viene usata per memorizzare questa struttura**. Si memorizza il primo blocco da cui si parte per memorizzare il file. Poi **per ogni blocco si ha un entry sulla FAT contenente l'indirizzo del prossimo blocco**. In questo modo



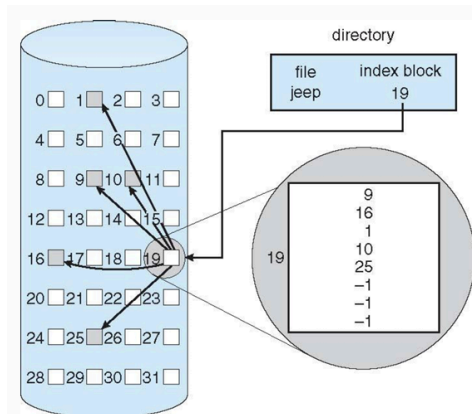
per **accesso diretto, invece di scandire la memoria di massa, scandisco questa FAT** (memorizzata in memoria piu' veloce solitamente).

Quindi invece di avere il puntatore al blocco successivo nel blocco i-esimo del file ho questa FAT dove memorizzo solo gli indirizzi.

Quindi nella FAT ho **per ogni entry il numero del blocco di dati**.

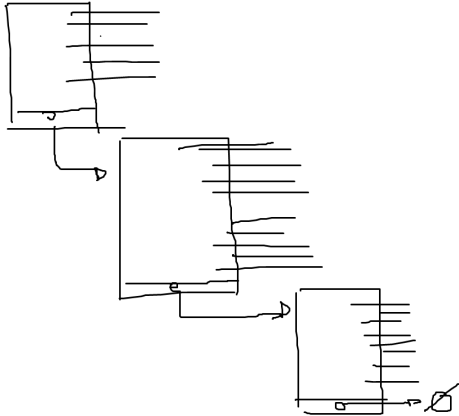
Scandendo la FAT a partire da una determinata directory entry ottengo l'**insieme dei numeri di blocchi dato che corrisponde al file i-esimo della directory**.

7.1.3 Allocazione indicizzata

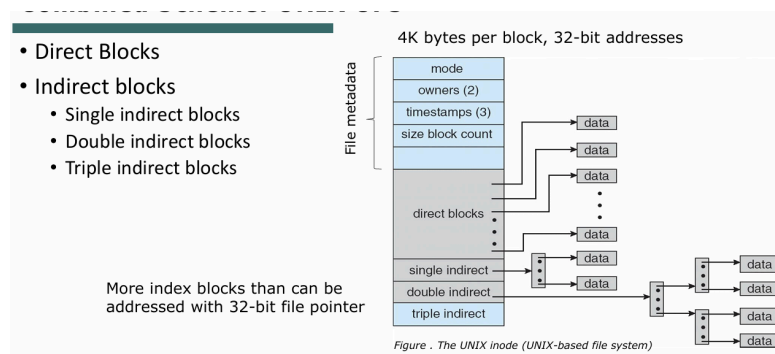


Nell'allocazione linkata ogni blocco aveva i dati del file e il puntatore al prossimo blocco. Nell'allocazione **indicizzata si dedicano alcuni blocchi di memoria interamente per la memorizzazione dei puntatori ai blocchi relativi ad un file**.

In questo modo risolvo il problema dell'accesso diretto.



Posso anche avere uno schema di allocazione indicizzata di questo tipo, dove ho un **blocco indice che contiene i puntatori ai blocchi dato**, e poi ha un **puntatore al prossimo blocco indice**.



La strategia che si usa realmente e' una **strategia mista**.

Oltre ai metadata un file contiene altre informazioni. Una parte dei blocchi di un file vengono **allocati in modo diretto**, quindi ho

direttamente l'indirizzo di tali blocchi.

Se il file risulta essere troppo grande quello che posso fare e' memorizzare dei **puntatori indiretti ai blocchi**. Essi possono essere:

- **Single indirect blocks**, puntatore ad un blocco che contiene altri puntatori ai blocchi in cui memorizzo parti del file.
- **Double indirect blocks**.
- **Triple indirect blocks**.

Se ho indirizzi a 32 bit ad esempio:

- Direct blocks => 10 indirizzi a 32 bit che puntano a 10 blocchi da 4kb ciascuno.
- Single indirect => 1 indirizzo a 32 bit. Che punta ad un blocco di 4kB che contiene solo indirizzi da 32 bit, per un totale di 1024 ($4kb / 32 \text{ bit} = 1024$) puntatori. Quindi in totale posso puntatore fino a 1024 blocchi da 4KB.

- Double indirect blocks $\Rightarrow$ doppio single indirect, quindi $1024 * 1024 * 4KB$

7.2 Free-space management

Una volta che si cancella un file e' necessario capire come **tracciare i blocchi di memoria che vengono liberati**:

- **Bitmap o bit vector** $\Rightarrow$ Ad ogni blocco viene dedicato un bit. Se ad 1 il blocco e' disponibile, se a 0 allora il blocco e' occupato. Il problema e' che devo avere un numero di bit pari al numero di blocchi in memoria. E per trovare un blocco libero devo scandire l'intera bitmap. Per cercare un numero N contiguo di blocchi liberi la complessita' rimane invariata, sempre $O(N)$, e lo stesso se si vuole allocare un numero N di blocchi liberi.

- **LISTA CONCATENATA (FAT)** $\Rightarrow$ Ovvero ogni blocco libero ha il puntatore al prossimo blocco libero in memoria. Anche in questo caso come in 7.1.2 posso creare cluster di blocchi liberi, in modo da puntare direttamente a questi.

La **FAT** puo' essere utilizzata anche per **tenere traccia dei blocchi liberi in memoria**. In particolare, si mantiene un unico puntatore (**head_ptr**) che indica il primo blocco libero del volume. Se questo puntatore ha il valore k allora k e' il primo blocco libero e $FAT[k]$ conterra' l'indice del secondo blocco libero e cosi' via. Se $FAT[x] = EOF$ (valore speciale) allora si ha fine della lista.

Questa struttura risulta essere **particolarmente efficiente se serve un solo blocco libero** ad esempio, in quanto **in questo caso basta guardare il blocco puntato da head_ptr (Complessita' $O(1)$)**.

Se invece si vogliono **cercare N blocchi liberi** nella FAT, la complessita' risulta essere $O(N^2)$. **Se la free list e' ordinata questo richiede $O(N)$** .

Per quanto riguarda l'**allocazione** di blocchi contigui, se la fat e' ordinata questa operazione richiede una complessita' di $O(N)$, **altrimenti di $O(N*NF)$** con NF =numero di frame ed N =numero blocchi liberi.

- **Counting** $\Rightarrow$ Invece che tenere una lista di n indirizzi liberi, e' sufficiente tenere l'indirizzo del primo blocco libero e il numero n di blocchi liberi contigui che seguono il primo blocco.

CONFRONTO BITMAP E LINKED LIST

- Le bitmap hanno performance migliori per quanto riguarda la ricerca di blocchi contigui liberi in quanto, a differenza di una FAT-free-list, posso avere performance logaritmiche $O(\log n)$.
- Le free-list sono ottime per sistemi che hanno memoria paginata, in quanto le pagine non necessitano di essere allocate in locazioni contigue. La memoria kernel d'altro canto, che deve essere gestita in modo contiguo (vedi 3.10) non ha vantaggi.
- In un file system con i-node, che non necessitano che i blocchi siano contigui, un sistema a bitmap e' sfavorito rispetto ad una linked list per la gestione dello spazio libero. Infatti se si vuole trovare un singolo blocco libero:
 - LinkedList $\Rightarrow O(1)$.
 - Bitmap $\Rightarrow O(N)$.
- Lo stesso vale per un sistema basato su FAT, in quanto anch'esso non necessita di blocchi contigui.

RACCOLTA ESERCIZI

Memory management

Consider a logical address space of 64 pages of 1,024 words each, mapped onto a physical memory of 32 frames.

- How many bits are there in the logical address?
- How many bits are there in the physical address?

64 pagine -> ho bisogno di 6 bit (page number)
1024 parole, quindi ho bisogno di 10 bit (offset)
logical address ha in totale 16 bit.

il physical address ha offset uguale, quindi 10. Ci sono 32 frame quindi 5 bit
physical memory = 15 bit

- Assuming a 1.KB page size, what are the page numbers and offsets for the following address references (provided as decimal numbers):

- 3085 $3085 / 1024 = 3$ e resto di 13 (offset)
- 42095
- 215201
- 650000
- 2000001

- Answer:

- Page = 3, offset = 13,
- Page = 41, offset = 111,
- Page = 210, offset = 161
- Page = 634, offset = 784
- Page = 1953, offset = 129

- Consider a computer system with a 21-bit virtual address that has only a 16-bit physical address. It also has a 2KB page size. How many entries are there in each of the following?

- A conventional, single-level page table
- An inverted page table

Logical address di 21 bit.
Physical di 16 bit
page size di 2kb.

Standard page table:
 $2\text{kb} = 11$ bit di offset
e quindi 10 bit ($21-11$) = 10 bit di numero di pagine
La page table ha quindi 2^{10} entry.

Inverted page table:
Ho sempre un offset di 11 bit.
Quindi mi rimangono $16-11 = 5$ bit per i frame.
Quindi avrò massimo 2^5 entry nella page table.

Sia dato un sistema di memoria virtuale con paginazione, nel quale vengono indirizzati i Byte. Il sistema dispone di **TLB** (Translation Look-aside Buffer), su cui si misura sperimentalmente un **“hit ratio” del 99 %**. La tabella delle pagine (**“page-table”**) viene realizzata con uno schema **a due livelli**, nel quale un **indirizzo logico di 32 bit** viene suddiviso (da MSB a LSB) in 3 parti: **p1, p2, d, rispettivamente di 10 bit, 11 bit, 11 bit**. Non si utilizzano ulteriori strutture dati (quali tabelle di hash o inverted page table) per velocizzare gli accessi.

- Si dica che cosa si intende con **“hit ratio”**
- Si illustri lo schema della page-table e la sua dimensione complessiva, per un processo **P1** avente spazio di indirizzamento virtuale di **128 MByte**.
- Supponendo che la memoria RAM abbia tempo di accesso di 300 ns, si calcoli il tempo effettivo di accesso (EAT) per il caso proposto (hit ratio = 99 %)

Si ha quindi page table a 2 livelli con 32 bit di indirizzi logici, quindi:

p1=10

p2=11

d=11 -> 2^{11} = page size quindi

Serve dimensione complessiva di page table per gestire processo di 128MByte (non dimensione massima!)

Numero minimo frame = process size / page size = (128MB) / 2^{11} = 2^{16} frame

Si hanno due livelli di page table. Ogni inner table ha dimensione 2^{11} (2^{11} entry) quindi con ogni inner table si può puntare a 2^{11} frame, servono quindi

2^{16} inner table -> (2^{16} / 2^{11}).

Quindi per riferire a 2^{16} inner table la outer table dovrà avere 2^{16} entry, alla fine per soddisfare il processo P1 la dimensione complessiva dello schema di page table sarà

outer_table(2^{16} * 4Byte) + inner_table(2^{11} * 2^{16} * 4B) =

Non si specificava nessun bit di validità ecc, quindi non si considerano.

EAT = hit_ratio*MM + (1-hit_ratio)*3MM = $0.99 \cdot 300\text{ns} + 0.01 \cdot 900\text{ns}$ ---> ESSENDO PAGE TABLE A 2 LIVELLI GLI ACCESSI TOTALI IN CASO DI MISS SONO 3!!

Sia dato un processo avente **spazio di indirizzamento virtuale di 32 GB**, dotato di **8GB di RAM**, su una architettura a 64 bit (in cui si indirizza il Byte), con gestione della memoria paginata (**pagine/frame da 1KB**). Si vogliono confrontare una soluzione basata su **tabella delle pagine standard** (una tabella per ogni processo) e una basata su **IPT**. Si calcolino le dimensioni della tabella delle pagine (ad un solo livello) per il processo e della IPT. Si ipotizzi che il pid di un processo possa essere rappresentato su 16 bit. Si utilizzino 32 bit per gli indici di pagina e/o di frame. Si dica infine, utilizzando la IPT proposta (32 bit per un indice di pagina/frame).

memoria logica = 32gb

memoria fisica = 8gb

con architettura a 64 bit con pagine da 1kb, si vuole confrontare standard con inverted page table.

PID a 16 bit e ogni entry della page table è di 4 byte (indici a 32 bit).

Quindi ci vogliono 4 byte per puntare ai frame della memoria, per questo ogni entry 4 byte.

Standard

(memoria) / (size pagine) = numero pagine = (32gb)/(1kb) = 2^{25} pagine

Ci sono quindi 2^{25} entry nello standard page table, la dimensione di ogni page table sarà

(2^{25} * 4B) = 128MB

Inverted page table

numero frame = 8gb/1kb = 2^{23} .

Quindi ci sono 2^{23} entry, ogni entry ha PID e i 32bit per indirizzare la pagina.

Quindi (2^{23} * (16+32)) = 48MB di dimensione della inverted page table. (Inferiore alla standard giustamente)

Virtual memory

int A[100][100] - pages of size 200 - For three-page frames, **LRU replacement - page frame 1 contains process**

```
a. for (int j = 0; j < 100; j++)
    for (int i = 0; i < 100; i++){
        A[i][j] = 0;
    }
```

answer = 5,000

```
b. for (int i = 0; i < 100; i++)
    for (int j = 0; j < 100; j++){
        A[i][j] = 0;
    }
```

answer = 50

Bisogna capire innanzitutto quante pagine sono allocate per la matrice:

$A[i][j] = A[0][0] - A[0][1] \dots A[0][99] \approx 100$ elementi, la pagina è da 200 quindi ancora non la ha occupata tutta

$A[1][0] \dots A[1][99]$ occupo la prima pagina

quindi la prima pagina va da

$A[0][0]$ a $A[1][99]$ -> due righe della matrice per pagina quindi, essendo la matrice 100x100

In totale ho 50 pagine che devo dedicare a questa matrice.

a)

$A[0][0]$ -> serve pagina 0 -> page fault

$A[1][0]$ -> pagina 0 -> ho già 0, no page fault

$A[2][0]$ -> pagina 1 -> page fault

$A[3][0]$ -> pagina 1 -> no page fault

Ogni due elementi devo quindi accedere ad una nuova pagina. Solo il loop interno (100 iterazioni dove ogni 2 mi

chiede una pagina) mi richiede accesso a 50 pagine diverse, quindi 50 page fault. Il loop esterno viene eseguito

100 volte, quindi $100 \times 50 = 5000$. Ho 3 frame liberi, 1 sempre occupato, quindi ne ho 2 disponibili.

b)

Si accede a:

$A[0][0]$ -> serve pagina 0 (page fault) -> Nel loop interno $A[0][0] \dots A[0][99]$, non mi serve altro, sempre solo pagina 0

$A[1][0]$ -> sempre pagina 0 quindi niente page fault, nel loop interno arrivo fino a $A[1][99]$, quindi niente page fault

$A[2][0]$ -> serve pagina 1 (page fault) ma nel loop interno sempre niente page fault.

Quindi ho page fault solo relativamente a loop esterno e, essendo che mi serve una pagina diversa ogni 2 righe,

avro' in totale solo $100/2 = 50$ page faults.

Quindi meglio fare il loop come nel secondo caso ooooooooooooooli

Si consideri la seguente sequenza di riferimenti in memoria nel caso di un programma di 1000 parole:

261, 409, 985, 311, 584, 746, 632, 323, 470, 915, 858.

page size = 200 parole
quindi pagina 0 va da 0 a 199, la pagina 1 da 200 a 399 e così via.

261 fa quindi riferimento alla pagina 1
409 alla 2, 985 alla 4 e così via come specificato sotto

passi:

- Pagina 1 già in memoria. Ha reference bit pari a 0, quindi il suo reference bit si mette ad 1 -> FIFO = 4, 3, 1
- Pagina 2 non è in memoria, quindi cerco la pagina da sostituire. 4 è il primo della fifo, ha reference bit a 0, quindi lo tolgo e metto 2. la fifo si aggiorna: 3(1), 1(1), 2(0) -> l'ultimo messo viene messo come ultimo elemento
- Pagina 4 non in memoria, inizio da 3(1), avendo reference bit ad 1 passo al successivo e metto a 0 reference bit a 0. Lo stesso per 1(1). Poi 2(0) lo sostituisco. FIFO : 3(0), 1(0), 4(0).
- Pagina 1 già in memoria, si mette ad 1 il suo reference bit. FIFO: 3(0), 1(1), 4(0).
- Pagina 2 va messa. Sostituisco 3 -> FIFO: 1(1), 4(0), 2(0).
- Pagina 3 va messa. Sostituisco 4(0) e abbasso 1(0). FIFO: 1(0), 2(0), 3(0)
- Per i prossimi 2 ci sono già, aggiorno solo i reference bit : FIFO 1(1), 2(1), 3(1)
- Per pagina 4 ciclo la fifo la prima volta metto tutti i reference a 0 e quindi al secondo ciclo 1 viene sostituito : FIFO : 2(0), 3(0), 4(0)

	261	409	985	311	584							
	1	2	4	1	2	3	3	1	2	4	4	
4 ₀	4(0)	2(0)	4(0)		4(0)	3(0)	3(1)	3(1)	3(1)	3(0)	3(0)	
3 ₁	3(1)	3(1)	3(0)		2(0)	2(0)	2(0)	2(0)	2(1)	4(0)	4(1)	
1 ₀	1(1)	1(1)	1(0)		1(1)	1(0)	1(0)	1(1)	1(1)	1(0)	1(0)	
		fault	FAULT		fault	FAULT				FAULT		

Page-Fault Frequency Algorithm

Tempo	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
Stringa	6	4	4	3	2	4	4	4	1	3	3	2	4	4	5	1	2	2	2	6	1	2	5	4
Frame 0	6	6	6	6	6	6	6	6	1	1	1	1	1	1	5	5	5	5	5	6	6	6	6	6
Frame 1		4	4(1)	4	4	4(1)	4(1)	4(1)	4(0)	4	4	4	4(1)	4(1)	4(0)	4	4	4	4		1	1	1	1
Frame 2				3	3	3	3	3		3	3(1)	3(1)	3(1)	3(1)	3(0)	3	3	3	3				5	5
Frame 3					2	2	2	2				2	2	2		1	1	1	1					4
Frame 4																	2	2(1)	2(1)	2(0)	2	2(1)	2(1)	2(1)
Fault	x	x		x	x				x	x	x	x			x	x	x			x	x		x	x
Ref Bit									x						x					x				

C = 3

d = 1
d < C,
quindi 4 si aggiunge

d = 2
d < C,
3 si aggiunge

d = 4
d > C
rimuovo
pagine che
hanno ref bit
a 0

d = 3
rimuovo
pagine
con ref bit
a 0 e metto
a quelle con
ref bit a 1 = 0

d è la distanza tra il page fault attuale e quello precedente.

Se tutti hanno reference bit ad 1 e una pagina non è tra i frame -> si mettono tutti i reference bit a 0 e si sceglie il primo da togliere (come second chance)

3(a). Si consideri la seguente sequenza di riferimenti in memoria nel caso di un programma in cui, per ogni accesso (indirizzi in **esadecimale**, si indirizza il Byte), si indica se si tratta di lettura (R) o scrittura (W): **R 3F5, R 364, W 4D3, W 47E, R 4C8, W 2D1, R 465, W 2A0, R 3BA, W 4E6, R 480, R 294, R 0B8, R 14E.**

Supponendo che sia **indirizzi fisici** che logici siano su **12 bit**, che si usi paginazione con pagine di dimensione **128 Byte** e che il massimo indirizzo utilizzabile dal programma sia **C10**, si dica **quante pagine sono presenti nello spazio di indirizzamento** del programma e se ne **calcoli la frammentazione interna**.

C10 >> 1100 0001 0000

Page size= 128 byte (2⁷) >> page offset = 7 bits

C10 >>

11000 0010000

Page Number

Page Offset

Max Page number = 2³+2⁴= 24

Fragmentation:

Last page is filled until 0010000 = 2⁴ = 16 bytes (0 >> 16 = 17 bytes)

Fragementation >> page size (128) – used one (17 bytes) = 111 bytes

3(b). Si determini la stringa dei riferimenti a pagine (Si consiglia di passare da esadecimale a binario, per determinare correttamente il numero di pagina e, se necessario, il displacement/offset). **R 3F5, R 364, W 4D3, W 47E, R 4C8, W 2D1, R 465, W 2A0, R 3BA, W 4E6, R 480, R 294, R 0B8, R 14E.** Supponendo che sia **indirizzi fisici** che logici siano su **12 bit**, che si usi paginazione con pagine di dimensione **128 Byte**.

3F5	00111 1110101	7
364	00110 1100100	6
4D3	01001 1010011	9
47E	01000 1111110	8
4C8	01001 1001000	9
2D1	00101 1010001	5
465	01000 1100101	8
2A0	00101 0100000	5
3BA	00111 0111010	7
4E6	01001 1100110	9
480	01001 0000000	9
294	00101 0010100	5
0B8	00001 0111000	1
14E	00010 1001110	2

B) Si determini la stringa dei riferimenti a pagine (Si consiglia di passare da esadecimale a binario, per determinare correttamente il numero di pagina e, se necessario, il displacement/offset). Si utilizzi un algoritmo di sostituzione pagine di tipo **LRU** (Least Recently Used). Si assuma che siano disponibili 3 frame, agli indirizzi fisici (espressi in esadecimale) **780, A00, B00**. Si richiede la visualizzazione (dopo ogni accesso) del resident set (i frame fisici contenenti pagine logiche).

Determinare quali e quanti page fault (accessi a pagine non presenti nel resident set) si verificheranno. Si dica infine a quali indirizzi fisici vengono effettuati gli accessi (tra quelli sopra elencati) R 3F5, W 4D3, R 3BA

Riferimenti		7	6	9	8	9	5	8	5	7	9	9	5	1	2
Resident Set	780	7	7	7	8		8			8	9			9	2
	A00		6	6	6		5			5	5			5	5
	B00			9	9		9			7	7			1	1
Page Fault		X	X	X	X		X			X	X			X	X

3(d). Si dica infine a quali indirizzi fisici vengono effettuati gli accessi (tra quelli sopra elencati) **R 3F5, W 4D3, R 3BA**

Logical Address	Page Address (binary)	Page Number	Frame Address	Frames Physical Address	Page Physical Address
3F5	00111 1110101	7	780	01111 0000000	01111 1110101
4D3	01001 1010011	9	B00	10110 0000000	10110 1010011
3BA	00111 0111010	7	B00	10110 0000000	10110 0111010

Exercise (011/09/2018):

A) Si consideri la seguente sequenza di riferimenti in memoria nel caso di un programma di 4K parole in cui, per ogni accesso (indirizzi in esadecimale), si indica se si tratta di lettura (R) o scrittura (W): W 3A1, R 3F5, R A64, W BD3, W 57E, R A08, R B85, W 3A0, R A1A, W A36, R B20, R 734, R AB8, R C4E, W B64.

lettura e scrittura per eventuale modify bit

Si determini la stringa dei riferimenti a pagine, supponendo che la loro dimensione sia di 512 parole. Si utilizzi un algoritmo di sostituzione pagine di tipo **Enhanced Second-Chance**, per il quale, al **bit di riferimento** (da inizializzare a 0 in corrispondenza al primo accesso a una nuova pagina dopo il relativo page fault), si unisce il **bit di modifica** (modify bit). Si assuma che una pagina venga sempre modificata in corrispondenza a una scrittura (write), che siano disponibili 3 frame e che l'algoritmo operi con il criterio seguente: dato il puntatore alla pagina corrente (secondo la strategia FIFO) si fa un primo giro, senza modificare il reference bit, sulle pagine per localizzare la vittima (l'ordine di priorità è (reference, modify):

(0,0), (0,1), (1,0), (1,1)); una volta determinata la vittima, si fa un secondo giro per azzerare i reference bit delle pagine "salvate" (comprese tra la posizione di partenza e la vittima).

Determinare quali e quanti page fault (accessi a pagine non presenti nel resident set) si verificheranno. Si richiede la visualizzazione (dopo ogni accesso) del resident set, indicando per ogni frame i bit di riferimento e modifica. Si numerino le pagine a partire da 0.

4(a). un programma di **4K parole** in cui, per ogni accesso (indirizzi in esadecimale), si indica se si tratta di lettura (R) o scrittura (W): **W 3A1, R 3F5, R A64, W BD3, W 57E, R A08, R B85, W 3A0, R A1A, W A36, R B20, R 734, R AB8, R C4E, W B64**. Si determini la **stringa dei riferimenti a pagine**, supponendo che la loro dimensione sia di **512 parole (2⁹)**

3A1	001110100001	1 (si guardano i primi bit a sinistra)
3F5	001111110101	1
A64	101001100100	5
BD3	101111010011	5
57E	010101111110	...
A08	101000001000	
B85	101110000101	
3A0	001110100000	
A1A	101000011010	
A36	101000110110	
B20	101100100000	
734	011100110100	
AB8	101010111000	
C4E	110001001110	
B64	101101100100	

stringa dei riferimenti a pagine = 1,1,5,5,2,5,5,1,5,5,3,5,6,5

process size = 4k parole
page size = 512 parole

calcoliamo prima di tutto i bit di indirizzamento
pagine da 512 parole, quindi $2^9 \rightarrow 9$ bit di offset
processo ha $4k/512 = 8$ pagine = $2^3 \rightarrow 3$ bit di page number per il processo

quindi indirizzi a 12 bit.
I primi 3 rappresentano la pagina
Gli altri 9 l'offset

4(b). Si utilizzi un algoritmo di sostituzione pagine di tipo **Enhanced Second-Chance**, per il quale, al **bit di riferimento** (da **inizializzare a 0** in corrispondenza al primo accesso a una nuova pagina dopo il relativo page fault), si unisce il bit di modifica (modify bit). Si assuma che una **pagina venga sempre modificata** in corrispondenza a una scrittura (**write**), che siano disponibili 3 frame e che l'algoritmo operi con il criterio seguente: dato il puntatore alla pagina corrente (secondo la strategia FIFO) si fa un primo giro, senza modificare il reference bit, sulle pagine per localizzare la vittima (l'ordine di priorità è (reference, modify). una volta determinata la vittima, si fa un secondo giro per azzerare i reference bit delle pagine "salvate" (comprese tra la posizione di partenza e la vittima).

Non si modifica il ref bit di 5, in quanto inserendo 3 esso e' nel top della filo, quindi il controllo parte da esso.
In questi esercizi, meglio tenere traccia della FIFO in ogni caso

Riferimenti	1	1	5	5	2	5	5	1	5	5	5	3	5	6	5
Read/Write	W	R	R	W	W	R	R	W	R	W	R	R	R	R	W
Resident Set	1_0,1	1_1,1	1_1,1	1_1,1	1_1,1	1_1,1						1_0,1	1_0,1	1_0,1	1_0,1
			5_0,0	5_1,1	5_1,1	5_1,1						5_0,1	5_1,1	5_1,1	5_1,1
					2_0,1	2_0,1						3_0,0	3_0,0	6_0,0	6_0,0
Page Fault	x		x		x									x	

si mette modify

Si toglie 2 in quanto ha ref bit a 0. Agli altri si mette ref a 0. Se avessimo avuto altre pagine, si controllava se c'era una con (0,0) e in caso si sceglieva quella!!!!!!

Exercise (10/09/2024):

1. Consider a 64-bit system with a virtual address space of 248 bytes, a page size of 4 KB, and 16 GB of physical memory. Assume the system uses an inverted page table. Calculate and explain the following:

- The number of bits required for the physical frame number.
- The total number of entries in the inverted page table.
- The virtual address given to a process is 0x00007FFFFFFF000. Determine the physical address if this virtual address is mapped to the 1,024th physical frame. Show all steps, including the calculation of page number and offset.

Answers:

A. Physical Frame Number Calculation:

- Page size = 4 KB = 2^{12} bytes.
- Physical address space = 16 GB = 2^{34} bytes.
- Number of physical frames = $2^{34}/2^{12} = 2^{22}$.
- Therefore, the number of bits required for the physical frame number is 22 bits.

Inverted page table -> unica page table condivisa e ad ogni frame si ha entry nella page table

Offset -> da page size = 4KB = 2^{12} -> 12 bit di offset
frame number: $2^{34}/2^{12} = 2^{22}$, ovvero 16GB/4KB = numero di frame totali. Essendo inverted page table si guarda appunto la memoria fisica. Quindi 22 bit di frame number

Il numero totale di frame/entry nella page table e' quindi 2^{22} , che nell'inverted page table corrisponde appunto al numero di frame totali della memoria
Dato 0x00007... i primi 12 bit meno significativi sono dedicati all'offset. Quindi l'offset e' 000
Il 1024esimo frame sarebbe 0x0000400 (mappato in 22 bit, che sono appunto il frame number calcolato prima)
Quindi a questo si aggiungono i 12 bit di offset:
0x00000400000

Mass Storage

- Sia dato un disco organizzato con struttura DAIR. Che cosa si intende, in relazione a tale disco e alla probabilità di guasto e/o tempo medio tra Guasti (MMTF), con intermini seguenti?
 - Mean Time to Repair (MTTR) – Mean Time to Data Loss (MTTDL)
 - Si consideri una struttura DAID con 2 dischi in configurazione “mirrored”. Se ognuno dei dischi può guastarsi in modo indipendente dall’altro, con MMTF = 5000 ore e MTTR = 20 ore, quanto sarà il MTTDL?
- $MTTDL = MMTF^2 / (2 * MTTR) = 50000^2 / (2 * 20) = 25 / 4 * 10^7 \text{ ore} = 6.25 * 10^7$

Exam 07/07/2023

Consider a Hard Disk Drive (HDD) with standard technology and geometry. The disk has

- sector size of 512 Bytes,
- 2480 tracks per surface, 2048 tracks
- 50 sectors per track,
- 5 double-sided platters,
- average seek time of 10 msec.

1. What is the capacity of a track in bytes? What is the capacity of each surface? What is the capacity of the disk? How many cylinders does the disk have?

2. The CHR (Cylinder-Head-Sector) disk addressing method allows a disk block to span over multiple sectors of a given track (identified by a (cylinder,head) pair). Are the following valid block sizes: 256 B, 2048 B, 51200 B?

3. If the disk platters rotate at 5400 rpm (revolutions per minute), what is the maximum rotational delay? What is the average rotational delay? If one track of data can be transferred per revolution, what is the transfer rate

1. Ogni traccia = $512B * 50$ (in quanto ogni traccia ha 50 settori da 512 bytes) = 25KB.
Capacità superficie = Ogni superficie ha 2048 tracks, allora $2048 * 25KB = 50MB$.
Capacità del disco = $50MB * 2$ per platter, avendo 5 platter = $100MB * 5 = 500MB$.
Il numero di cilindri e' pari al numero di tracks.

2. La dimensione dei blocchi che posso leggere deve essere multiplo del settore, il quale e' la dimensione MINIMA a cui posso accedere. Inoltre il massimo corrisponde alla dimensione della traccia.

3. $60s/5400 = 11ms$ e tempo medio = 5.5ms
transfer rate per un track di data -> un track di data sono 25kb -> $25k/revolution = 25KB/(60/5400) = 90 * 25KB/s = 2.25 * 8Mbit/s$. Una traccia e' circolare, quindi serve l'intero giro di rivoluzione, c'e' scritto nella consegna della traccia.

FILE SYSTEM

Siano dati due file system F1 e F2, su due volumi, basati rispettivamente su FAT e su Inode. I **puntatori hanno dimensione di 32 bit**, i blocchi hanno dimensione 4KB, entrambi i volumi hanno una parte riservata ai metadata (direttori, FAT e FCB –file control block) oppure Inode, blocchi indice, ecc) e una parte ai blocchi di dato. Si supponga che lo spazio riservato ai blocchi di dato sia lo stesso per F1 e F2. Si sa che la **FAT occupa 150 MB**.

- Siano dati un file “a.mp4” (in F1) di dimensione 317 MB e un file “b.mp4” (in F2) di dimensione **751 MB**.

F) Quanti blocchi di dato e di indice (escluso l’inode) occupa b.mp4? (10 direct blocks, 1 single indirect blocks and 1 double indirect blocks)

b(inode) = 751MB, pointers da 32 bit e dimensione di 4KB.

Il numero di blocchi di data che il file richiede sono -> $\text{file_size} / \text{block_size} = 751\text{MB} / 4\text{KB} = 187.75\text{Kblocks} = 188\text{ Kblocks}$

Devo capire dei vari blocchi disponibili, devo capire quanti ne uso effettivamente

Ho 10 direct block, quindi 10 dei 188k sono allocati tramite direct blocks -> $188k - (10 \times 4k) = 178k$
Single indirect -> $4k / 32\text{ bit} = 1024$ puntatori, quindi posso puntare a 1k di blocchi -> $178k - (10 \times 4k) = 178k = 191478\text{ blocchi}$
Double indirect -> ho $1k \times 1k$ blocchi -> per un totale di 1024k blocchi possibili che posso puntare. Il mio file e' solo 188k, quindi non mi servono tutti
Ma $191478 / 1024 = 187$ blocchi di indice -> quindi da 1024k blocchi me ne servono solo 187 blocchi di indice
In totale $187\text{ double indirect} + 1\text{ direct access} + 1\text{ single indirect} = 189$

A) Quanti blocchi di dato possono contenere complessivamente, al massimo, i file presenti nei due file system?

G) Quanti blocchi di dato occupa a.mp4?

H) Quanti indici nella FAT sono utilizzati per a.mp4?

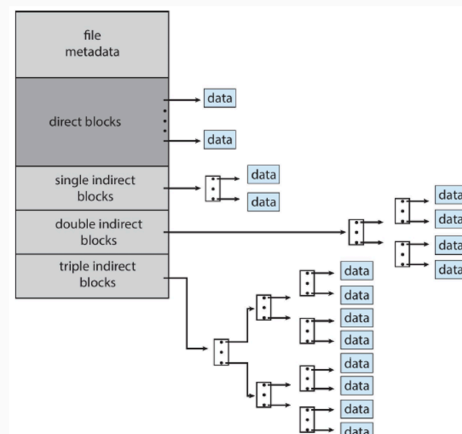
Sia dato un file system Unix, basato su inode aventi 13 puntatori (**10 diretti, 1 indiretto singolo 1 doppio e 1 triplo**). I **puntatori hanno dimensione di 32 bit** e i **blocchi hanno dimensione 2KB**. Si sa che il file system contiene 1000 file di dimensione media 15MB e che la frammentazione interna totale è di 1MB, si calcoli il massimo numero possibile di file con indice indiretto triplo e con indice indiretto doppio che possono essere presenti nel file system.

pointer_size = 32 bit.
Block_size = 2KB.
1000 file da mediamente 15MB con frammentazione interna totale di 1MB

Total_file_size = $1000 \times 15\text{MB} + 1\text{MB} = 15\text{GB}$

single indirect = $2k / 32\text{ bit} = 512$ blocchi
double indirect = 512×512
data blocks = 10 (diretti) + 512 (single indirect) + 1 (un solo blocco per il double indirect) = 523
quindi 523 blocchi minimi per usare il double indirect -> quindi dimensione minima dei file in blocchi (oppure dimensione file -> $523 \times 2k$).

data blocks (contando triple indirect) = 10 (diretti) + 512 (single) + 512^2 (double) + 1 (un solo minimo per arrivare al triplo).



Quindi si hanno 1000 file da 15mb in media l'uno. Questo significa che il file system ha una dimensione di $1000 \times 15\text{mb} = 15\text{GB}$.

Dato un blocco con dimensione 2KB, il numero di blocchi per contenere questi file e' di: $15\text{GB} / 2\text{KB} = 15001 \times 512$ blocchi.
Ora calcolo il numero di blocchi che un double indirect puo' puntare:

MIN2 = $(10 + 512 + 1) = 523$ blocchi.

MIN3 = $(10 + 512 + 512^2 + 1) = x$ blocchi

Ora supponendo che tutti i file abbiano la medesima dimensione, ovvero che occupino una dimensione tale che possano essere puntati dal minimo double indirect, avrei:

$15001 \times 512 / 523$ blocchi = 14000 file → ovvero se tutti i file fossero double indirect per riempire il file system dovrei avere 14000 file, ma visto che ne ho 1000 (massimo numero presente) sforo. Quindi, ORA, con un massimo di 1000 file, con double indirect non riuscirei a, ORA, coprire l'intero file system come da condizione iniziale.

Se invece fossero tutti triple indirect:

$15001 \times 512 /$ tanti blocchi = 29, quindi con triple indirect riuscirei a coprire l'intero file system con 29 file se essi appunto occupassero il minimo per essere puntati da triple indirect.

RACCOLTA DOMANDE OS161

Si descrivano le caratteristiche di semafori e locks evidenziandone le differenze.

Cosa sono i wait_channel? Per cosa sono utilizzati all'interno del sistema operativo?

- I semafori e locks sono meccanismi di sincronizzazione che vengono utilizzati quando piu' thread devono accedere in maniera concorrente a delle risorse comuni. In particolare, i semafori sono meccanismi che si basano su un contatore che determina il numero di risorse disponibili e su due funzioni:

- P() -> Se il contatore del semaforo e' > 0 si decrementa il valore del contatore e si entra in sezione critica. Altrimenti se il contatore e' = 0 il thread rimane in attesa.

- V() -> Uscito dalla sezione critica il thread incrementa il contatore per far eventualmente entrare un altro thread nella sezione critica.

I lock o mutex sono particolari tipologie di semafori che prendono il nome di semafori binari, in quanto il contatore associato puo' assumere solo valore 0 od 1. Mentre quindi con i lock se la risorsa e' bloccata (counter = 0) il processo entra in attesa (da ready passa alla waiting queue), nel caso di spinlock i processi rimangono in attesa attiva (rimangono nella ready queue), occupando tempo di CPU. Gli spinlock sono in generale piu' veloci in quanto il thread non deve andare avanti e indietro tra wait e ready queue e non causano cambio di contesto, ma al contempo occupano la CPU. Per questo motivo e' consigliato usarli solo quando la sezione critica e' molto corta.

- I wait_channel sono strutture dati kernel utilizzate per sospendere e risvegliare thread in attesa di una condizione. Per questo motivo vengono utilizzati per realizzare lock, semafori e condition variable.

Si riporta la funzione wchan_wakeone:

```
wchan_wakeone(struct wchan *wc, struct spinlock *lk) {  
    struct thread *target;  
    KASSERT(spinlock_do_i_hold(lk));  
    target = threadlist_remhead(&wc->wc_threads);  
    if (target == NULL) { return; }  
    thread_make_runnable(target, false);  
}
```

Si motivi l'istruzione KASSERT(spinlock_do_i_hold(lk));

Perché si rimuove un thread dalla lista wc->wc_threads ?

- L'istruzione KASSERT(..) viene utilizzata per verificare che il thread che ha acquisito il lock e' lo stesso che sta accedendo alla funzione. Si rimuove inoltre un thread dalla lista in quanto la wakeone viene utilizzata per svegliare uno dei thread in attesa nella coda a seguito del verificarsi della condizione desiderata.

Che differenza c'è in OS161 tra un indirizzo logico USER e uno KERNEL ? E' possibile che, in momenti diversi, un indirizzo logico KERNEL e un successivo indirizzo USER possano corrispondere allo stesso indirizzo fisico in RAM ? (rispondere, e motivare la risposta, sia per il caso dubmvm, versione base, che per il caso di modifica del kernel, tale da gestire la "restituzione" di memoria)

- Gli indirizzi logici user fanno riferimento agli indirizzi generati dal processore MIPS che vanno da 0x00000000 a 0x80000000, ovvero che fanno riferimento ai 2GB che prendono in nome di spazio Kuseg. Mentre gli indirizzi logici KERNEL vanno da 0x80000000 a 0xffffffff (kseg0/1/2). Mentre per quanto riguarda il caso base (dumbvm), a causa del fatto che la memoria non viene mai rilasciata non e' possibile che indirizzo logico KERNEL e USER corrispondano allo stesso indirizzo fisico in RAM. Nel secondo caso, se si usa un allocatore comune (tra vm e kmalloc) allora e' possibile che quando un processo utente viene rilasciato l'allocatore assegni ad un processo kernel lo

stesso indirizzo di memoria. Se si utilizzano allocatori separati questo non e' possibile.

Perché in OS161, per gestire gli argomenti al main, è necessario creare una copia di argv e argc ? Dove va creata tale copia ? Perché non sono sufficienti puntatori alle copie originali di argv e argc, create dal monitor di sistema, a partire da una stringa di comando ?

- E' necessario in quanto gli argomenti al main debbono essere in memoria user, affinché il programma utente possa farvi accesso. Siccome gli argomenti al main sono in origine in memoria kernel, si rende necessario farne una copia. La copia va fatta in memoria user, In particolare, in una parte accessibile dell'address space: la soluzione più semplice è l'inizio (indirizzi alti) dello (user) stack. Nello specifico viene usata copyout() per copiare dati dal kernel verso lo user space.
- Perché le copie originali argc e argv create dal monitor di sistema sono allocate in memoria kernel mentre il processo che li utilizzerà risiede in memoria user
Le copie sono create in memoria user
Perché i puntatori punterebbero alla memoria kernel e il processo non ha diritto di accesso

Si descrivano le strutture dati di un file system utilizzate dalle operazioni open e read/write, distinguendo tra strutture in memoria kernel, user e su memoria di massa (secondaria). Si metta in evidenza, in particolare la sequenza di operazioni che interessano l'accesso ai direttori, ai dati del file, alle tabelle di sistema interessate. Nel caso in cui un file sia condiviso e aperto contemporaneamente da due processi, è possibile che i due processi lo identifichino con due indici (file descriptor) diversi? (ad esempio, per un processo 5 e per un altro 8).

- Per realizzare una open() inizialmente si controlla se il file e' aperto guardando la system-wide table (kernel memory) . Se non e' presente si effettua una ricerca mediante struttura di direttori (implementate mediante liste che permettono di facilitare la ricerca su file system) del file. Una volta trovato il FCB (memorizzato

nella directory appunto) si copia quest'ultimo in una entry della system-wide table e si incrementa il file-open counter. Successivamente si aggiunge una entry nella per-process table del processo che ha chiamato la open(), con il puntatore alla system-wide. Infine, la open() restituisce l'indice della per-process table (file descriptor).

- Per realizzare la read e la write si accede esclusivamente alla per-process table per ricavare il puntatore alla system-wide (dove ho il FCB e quindi l'indirizzo del blocco di partenza dove ho il file) e la modalita' di accesso.
- Il file descriptor rappresenta l'indice della entry della per-process file table che punta al FCB del file di interesse nella system-wide table. Questo significa che 2 file distinti, visto che hanno per-process table differenti, possano avere file descriptor differenti per lo stesso file.

Si vuole realizzare un nuovo tipo di filesystem, nel quale è possibile avere sia file che direttori condivisi (l'albero di direttori è un DAG!). Perché eventuali cicli nel grafo dei direttori rappresentano un problema?

- Appunto perché con i cicli il grafo non è un DAG, cioè un direttorio avrebbe come discendente/successore nell'albero/DAG un suo antenato/predecessore! Questo va evitato.
- Quali strategie è possibile adottare per ovviare al problema (dei cicli)? a) Condividere solo file (le foglie) e non i direttori (ma questo limiterebbe la richiesta: si vogliono "sia file che direttori condivisi") b) Garbage collection: si accettano i cicli (per non fare controlli ad ogni operazione di link/condivisione di file/direttorio), ma periodicamente si fa eseguire sul file system una funzione di controllo che individua e rimuove eventuali cicli c) Ad ogni generazione di un nuovo link (a file/direttorio esistente) si fa un controllo di presenza di eventuali cicli, che determina la validità o meno dell'operazione.

Perché, in un contesto multi-core (sono presenti più CPU) non è possibile realizzare la mutua esclusione semplicemente disabilitando e riabilitando l'interrupt ?

- Perché l'interrupt verrebbe disabilitato sulla sola CPU corrente, questo non precluderebbe quindi l'esecuzione di altri thread o processi sulle altre CPU. ATTENZIONE: Non sarebbe neppure sufficiente estendere la disabilitazione degli interrupt alle altre CPU, in quanto su queste ci potrebbero già essere altri thread in esecuzione. La disabilitazione degli interrupt serve quindi solo nel caso di un single core, in quanto significa che il thread corrente è l'unico in esecuzione sul sistema.

A cosa servono le funzioni OS161 copyin e copyout?

- Servono a effettuare copie di dati tra memoria user e kernel, copyin ha come destinazione la memoria kernel, dualmente la copyout. La principale differenza rispetto ad altre forma di copia da memoria a memoria consiste nel fatto che vengono gestite in modo consistente eventuali errori/eccezioni legate a indirizzi/puntatori user non validi, impedendo così al kernel di terminare in modo anomalo (es. crash/panic).

Di ognuno questi indirizzi logici, si dica se è plausibile (in un sistema OS161 con processore MIPS) come indirizzo iniziale di uno switchframe o trapframe: A)

- 0x7FFFFFF0 (NO. E' un indirizzo logico USER) B)
0x80044DB8 (SI. E' un indirizzo logico kernel valido) C)
0x80056B0A (NO. Pur essendo un indirizzo kernel valido, non è multiplo di 4: per salvare registri serve un frame allineato a indirizzi multipli di 4).

```
load_elf(struct vnode *v, vaddr_t *entrypoint)
{
    Elf_Ehdr eh;    /* Executable header */
    int result;
    struct iovec iov;
    struct uio ku;
    /*
     * Read the executable header from offset 0 in the file.
     */
    result = VOP_READ(v, &eh, sizeof(eh));
    ...
}
```

Il programma riportato è errato. Si spieghi perché e si proponga la correzione necessaria.

La chiamata a VOP_READ è errata. La lettura va fatta con una strategia diversa: prima si definisce l'operazione da effettuare, mediante uio_kinit (per operazione in memoria kernel), usando le variabili ku e iov, poi si chiama VOP_READ, utilizzando ku. La versione corretta è (non è fondamentale l'ordine "esatto" dei parametri a uio_kinit):

```
uio_kinit(&iov, &ku, &eh, sizeof(eh), 0, UIO_READ);
result = VOP_READ(v, &ku);
```

Si dica poi (in breve) che cosa sono (e a cosa servono nel programma proposto):

- il parametro v: è il puntatore a vnode (il file control block) del file ELF da cui si legge
- la variabile ku: la struct nella quale va impostata l'operazione di IO (mediante uio_kinit) prima di effettuarla (mediante VOP_READ). Punta a iov (seguente), contiene campi per definire lettura/scrittura kernel/user, e altro
- la variabile iov; la struct in cui effettivamente vengono caricati indirizzo in memoria e dimensione, per la destinazione di una VOP_READ (oppure sorgente di una VOP_WRITE)

A) Spiegare brevemente la differenza tra switchframe e trapframe. Quale viene utilizzato per avviare un processo utente? Quale per gestire una chiamata di sistema? Quale per la thread_fork?

Entrambe le strutture vengono utilizzate per salvare il contesto del processo (registri + altre informazioni): lo switchframe viene utilizzato per il context switch (un cambio del processo in esecuzione su una cpu), il trapframe è correlato a una trap, il che significa che siamo ancora nel contesto del processo ma in modalità kernel. La funzione runprogram utilizza un trapframe per avviare un processo utente con mips_usermode. Una chiamata di sistema sfrutta il trapframe, poiché viene attivato come trap. La thread_fork usa uno switchframe in quanto prepara un thread pronto per essere inserito nella coda ready.

B) Lo switchframe è allocato nello stack utente o nello stack di processo a livello di kernel? (motivare)

Nello stack a livello di kernel, poiché non può essere visibile in modalità utente

C) Perché i dispositivi di IO sono mappati su kseg1? Sarebbe possibile mappare un dispositivo di IO in kseg0?

I dispositivi di IO devono essere mappati nello spazio del kernel. Sono mappati in kseg1 in quanto non è memorizzato nella cache: un dispositivo di IO non può essere letto / scritto utilizzando la cache, poiché qualsiasi operazione di lettura / scrittura deve essere eseguita sul dispositivo di IO. Per lo stesso motivo, il dispositivo non può essere mappato su kseg0, che è memorizzato nella cache.

D) Cosa succede se un processo utente effettua una write(fd, buf, nb), dove buf è un indirizzo in kseg0? Cosa dovrebbe fare la syscall SYS_write per gestire correttamente questo caso?

Il processo utente non deve poter leggere un indirizzo logico di tipo kernel. Il puntatore buf è l'origine dell'operazione di scrittura, mentre la destinazione è il file, che potrebbe essere un normale file o la console. In generale, qualsiasi indirizzo di memoria legale è corretto come sorgente, ma, dato un processo utente, un puntatore legale dovrebbe essere mappato nello spazio utente, quindi kseg0 è proibito. Per gestire correttamente questo evento, la syscall SYS_write dovrebbe controllare che il parametro buf sia un indirizzo user, tornando un errore in caso contrario.

B) Cosa sono i wait_channel? Per cosa sono utilizzati all'interno del sistema operativo?

Il Wait channel: è una struttura/primitiva di sincronizzazione del kernel, che consente operazioni di wait e signal (dette wchan_sleep, wchan_broadcast, wchan_wakeone), connesse a una condizione osservata, di tipo simile alla "condition variable". È stato introdotto in OS161 come primitiva di basso livello, associata a spinlock (mentre la condition variable è associata a lock). Il wchan può essere utilizzato solo dal kernel thread che ha acquisito lo spinlock

DOMANDE SU SYSCALL READ() IN OS161

A) Can the read destination be in kernel memory, or does it have to be (always) in user memory?

Quando si effettua una `read()` questo e' quello che accade:

- L'utente chiama la `read()`.
- A questo punto viene alzata l'eccezione trap che salva lo stato del processo corrente nella struttura `trapframe`, compresi i parametri passati alla `read()`. Il controllo passa al kernel che esegue la `syscall` corretta, in base al valore di `callno`.
- Se la lettura era su file, allora si ricava l'FCB a partire dal `fd` passato come parametro (`tp->a0`). Si richiama la funzione `uio_uinit` e si effettua la `VOP_READ`.
- I dati vengono messi in un buffer kernel per poi essere copiati (`copyout`) nel buffer in memoria utente specificato tra i parametri della `syscall` (`buf_ptr`).
- Alla fine la `system call` restituisce i dati letti.

La destinazione quindi (`buf_ptr`) deve essere necessariamente in user memory, in quanto la `system call` e' appunto per processi utente.

B) Suppose two user processes, running concurrently, open the same file for reading and call `read(fd,v,N)`, on that file, do you expect that (for each option answer YES/NO and motivate):

1) `fd` is the same (integer number) for both processes.

- No il file descriptor rappresenta la entry della per-process open-file table. Esse devono contenere un puntatore allo stesso FCB (`vnode`) se si tratta appunto dello stesso file, ma, essendo le tabelle indipendenti per i processi, il `fd` puo' essere diverso.

2) The two processes read the file independently from each other (or they share the same read flow/sequence?).

- Ogni `open()` crea una propria struttura interna (`file-description`) con il suo offset. Quindi il primo processo puo' essere al byte 100 e l'altro al 0, e muovono l'offset indipendentemente.

3) A lock is needed, because operations have to be done in mutual exclusion.

- No in quanto le letture non richiedono accesso mutuamente esclusivo.

4) If the implementation of the read system call uses a kernel buffer, the two processes will use the same buffer.

- Se il buffer e' allocato dinamicamente allora no. Se globale si, pero' e' necessaria sincronizzazione.

A) Consider the functions `thread_switch` and `thread_yield`. Are they equivalent, or do they perform different tasks? (motivate)

- La `thread_switch` si usa per il semplice context switching, mentre la `yield` per forzare il passaggio dall'esecuzione di un processo ad un altro (forza il context switching).

B) when a new kernel thread is created in OS161, does it run immediately or is enqueued for running?

Viene messo in coda dei processi pronti, in attesa di un `thread_switch`.

C) The implementation of `thread_exit()` ends with a call `switchframe_switch(&cur->t_context, &next->t_context);`

What do the two parameters represent (it is not enough to say "thread context")? Which of the two parameters refers to the saved context and which one to the restored context?

- Il contesto del current thread viene salvato nel primo parametro, mentre il contesto del nuovo thread e' letto (restored) dallo `switchframe` puntato dal secondo parametro.

Si consideri, in OS161, la realizzazione della gestione della memoria virtuale. Si risponda alle seguenti domande

A) Si supponga di voler realizzare in dumbvm la gestione dei frame liberi mediante free-list, invece che con bitmap (come fatto nella soluzione proposta per il lab 2).

A1) si dica se la free list può essere una lista non ordinata oppure se deve essere ordinata. (motivare)

A2) Qualora la lista fosse ordinata, andrebbe ordinata in base a indirizzi fisici oppure logici? (motivare)

B) Si supponga di voler invece realizzare una PAGE TABLE a livello di singolo processo. Viste le dimensioni ridotte della RAM fisica, si ipotizzi che una casella (entry) della PT occupi solo 2 Byte. Si ipotizzi di NON voler gestire un heap per il processo: il processo contiene quindi unicamente (come fatto in dumbvm) i segmenti di codice, di dati e lo stack. Si sa che:

- La costante PAGE_SIZE è definita come 4096
- Il segmento 1 (codice) inizia all'indirizzo logico 0x400000 e ha dimensione 10112 Byte.
- Il segmento 2 (dati) inizia all'indirizzo logico 0x412000 e ha dimensione 8028 Byte
- Lo stack ha dimensione 18 pagine, collocate ai massimi indirizzi logici possibili.

B1) Si calcoli la dimensione complessiva dell'address space e, al suo interno, il numero di pagine effettivamente usate (valide).

B2) si calcoli quale sarebbe la dimensione di una page table standard.

B3) si calcoli quale sarebbe la dimensione di una page table gerarchica a due livelli, supponendo di utilizzare 11 bit per p2: si ricorda che l'indirizzo logico è (p1,p2,d).

- A1) La free list può essere realizzata sia ordinata per indirizzi crescenti che non ordinata. Il fatto che sia ordinata permette di trovare locazioni contigue di memoria con complessità $O(N)$, mentre se non ordinata con complessità $O(N^2)$.
- A2) Se fosse ordinata andrebbe ordinata per indirizzi fisici, in quanto si sta facendo riferimento ai frame di memoria fisica.
- B)
 - Se lo stack è collocato ai massimi indirizzi logici possibili, allora arriva fino a 0x7FFFFFFF (i logici per i processi utente finiscono appunto a 0x80000000).
 - B1) Quindi la dimensione dell'address space va da 0x40000000 a 0x7fffffff per quelli validi, ma qui dice dimensione complessiva, che parte quindi da 0x0.
Allora va da 0x0 a 0x7fff... che è 2GB (in decimale, semplice conversione).
TRA QUESTE, il numero di pagine effettivamente usate fa riferimento agli indirizzi in cui ci sono segmento 1, segmento 2 e stack:
 - Segmento 1 $\Rightarrow 10112 \text{ B} \rightarrow 10112 \text{ B} / 4096 \text{ B} = 2,4 = 3$ pagine.
 - Segmento 2 $\Rightarrow 8028 \text{ B} \Rightarrow 8028 / 4096 = 2$ pagine.
 - Stack $\Rightarrow 18$ pagine.
 - B2) Una page table standard deve avere un numero di entry in modo da coprire lo spazio di indirizzamento.

Considerando lo spazio di 2 GB $\Rightarrow 2\text{KKKB}/4\text{KB} = 512\text{K}$ di blocchi. In totale quindi $512\text{K} * 2\text{B}$ (dimensione singola entry) = 1MB di Page table.

- B3) Se si vuole fare una page table gerarchica a 2 livelli con p2 di 11 bit:

- d rappresenta l'offset di pagina, considerando pagine da 4096B $\rightarrow \log_2(4096) = 12$.

Allora $p1 \rightarrow 32 - 11 + 12 = 9$ (si sa che si usa processore MIPS a 32 bit, per questo 32).

Allora si ha : $p1=9$, $p2=11$, $d = 12$.

Si avrebbe una dimensione di:

$(2^9 \text{ entry} * 2\text{B})$ per la outer table.

$(2^{11} \text{ entry} * 2\text{B})$ per la inner table.

Ora bisogna capire quante inner table servono.

`0x00400000 = 00000000 0,1000000 0000,0000 00000000`

`0x00412000 = 00000000 0,1000001 0010,0000 00000000`

Questi sono gli indirizzi di segmento 1 e 2 (i primi 9 bit sono a 0, quindi fanno riferimento alla medesima outer). Stack rientra all'interno di una singola inner ma fa parte di un altro indirizzi outer. Quindi in totale la dimensione è data da:

- Le outer + 2 inner.

In un sistema con tabella delle pagine invertita, una singola voce della tabella può essere condivisa da più pagine virtuali di processi diversi senza causare un conflitto nella traduzione degli indirizzi? (SÌ/NO, Spiega brevemente)

NO. La tabella delle pagine invertita è caratterizzata da un numero di entry pari al numero di frame della memoria primaria ed è condivisa tra tutti i processi. Questo significa che ogni entry ha il PID che identifica a quale processo quella determinata pagina appartiene, quindi una stessa entry non può essere condivisa tra più processi. Se invece si utilizzasse una tabella hash, a quel punto potrei avere una entry con diverse coppie (chaining) e devo effettuare il controllo sul numero di pagina per capire a quale frame in memoria mi sto riferendo.

In un sistema che utilizza la sostituzione locale delle pagine, un processo con un tasso molto elevato di page fault può comunque causare una riduzione della memoria fisica

disponibile per gli altri processi? (SÌ/NO, spiega/motiva brevemente)

No, in quanto con una strategia di local replacement, nel momento in cui un processo viene caricato in memoria gli vengono assegnate un certo numero di pagine e, se quando entra in esecuzione e necessita di ulteriore spazio, non può occupare frame riservati ad altri processi. Questa strategia permette di limitare il trashing, ma allo stesso tempo il throughput massimo possibile è ridotto rispetto ad un global replacement.

In un sistema di memoria virtuale con un grado fisso di multiprogrammazione, il thrashing può ancora verificarsi se la somma dei working set di tutti i processi è inferiore alla memoria fisica disponibile, ma l'algoritmo di sostituzione delle pagine non è ottimizzato? (SÌ/NO, spiega/motiva brevemente)

Sì, in quanto il trashing può comunque verificarsi se l'algoritmo non predice correttamente i working sets del processo.

In un sistema con più dispositivi di I/O e una sola CPU, le operazioni di I/O possono avvenire in parallelo mentre la CPU è occupata nell'esecuzione di un processo?

Sì in quanto posso avere controller di I/O come DMA che si occupano del trasferimento di dati in memoria senza intaccare la CPU.

In un sistema con memoria virtuale a richiesta (demand paging) e DMA, può verificarsi un page fault durante un trasferimento DMA, causando il fallimento del trasferimento se non vengono prese le dovute precauzioni?

Il page fault si verifica quando la CPU genera un indirizzo logico e la pagina a cui esso è associato non è presente in memoria RAM. Questo significa che i due concetti sono slegati e che quindi il DMA non ha a che fare con i page faults.

In un sistema che utilizza l'I/O gestito tramite interrupt, è possibile che un interrupt venga perso se il controller degli interrupt è occupato a gestire un altro interrupt e il

dispositivo che ha generato il secondo interrupt non supporta la messa in coda degli interrupt?

Si e' possibile che venga perso, in quanto il controller interrupt non soddisfa la richiesta la quale, non essendo stata posta in coda, viene persa.

La politica di sostituzione delle pagine Working Set è una politica a dimensione fissa?

La politica working set e' a dimensione variabile in quanto la dimensione del resident set varia sulla base alle pagine incluse nella working set window.

Un processo user vuole effettuare un input da un dispositivo di IO a caratteri (character device) mediante una strategia basata su polling del dispositivo. Per evitare un'attesa troppo lunga (nel loop di polling) da parte del driver del dispositivo, si chiede all'autore del programma eseguito dal processo user, di fare direttamente polling mediante un loop di lettura del registro di stato del dispositivo.

- A) E' possibile effettuare questa operazione? (se si, dire come, se no, dire perché)
- B) Supponendo che il dispositivo sia associato alla tastiera e lo si gestisca in interrupt, ci si aspetta che venga generato un interrupt per ogni carattere ricevuto oppure un interrupt per ogni "invio" (cioè enter, fine-riga)? (motivare)

Dato un altro dispositivo di IO, gestito a blocchi:

- C) E' possibile che una sola scrittura (mediante chiamata a write ()), tenti di fare output di più di una coppia puntatore/dimensione?
- D) Dato il driver che realizza la scrittura, supponendo che il sistema sia dotato di DMA controller, è possibile che venga eseguita "prima" una operazione di lettura e/o scrittura attivata successivamente a questa, da un altro processo? E (stessa domanda) dallo stesso processo? (motivare)

A) Non e' possibile in quanto il programma non puo' prendere il controllo su come la CPU deve effettuare il polling.

B) L'interrupt deve essere generato per ogni carattere, in quanto altrimenti avrei bisogno di una struttura dati ausiliaria per il buffering dei dati.

C) Con coppia puntatore/dimensione si intende buffer/dimensione:

- `sys_write(int fd, userptr_t buf_ptr, size_t size).`

Quindi no non e' possibile in quanto la write riceve solo una coppia (puntatore, dimensione).

D) Sì, se l'altro processo ha priorità maggiore, oppure se l'altra scrittura fa riferimento ad un blocco su disco tale per cui gli algoritmi di scheduling del HDD decidono di eseguire prima l'altra operazione.

Si consideri in OS161 una possibile implementazione delle system call relative al file system, basate su una tabella di processo e su una di sistema, definite come segue

```
/* system open file table */
struct openfile {
    struct vnode *vn;
    off_t offset;
    unsigned int countRef;
};
/* this is a global variable */
struct openfile systemFileTable[SYSTEM_OPEN_MAX];
...
/* user open file table: this a field of struct proc */
struct openfile *fileTable[OPEN_MAX];
```

Si risponda alle domande seguenti (motivando le risposte):

A) Perché systemFileTable è una variabile globale, mentre fileTable è un campo di struct proc ?

B) Si supponga che, dati due processi user P1 e P2 (con p1 e p2 puntatori alle rispettive struct proc) siano vere le seguenti uguaglianze:

- (p1->fileTable[5] == &systemFileTable[20])
- (p2->fileTable[8] == &systemFileTable[20])

a quale/i file descriptor (fd) corrisponde/corrispondono le parti di tabella visualizzate? 5, 8, 20? (motivare)

Se P1 e P2 fanno solo letture sul/sui file di cui sopra, le letture sono indipendenti oppure c'è interazione tra i processi?

C) Si supponga di aver implementato il supporto per la system call `SYS_lseek`. Si riporta la descrizione delle funzione `lseek`:

```
off_t lseek(int fd, off_t offset, int whence);
```

lseek() repositions the file offset of the open file description associated with the file

9/22 3:37 PM

Pagina 8 d

descriptor fd to the argument offset according to the directive whence as follows:

SEEK_SET

The file offset is set to offset bytes.

SEEK_CUR

The file offset is set to its current location plus offset bytes.

SEEK_END

The file offset is set to the size of the file plus offset bytes.

Una chiamata a `lseek` effettuata da P1 (si veda domanda B) influenzerà la successiva read effettuata da P2:

1) sempre ?

2) mai ?

2) solo se il parametro offset è `SEEK_CUR` ?

(sono possibili selezioni multiple, per ogni selezione occorre motivare la risposta)

A) `systemFileTable` è globale in quanto rappresenta la system wide open file table, che contiene l'elenco dei file aperti (elenco dei FCB) e per ognuno di essi il numero di processi che hanno aperto e stanno usando quel determinato file. Quindi questa struttura è condivisa e deve essere accessibile a tutti. Mentre la `fileTable` è un campo di `proc` in quanto rappresenta la per-process file table, con l'elenco dei file aperti (come puntatori ad una riga della `systemFileTable`) solo dal singolo processo.

B) I file descriptor sono gli indici della per-process file table, in particolare per `p1` il `fd = 5`, mentre per `p2 = 8`, ma il file al quale puntano è il medesimo. Le letture sullo stesso file NON sono indipendenti in quanto, man mano che `p1` legge ad esempio, modifica il campo `offset` della struct `openfile`. Entrambi hanno un riferimento allo stesso, quindi

bisogna fare attenzione ed eventualmente gestire una sincronizzazione.

C) Sempre in quanto l'offset e' il medesimo appunto.

Un processo user vuole effettuare un input da un dispositivo di IO a caratteri (character device) mediante una strategia basata su polling del dispositivo. Per evitare un'attesa troppo lunga (nel loop di polling) da parte del driver del dispositivo, si chiede all'autore del programma eseguito dal processo user, di fare direttamente polling mediante un loop di lettura del registro di stato del dispositivo.

A) E' possibile effettuare questa operazione? (se si, dire come, se no, dire perché)

B) Supponendo che il dispositivo sia associato alla tastiera e lo si gestisca in interrupt, ci si aspetta che venga generato un interrupt per ogni carattere ricevuto oppure un interrupt per ogni "invio" (cioè enter, fine-riga)? (motivare)

Dato un altro dispositivo di IO, gestito a blocchi:

C) E' possibile che una sola lettura (mediante `read()`), tenti di acquisire un vettore di dimensione più grande di un blocco?

D) Dato il driver che realizza la lettura, tale operazione sarà realizzata mediante DMA (qualora possibile) oppure no? Perché?

A) No non e' possibile, in quanto il programma non puo' decidere il modo in cui la CPU effettua il polling.

B) L'interrupt viene generato ad ogni carattere, altrimenti servirebbe una struttura dati aggiuntiva gestita necessariamente a livello software che funga da buffer.

C) La chiamata `read` e' fatta in questo modo:

```
- int read(int fd, userptr_t buf_ptr, size_t size);
```

Quindi si e' possibile, la `read` serve per leggere su file e non ci sono limiti relativi alla lettura di un singolo blocco o simili.

D) Il DMA (Direct Memory Access) e' un tipo di controller di I/O che permette di trasferire blocchi dato dal dispositivo I/O alla memoria primaria senza intaccare troppo l'operato della CPU. Questo viene fatto mediante il cycle stealing, che si basa sull'effettuare trasferimenti sul bus di sistema, bloccando l'accesso della CPU a quest'ultimo in questi momenti. La CPU risulta essere poco intaccata se deve fare operazioni sulla cache locale o sui registri.

In generale quindi, si e' possibile che tale operazione venga realizzata da un DMA controller.

- A) Spiegare brevemente la differenza tra switchframe e trapframe. Quale viene utilizzato per avviare un processo utente? Quale per gestire una chiamata di sistema? Quale per la thread_fork?
- B) Lo switchframe è allocato nello stack utente o nello stack di processo a livello di kernel? (motivare)
- C) Perché i dispositivi di IO sono mappati su kseg1? Sarebbe possibile mappare un dispositivo di IO in kseg0?
- D) Un processo utente può eseguire una write(fd, buf, nb), dove buf è un indirizzo in kseg0?

A) Lo switchframe è una struttura dati che contiene il contesto (lo stato dei registri) di un processo/thread in seguito ad un context switching. Mentre il trapframe viene utilizzato per memorizzare lo stato dei registri a seguito di un'eccezione causata dal passaggio del controllo da modalità utente a kernel (o viceversa), ad esempio in caso di page fault (si genera TRAP) oppure a seguito di una system call. Il trapframe viene utilizzato per avviare un processo utente in modo da permettere il cambio di privilegi da kernel ad utente del thread creato. Mentre nella thread_fork() si utilizza lo switchframe, in particolare si utilizza switchframe_init() in modo da copiare nello switchframe il valore dei registri per il context switching per poi mettere il nuovo thread in ready queue con thread_make_runnable().

B) Viene allocato nello stack a livello kernel, perché non deve essere accessibile all'utente.

C) Un dispositivo I/O viene mappato in KSEG1 in quanto è una zona di memoria senza caching, in KSEG0 si usufruisce di un sistema di cache quindi non sarebbe possibile mappare un dispositivo di I/O in quest'area.

D) No, in quanto la write ha questa firma:

```
- int write(int fd, userptr_t buf, size_t size);
```

In particolare, l'origine dei dati che devono essere scritti su console o su file, sono nello spazio utente (in quanto è un processo in questa modalità ad averla chiamata), quindi no.

